

VHDL | FPGA synthesis

Environment

Nous utiliserons la suite logicielle **Vivado** de **Xilinx** (now AMD) qui regroupe sous une même interface graphique un ensemble intégrant les outils de compilation, élaboration, synthèse et simulation de code HDL. A noter également la présence de l'outil **Vitis_HLS** qui permet l'exploration architecturale à partir de code C/C++ En complément, nous disposons également du simulateur **QuestaSim** (formerly known as **modelsim**).

Enfin, accessibles à tous sans licence, les outils **ghdl** et **gtkwave** vous permettent vos premiers pas dans l'univers du VHDL.

Quelques fonctions de conversions :

- `conv_std_logic_vector (integer, size) ⇒ std_logic_vector` use ieee.std_logic_arith.all
- `conv_integer (std_logic_vector) ⇒ integer` use ieee.std_logic_unsigned.all
- `to_stdlogicvector (bit_vector) ⇒ std_logic_vector` use ieee.std_logic_1164.all
- `F <= std_logic(signal A or signal B) ⇒ std_logic` conversion en std_logic

Design Flow

- **Compilation** des entités, architectures, [configurations] et d'une architecture de test.
- **Élaboration** de l'architecture de test précédemment compilée générant un *snapshot*.
- **Simulation** du *snapshot*.

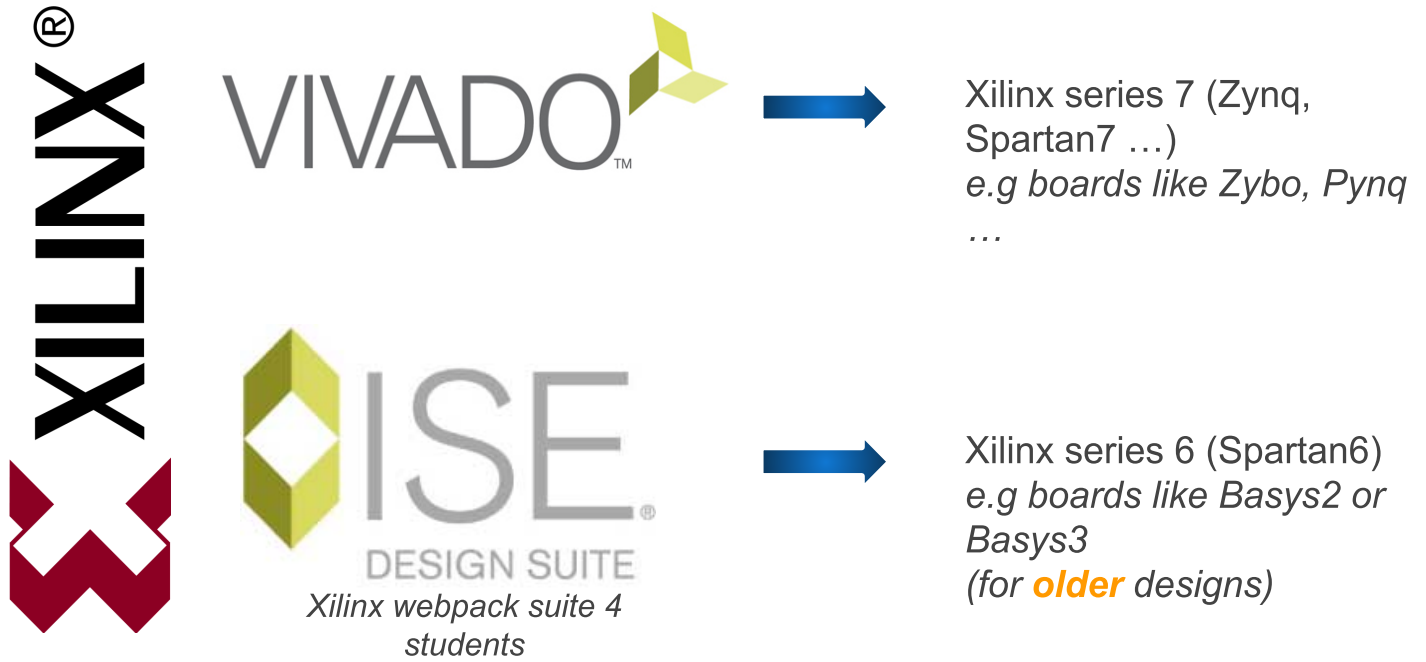
GHDL & GTKwave for behavioural simulation

```
* Compilation
ghdl -a --ieee=synopsys -fexplicit <packages.vhd> <components.vhd> <testbench.vhd>

* Elaborate
ghdl -e --ieee=synopsys -fexplicit testbench

* Run simulation
ghdl -r --ieee=synopsys -fexplicit testbench --wave=testbench.ghw

* View results
gtkwave testbench.ghw
```



Links

Master Secil/Siame VHDL

<https://moodle.univ-tlse3.fr/mod/page/view.php?id=456296>

(you'll find some useful links intended to speed up your learning curve)

VHDL course

https://secil.univ-tlse3.fr/teaching/francois/UE-VHDL/VHDL_course.pdf

VHDL 4 FPGA synthesis practical exercises (this file)

https://secil.univ-tlse3.fr/teaching/francois/UE-VHDL/VHDL4synthesis_practical_exercises.pdf

Table of contents

Environment.....	1
GHDL & GTKwave for behavioural simulation.....	1
Links.....	2
Xilinx Vivado setup.....	6
additional IP blocks.....	6
Practical exercises base files.....	8
TP1 - Pulse generator.....	9
project setup.....	9
pulse_gen design.....	13
behavioural simulation.....	13
pulse_gen generic parameter(s).....	15
design synthesis.....	16
post-synthesis simulation.....	16
TP2 - Pulse generator synthesis.....	17
top-level synthesis and generic parameters.....	17
comment-out generic map.....	18
post-synthesis timing simulation.....	19
pulse_gen with synchronous RST.....	19
TP3 - Pulse generator constraints.....	20
Timing constraints.....	20
pulse_gen unconstrained paths.....	21
Synthesis Constraints Wizard.....	22
Implementation Constraints Wizard.....	23
Full timings simulation.....	24
[master2] Bitstream generation and hardware download.....	24
TP4 - Synthesizable Log2 function.....	25
log2 @ cpu_package.....	25
log2 hardware component.....	25
[master2] optimized log2 component.....	27
TP5 - Registers bank for Risc processor.....	28
generic parameters.....	28
Behavioural simulation.....	28
Registers Bypass design and simulation.....	29
Synthesis with constraints.....	29
[master2] BRAM inference and maximum reachable frequency.....	30
TP6 - Memory bank for RISC processor.....	31
Simulation.....	31
[master2] bitstream & JTAG debug.....	33
TP7 - dual ports FIFO.....	34
Behavioural simulation.....	35
Synthesis.....	35
TP8 - BLDC controller.....	36
Speed control.....	38
BLDC controller.....	38
Use case.....	39

Links.....	39
[M2] Zybo Z7-20 board.....	41
Links.....	42
[M2] PWM.....	43
Behavioural simulation.....	43
Synthesis and Implementation.....	44
Post-synthesis simulation.....	44
IO planning (implementation).....	44
Bitstream generation.....	47
Hardware Manager.....	47
[optional] Bitstream readback.....	47
Download bitstream.....	48
Tests 😊.....	48
[M2] my AXI-enabled PWM IP.....	49
Links.....	51
Editing an IP from main project.....	52
IP customization.....	53
[optional] AXI simulation with the AXI VIP.....	57
Block design.....	58
Bitstream generation and export hardware.....	60
Launch Vitis IDE.....	62
Compilation.....	67
Zybo board JTAG mode.....	67
[M2] AXI IP encoder.....	69
[M2] PicoRV32 @ Zybo.....	70
[M2] GoWIN "Hello World" SystemVerilog.....	71
Required hardware.....	71
Physical Constraints.....	75
Synthesis.....	77
Programmer.....	77
[TODO] Hardware debugging.....	78
[M2] GoWIN 7 segments display.....	79
Required hardware.....	79
EMPU+GPIO FPGA part.....	80
Physical Constraints.....	82
Timing Constraints.....	84
Define used clock.....	84
Place 'n route Clock Summary:.....	85
Programmer.....	85
EMPU+GPIO MCU part.....	86
Create and configure MCU project.....	86
Add device specific libraries.....	88
Cross-ARM toolchain configuring.....	89
Add source files.....	92
Programmer.....	93
Debugging.....	93
[M2] GoWIN 7 segments display / part2.....	94

.....	95
Tp1-1 : Banc Registres double port lecture pour μ P RISC.....	96
Tp1-2 : Simulation Banc Registres.....	96
Tp1-3 : Banc Registres générique.....	99
Tp1-4 : Bypass D $\rightarrow$ Q.....	99
Tp2-1 : Banc Mémoire pour μ P RISC.....	104
Tp2-2 : Simulation Banc Mémoire.....	105
Tp3-1 : Additionneur Carry Look-Ahead.....	108
Tp3-2 : Simulation Adder_CLA.....	108
Tp4-1 : Unité Arithmétique Logique (ALU).....	112
Tp4-2 : Simulation ALU.....	113
Tp5-1 : Banc Mémoire pour μ P RISC.....	116
Tp5-2 : Simulation Banc Mémoire.....	117
Tp6-1 : DATAPATH.....	120
Tp6-2 : Simulation Processeur RISC.....	120
Tp7-1 : Unité de Contrôle.....	122
Tp8-1 : FIFO double port.....	124
Tp8-2 : Simulation FIFO double port.....	125
Tp9-1 : Mémoire Statique Synchrone.....	128
Tp9-2 : Banc mémoire pour processeur.....	130

Xilinx Vivado setup

Vivado is part of the '**Hardware targets**' tools from Xilinx ecosystem. It enables you to design FPGA's content through its UI, to synthesise and to simulate them.

*Note: it's also possible to simulate your design with external tools like **modelsim** ... some additional setup and extra care are required.*

According to the envisioned targets, there exists two flavours:

- Xilinx ISE ⇒ for series 6 and older FPGA
- Xilinx Vivado ⇒ for series 7 and newer boards

We'll make use of the **Xilinx Vivado** tools in combination with our **Zybo**, **Zybo-z7** and **Pynq** boards. For more advanced designs, for example those that make use of the embedded ARM processor (Hard IP¹), you'll need to have access to Intellectual Property blocks coming both from Xilinx and third-party designers.

Regarding the boards support files, extra care has already been undertaken leading to a seamless access to the boards definitions themselves. Henceforth, you'll just need to select the proper board from the Vivado's UI.

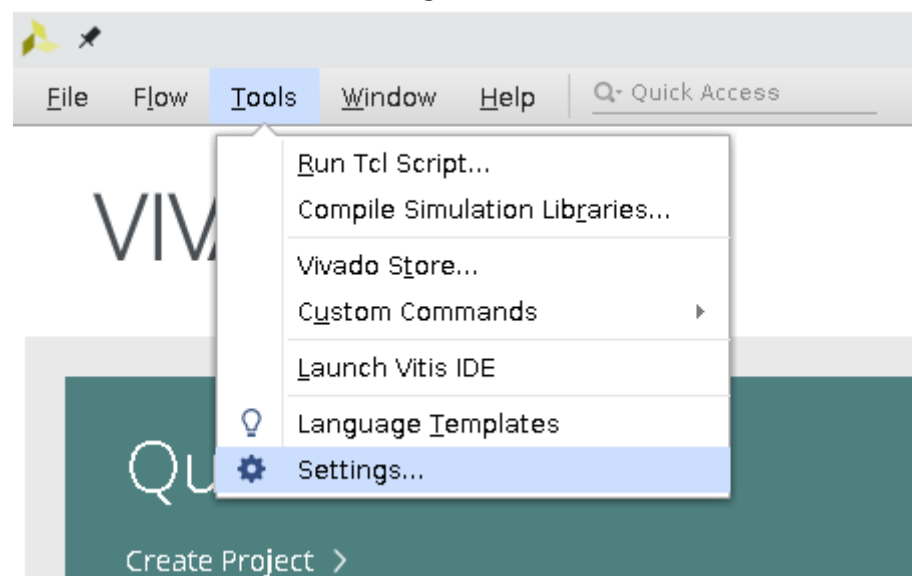
additional IP blocks

Zybo boards (and derivatives) from **Digilent** manufacturer makes use of specific IPs blocks. Those IPs have already been copied within the Xilinx tools repository, but you need to register them on a per user basis.

Launch the tools suite

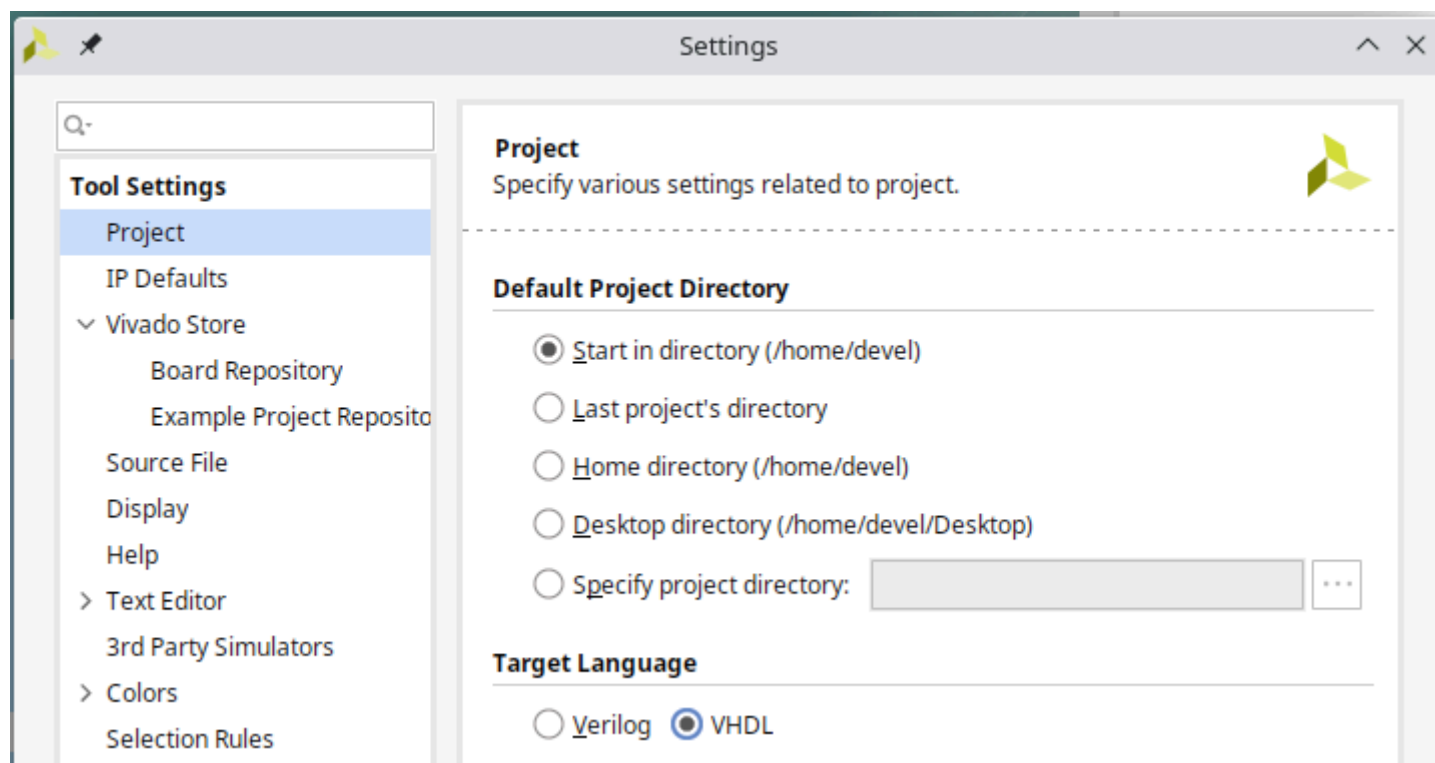
vivado

From main UI → *Tools* → *Settings*

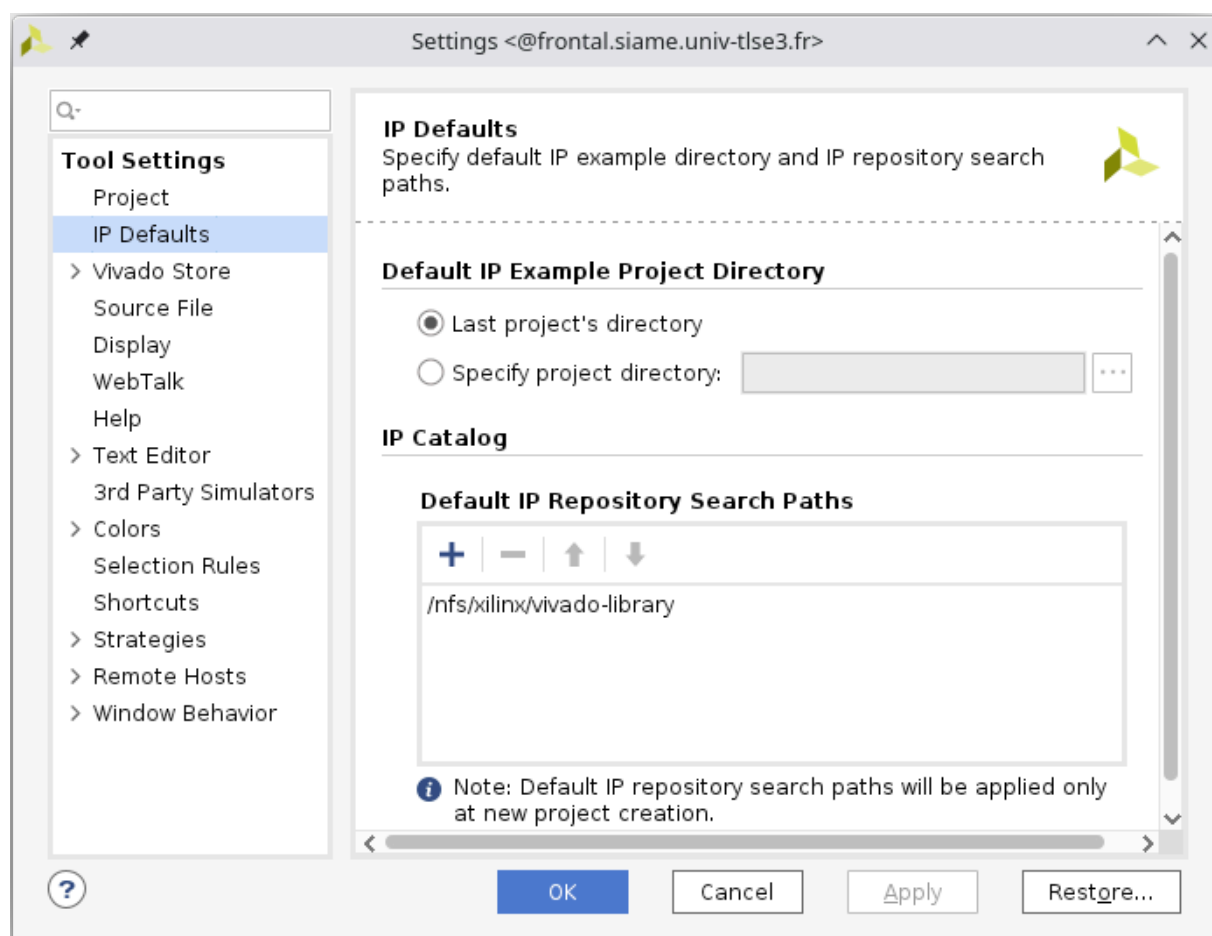


Project → *Target Language* → **VHDL**

¹ IP stands for Intellectual Property. ARM processors in Zynq devices are **Hard IPs** while others will be **Soft IPs**.



IP Defaults → IP catalog → add vivado-library path



Practical exercises base files

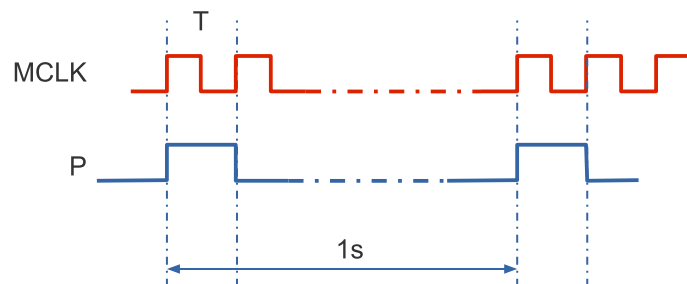
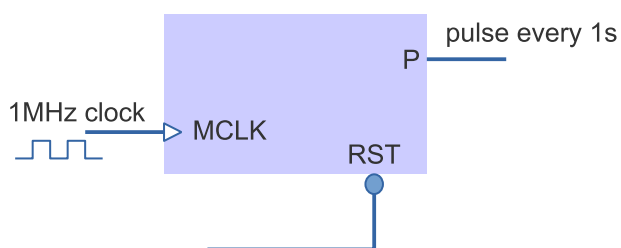
In order to ease things a bit, you can start from files available at

[M1] <https://secil.univ-tlse3.fr/teaching/francois/UE-VHDL/M1/>

[M2] <https://secil.univ-tlse3.fr/teaching/francois/UE-VHDL/M2/>

TP1 - Pulse generator

This first exercise will enable you to undertake a **behavioural** simulation of a simple component: a **pulse generator**.



As a first step, retrieve the following files

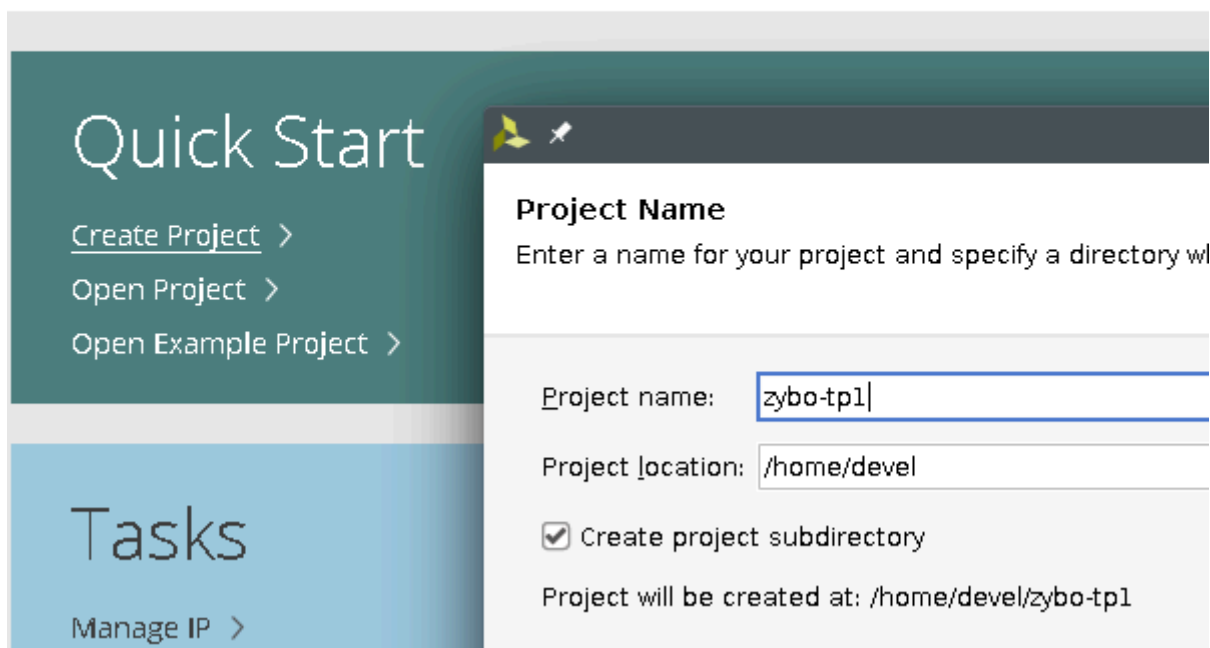
https://secil.univ-tlse3.fr/teaching/francois/UE-VHDL/M1/pulse_gen.vhd
https://secil.univ-tlse3.fr/teaching/francois/UE-VHDL/M1/test_pulse_gen.vhd

project setup

Launch the Vivado tools suite

vivado

You create a first project named 'zybo-tp1'



This is a RTL project but we'll add files later



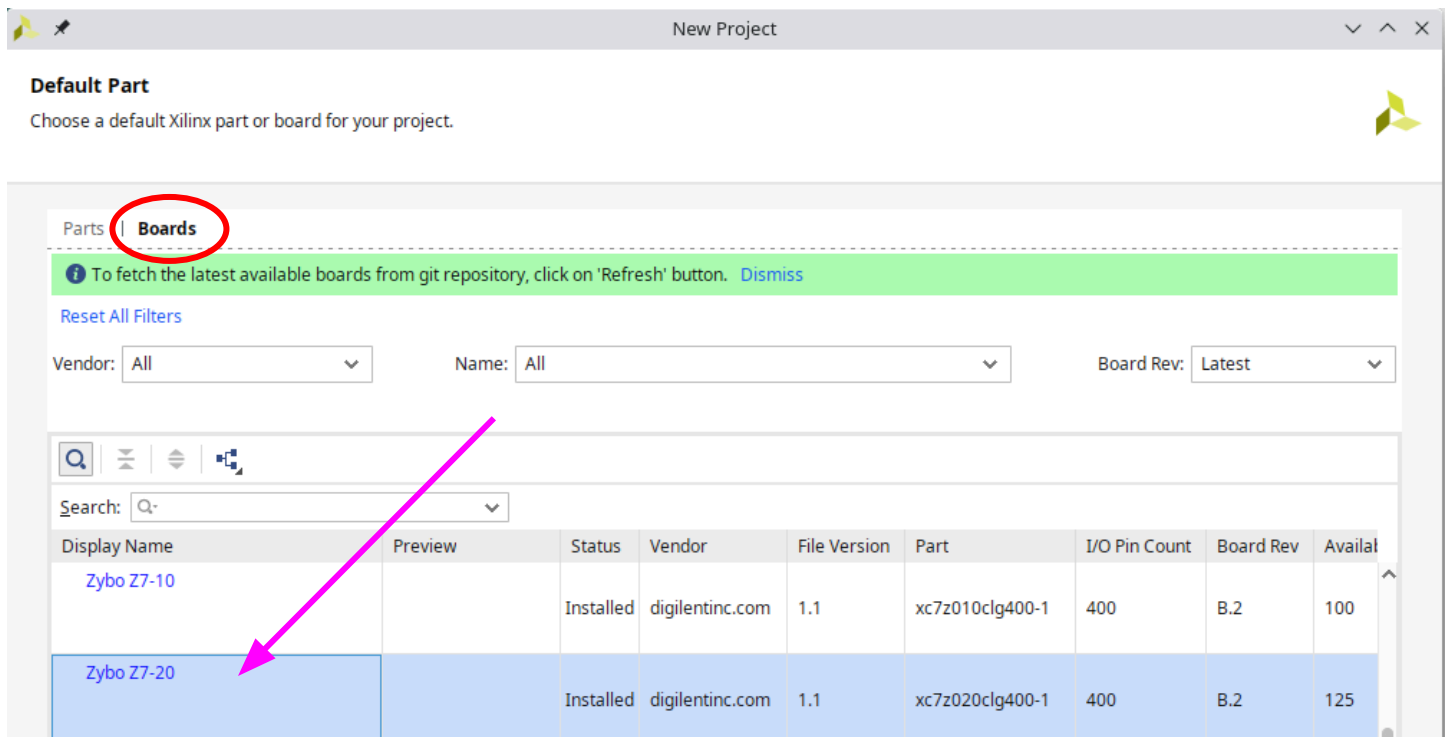
Project Type
Specify the type of project to create.

☒ **RTL Project**
You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL analysis, synthesis, implementation, design planning and analysis.

☒ Do not specify sources at this time

☐ Project is an extensible Vitis platform

... click on boards and select Zybo-Z7-20



Default Part
Choose a default Xilinx part or board for your project.

Parts | **Boards**

To fetch the latest available boards from git repository, click on 'Refresh' button. [Dismiss](#)

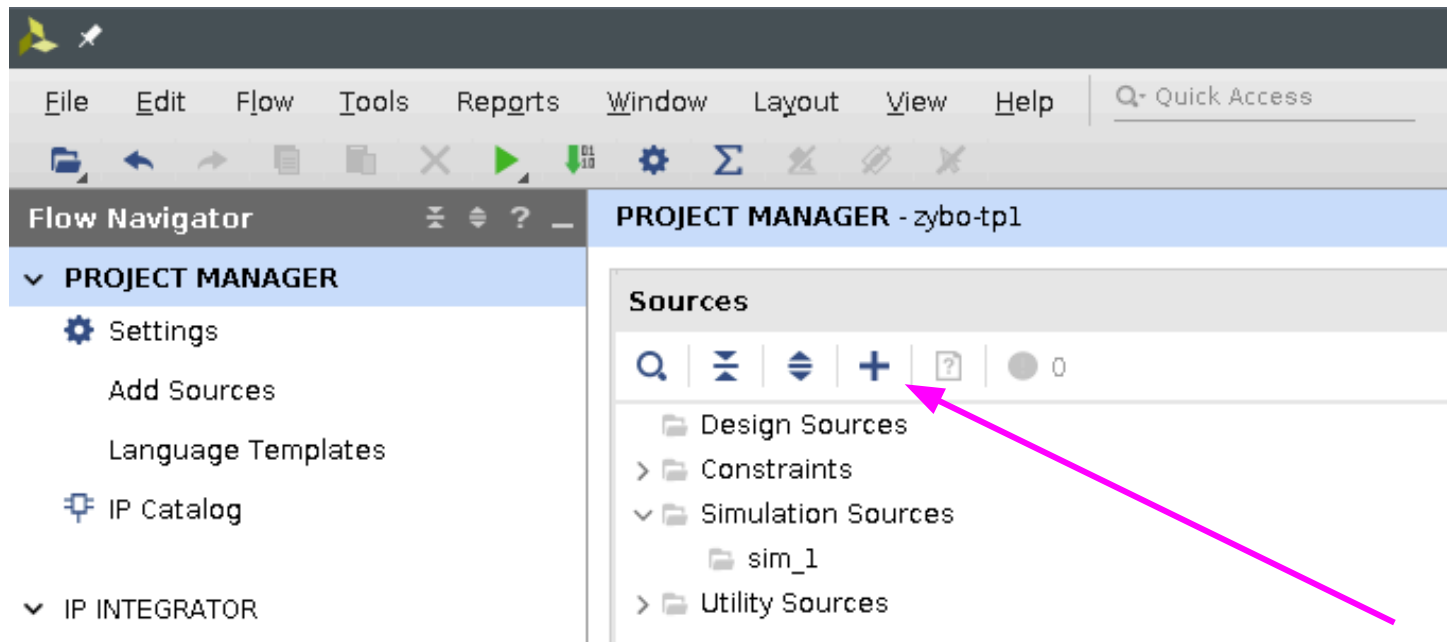
[Reset All Filters](#)

Vendor: Name: Board Rev:

Search:

Display Name	Preview	Status	Vendor	File Version	Part	I/O Pin Count	Board Rev	Availa
Zybo Z7-10		Installed	digilentinc.com	1.1	xc7z010clg400-1	400	B.2	100
Zybo Z7-20		Installed	digilentinc.com	1.1	xc7z020clg400-1	400	B.2	125

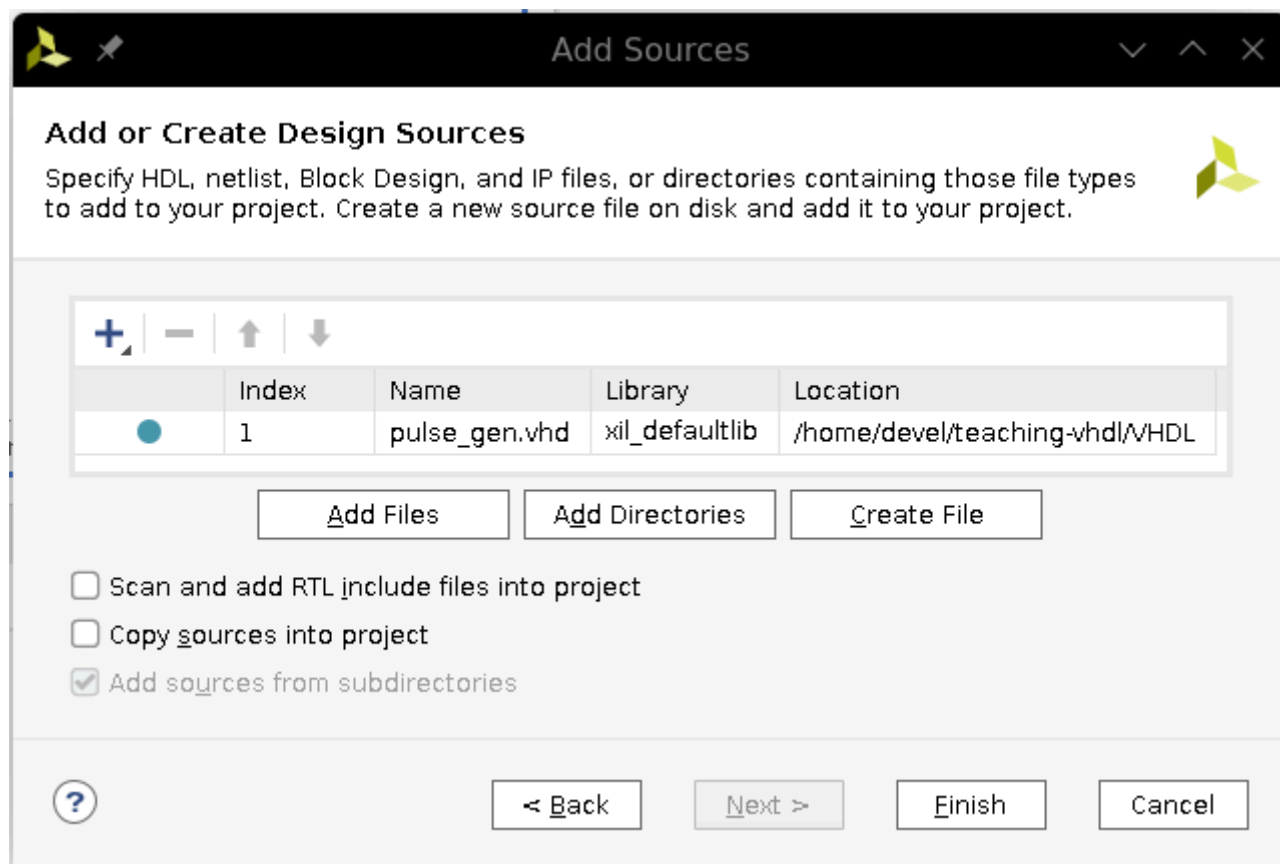
The project is now initialised, the main UI will appear and we'll start to add files



We'll start to add 'design sources' files



select 'pulse_gen.vhd' with 'copy sources into project' option **disabled**

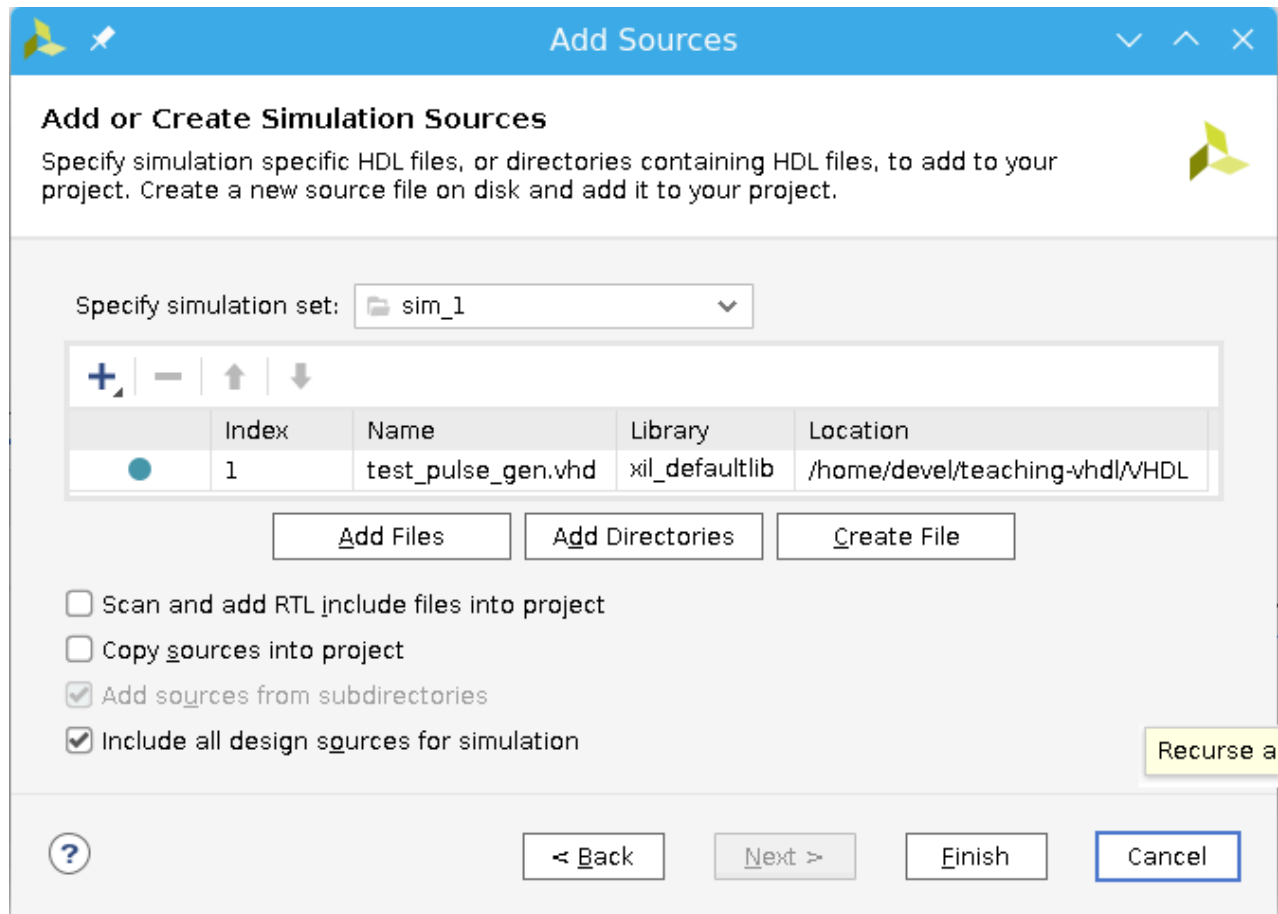


there are some errors ... quite normal since it's not finished

Now we'll add a simulation file in order to test our future pulse generator. Click on 'add file' in the project navigation bar and select 'simulation sources':



select 'test_pulse_gen.vhd' along with 'copy sources into project' option **disabled**

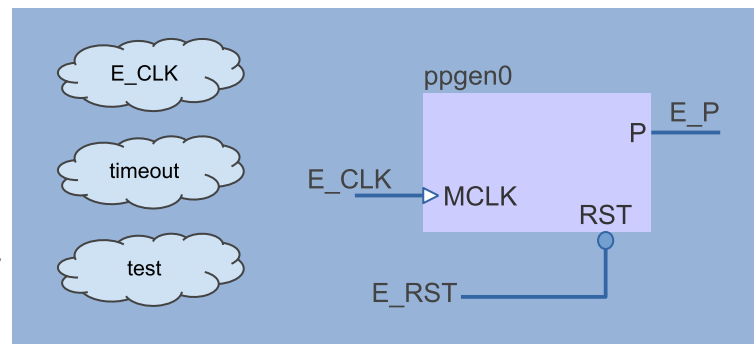


pulse_gen design

Our pulse generator will feature a **1MHz input clock**, an **asynchronous active low RST** line and will deliver a 1s output pulse. To achieve this, you need to complete the 'pulse_gen.vhd' file.

The test bench file 'test_pulse_gen.vhd' is a **test architecture** (i.e without I/O) that will define both a 1MHz internal clock, a timeout process along with another process that will feed signals on your pulse_gen component.

*Note: in order to clarify the simulation results, you'll set a pulse output every **ten** MCLK's rising edge.*



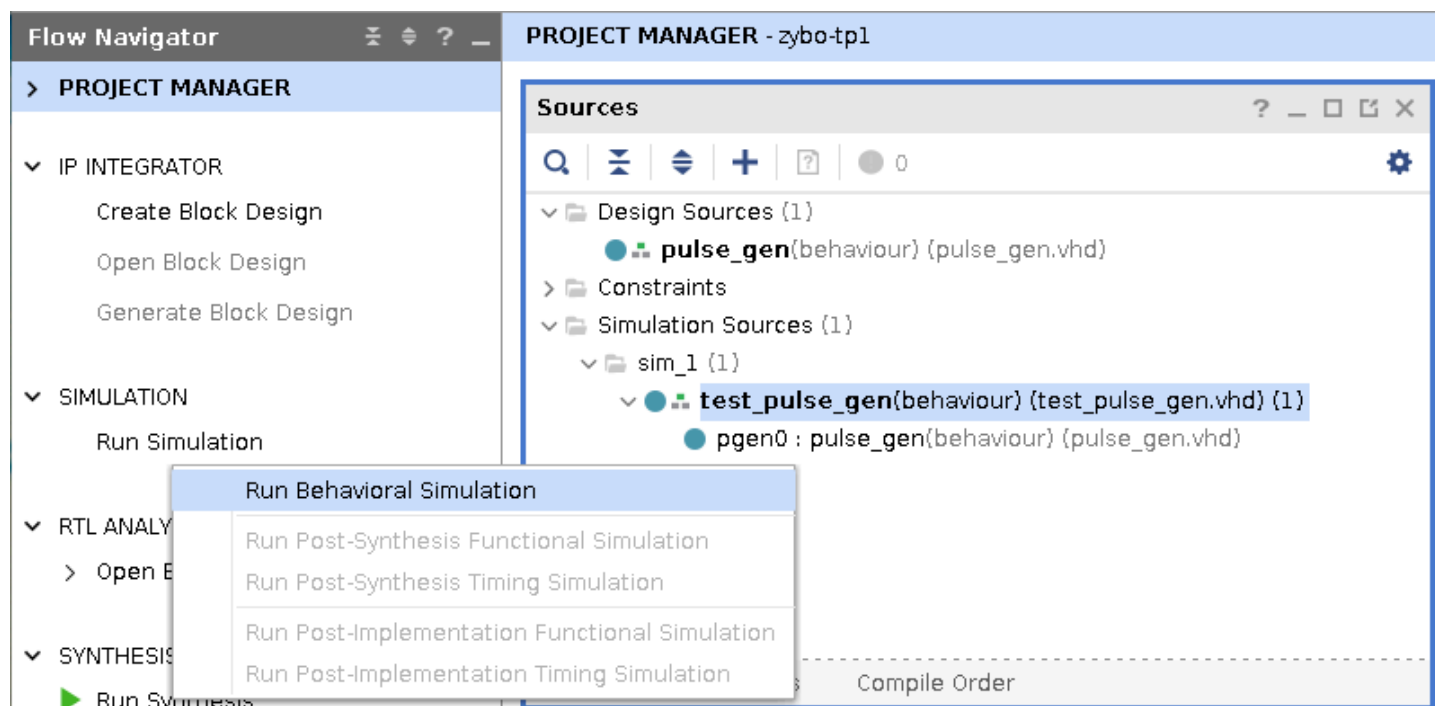
behavioural simulation

After having completed the 'pulse_gen.vhd' file, it's now time to achieve your first simulation. It is worth mentioning that there exists 3 levels of simulation:

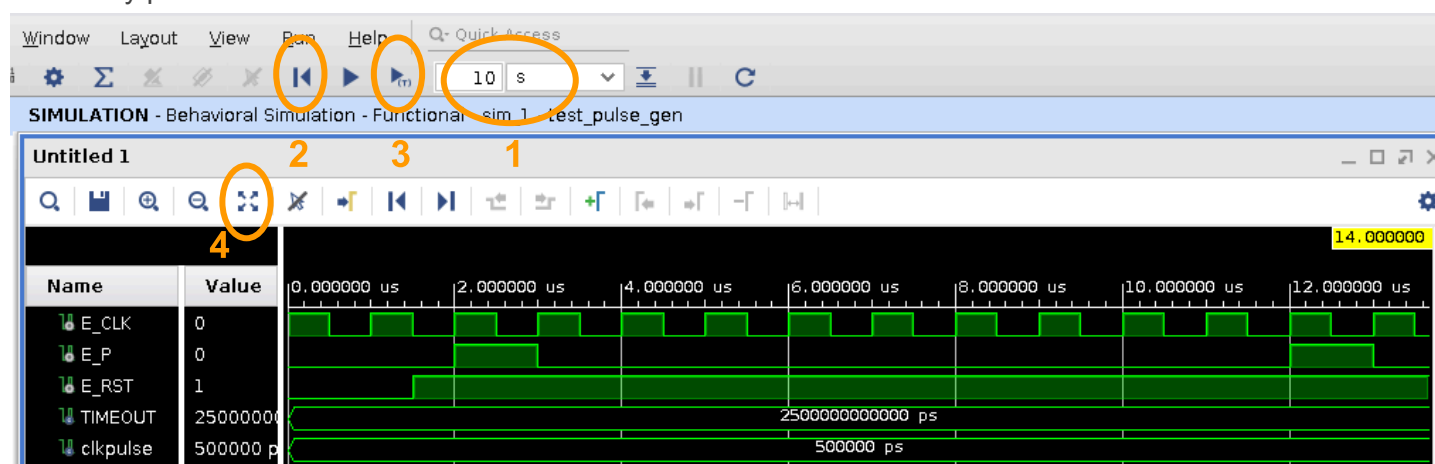
- **behavioural** → to test functionalities of your design,
- **post-synthesis** → inner components of your design are mapped on hardware resources, now you'll have propagation delays,
- **post-implementation** (i.e place and route) → post-synthesis + routing delays like in real life.

Hence we'll start with the first level of simulation: **behavioural**

On the vertical side panel, click 'Run Simulation' and select 'Run Behavioural Simulation'



You'll need to specify a **10s simulation time**, **restart simulation button** then **run for specified time button** and finally press **zoom fit** button.



Now ... Why did you get such an '**ASSERTION ERROR**' in your test bench ??



>>> WELL DONE, you successfully ran your simulation test bench <<<

pulse_gen generic parameter(s)

You'll now add the **MAX_CPT** generic parameter to your `pulse_gen` component. It's default value will be set to **1E06**.

Update `pulse_gen` architecture.

Update `test_pulse_gen.vhd` to apply **MAX_CPT=10** at the `ppgen0` instantiation, then **restart the behavioural simulation**.

```
entity pulse_gen is
  generic (
    MAX_CPT : natural := 1E06
  );
  port (
    RST, MCLK: in std_logic;
    P : out std_logic
  );
end pulse_gen;
```

design synthesis

Well now synthesise our design $\Rightarrow$ Vivado will instantiate hardware elements like flip-flops, logic cells etc to implement your design.

In the 'Design Runs' tab (lowest part of UI), click the green arrow and click OK not modifying default synthesis parameters.

Tcl ConsoleMessagesLogReportsDesign Runs x

Q

≡

⌵

⏪

⏩

▶

⏴

⏵

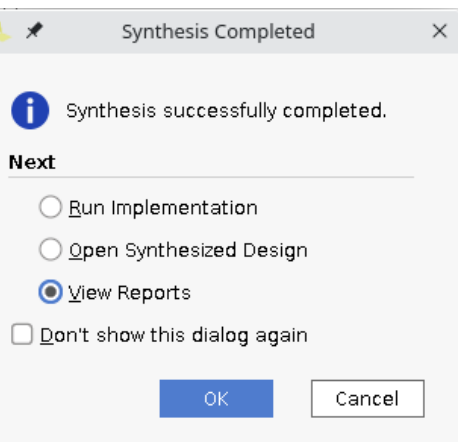
+

%

Name	Constraints	Status	WNS	TNS	WHS	THS
▼ ▶ synth_1	constrs_1	Not started				
▶ impl_1	constrs_1	Not started				

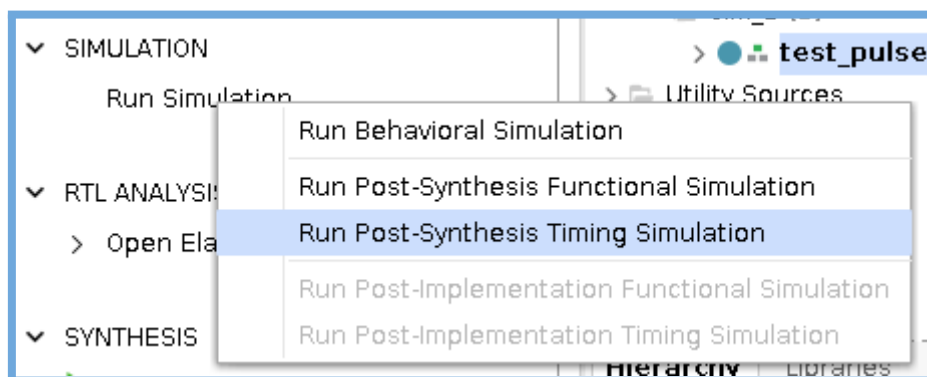
After your design has been successfully synthesised, select the 'View Reports' option.

Site Type	Used	Fixed	Prohibited	Available	Util%
Slice LUTs*	28	0	0	17600	0.16
LUT as Logic	28	0	0	17600	0.16
LUT as Memory	0	0	0	6000	0.00
Slice Registers	21	0	0	35200	0.06
Register as Flip Flop	21	0	0	35200	0.06
Register as Latch	0	0	0	35200	0.00
F7 Muxes	0	0	0	8800	0.00
F8 Muxes	0	0	0	4400	0.00



post-synthesis simulation

Now having your design synthesised, click on the 'Run Simulation' on the side panel and choose post-synthesis Timing Simulation



>>> What's happening ? What do you conclude ?? <<<

TP2 - Pulse generator synthesis

In the previous practical exercises [TP1 - Pulse generator](#), we've not been able to simulate the synthesised design ... because synthesis applied **default generic values** and thus removed the generic parameter !

Hence, we need to manage two issues:

- having our `pulse_gen` component synthesised with the requested generic value,
- remove **generic map** from `test_pulse_gen.vhd` since our **synthesised component** does not exhibit generic parameters

Note: it is worth mentioning that this issue only occurs with top-level designs having generic parameters.

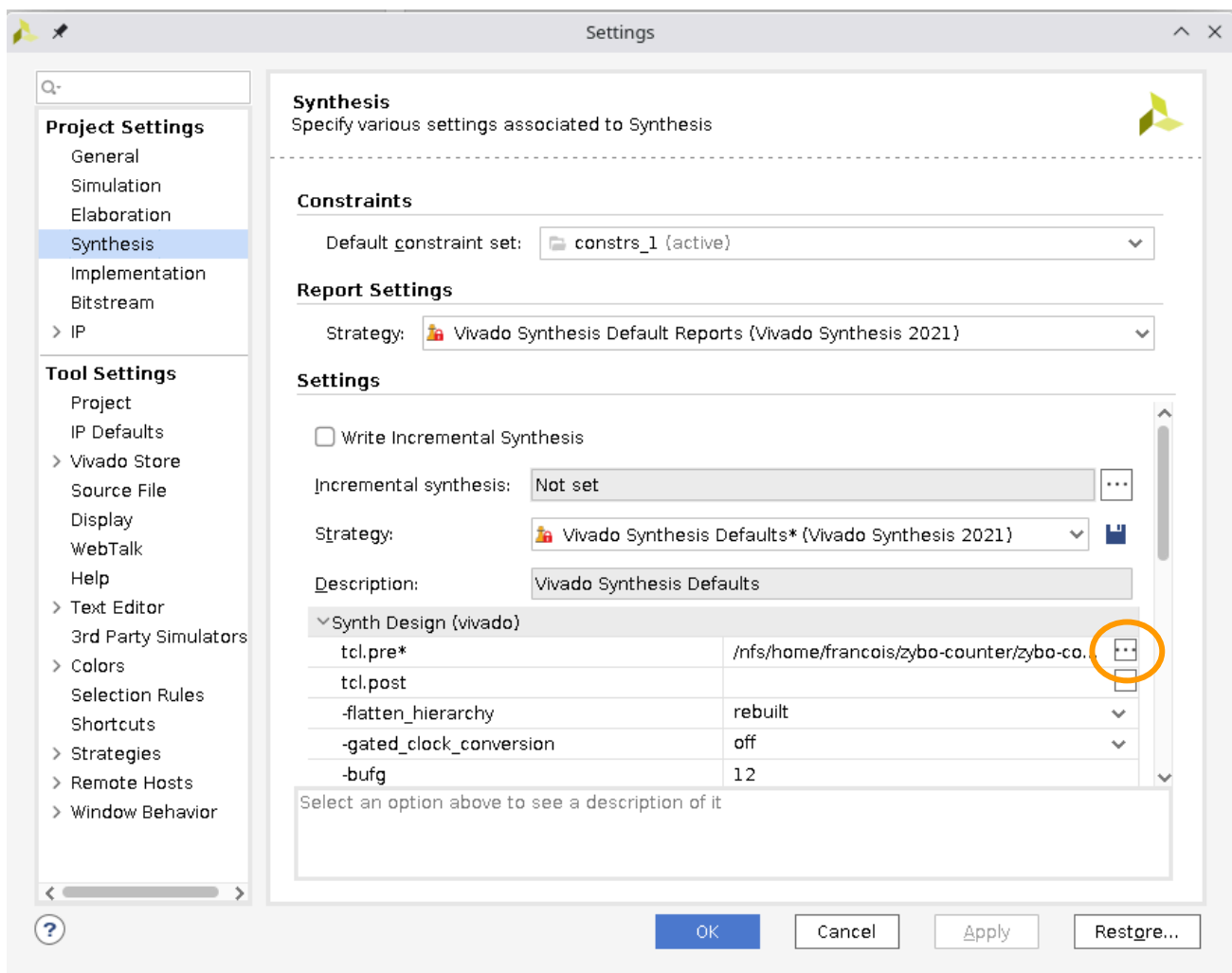
top-level synthesis and generic parameters

In order to give generic parameters values to top-level components, you'll need to create a TCL file at the root of your project:

- `pulse_gen_synth.pre.tcl`

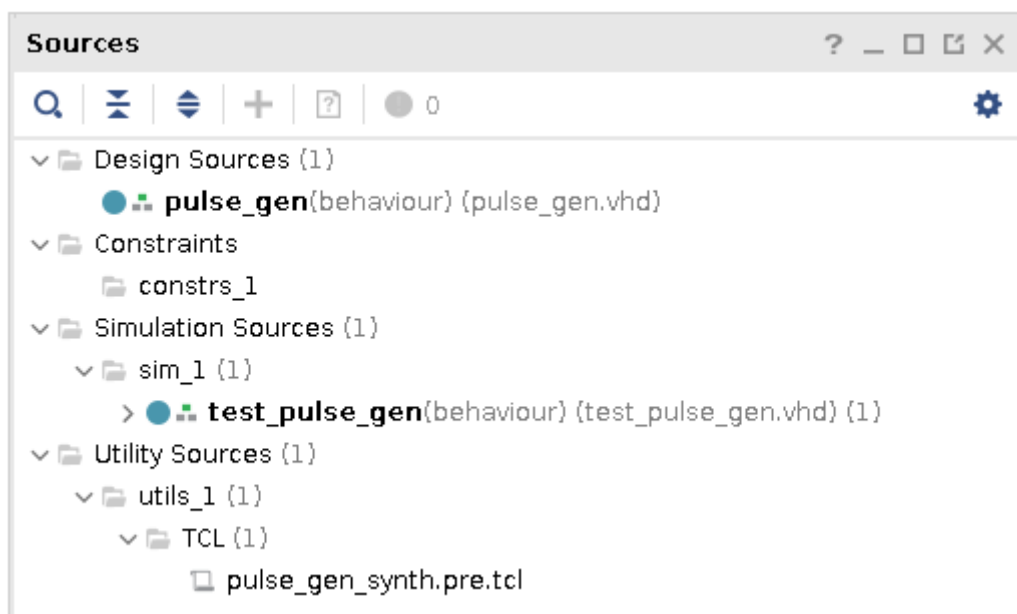
```
# set generic parameter for synthesis
set_property generic {MAX_CPT=8} [current_fileset]
```

Settings → Project Settings → Synthesis → tcl.pre field



Then,
this new

file will appear as an utility source file



New synthesis run will reports differently this time:

Site Type	Used	Fixed	Prohibited	Available	Util%
Slice LUTs*	3	0	0	17600	0.02
LUT as Logic	3	0	0	17600	0.02
LUT as Memory	0	0	0	6000	0.00
Slice Registers	4	0	0	35200	0.01
Register as Flip Flop	4	0	0	35200	0.01
Register as Latch	0	0	0	35200	0.00
F7 Muxes	0	0	0	8800	0.00
F8 Muxes	0	0	0	4400	0.00

Ref Name	Used	Functional Category
FDCE	4	Flop & Latch
LUT3	3	LUT
LUT1	2	LUT
IBUF	2	IO
OBUF	1	IO
BUFG	1	Clock

FDCE: D flip-flop with Clock Enable and Asynchronous Clear
 FDPE: D flip-flop with Clock Enable and Asynchronous Preset
 FDSE: D flip-flop with Clock Enable and Synchronous Set
 FDRE: D flip-flop with Clock Enable and Synchronous Reset
[\[ug901\] Xilinx Vivado design suite user guide: synthesis](#)

comment-out generic map

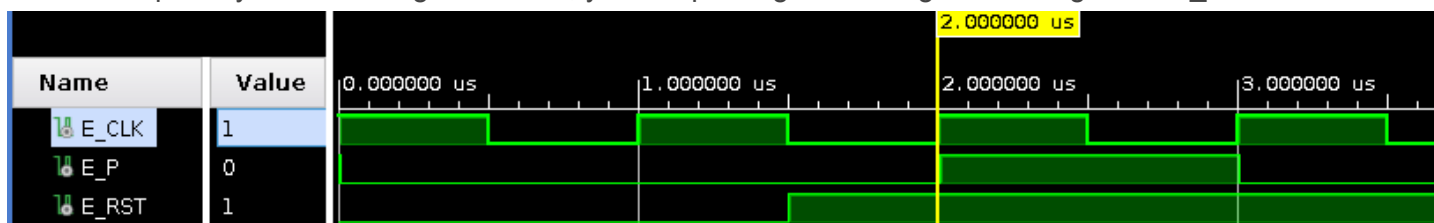
- test_pulse_gen.vhd

```

-----
-- instantiation et mapping du composant registres
pgen0 : entity work.pulse_gen(behaviour)
--      generic map (10)
      port map (MCLK => E_CLK,
                RST => E_RST,
                P => E_P);
    
```

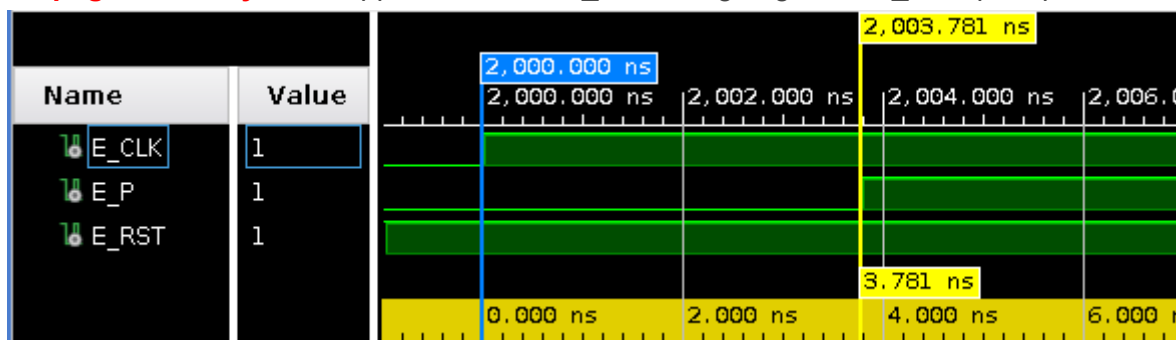
post-synthesis timing simulation

Restart the post-synthesis timing simulation, your output ought to change according to MAX_CPT value.



>>> Do you notice something ?? <<<

Propagation delays now appear between E_CLK rising edge and E_P output update:



⇒ What's the **MCLK's maximum frequency** of your pulse_gen component on this chip ?

$$f = \frac{1}{T}$$

pulse_gen with synchronous RST

This time, we'd like you to modify your pulse_gen architecture to implement a **synchronous** reset.

Ref Name	Used	Functional Category
FDRE	4	Flop & Latch
LUT4	2	LUT
IBUF	2	IO
obuf	1	IO
LUT3	1	LUT
LUT2	1	LUT
LUT1	1	LUT
BUFG	1	Clock

FDCE: D flip-flop with Clock Enable and Asynchronous Clear
 FDPE: D flip-flop with Clock Enable and Asynchronous Preset
 FDSE: D flip-flop with Clock Enable and Synchronous Set
 FDRE: D flip-flop with Clock Enable and Synchronous Reset
 see [\[ug901\] Xilinx Vivado design suite user guide: synthesis](#)

What do you notice on chronogrammes ? regarding the instantiated resources ??

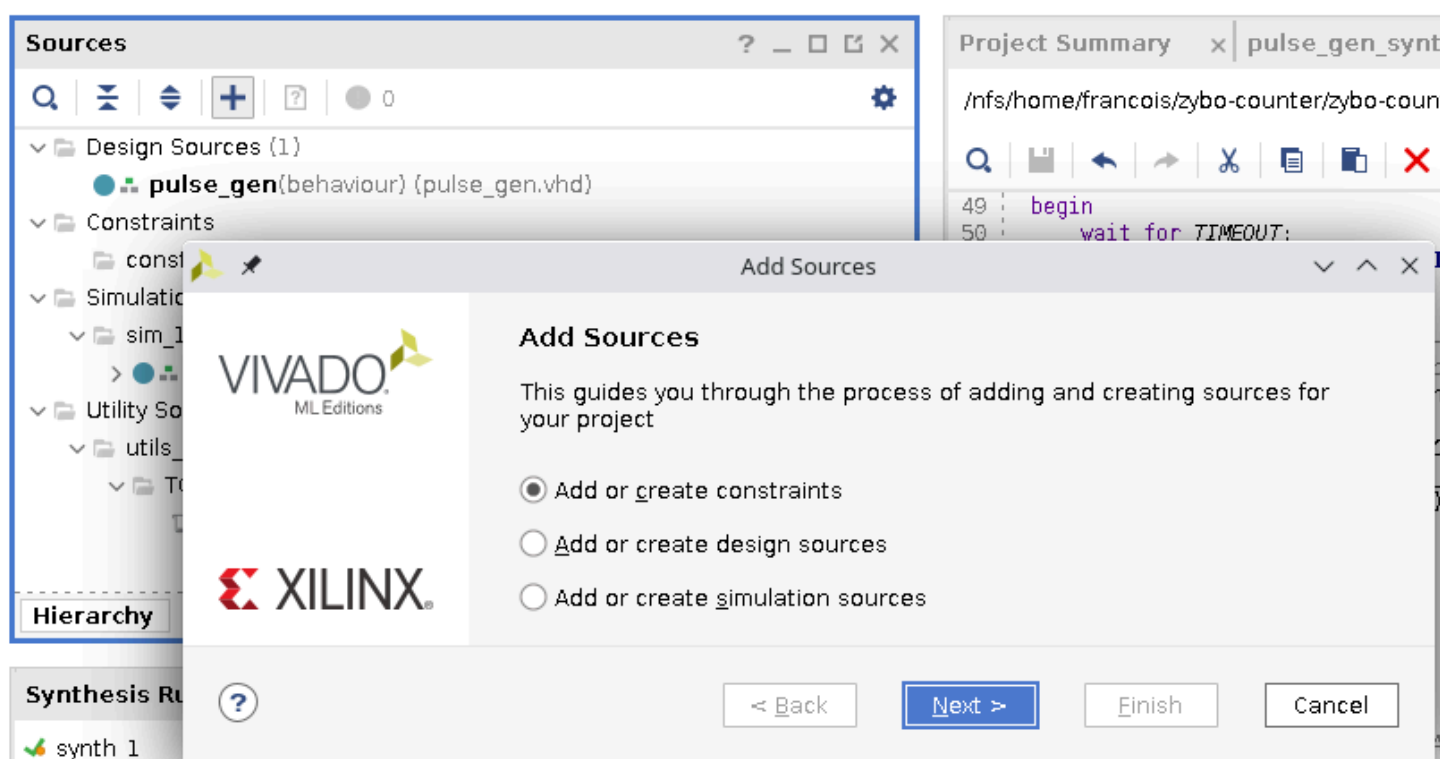
TP3 - Pulse generator constraints

In the previous exercises, we simulated our `pulse_gen` component **without any timing constraints** to validate against ... hence the unconstrained paths reported by the Vivado's **Report Timing summary** (left toolbar *Synthesis* → *Report Timing Summary*)
 see [pulse_gen unconstrained paths](#) on next page

Timing constraints

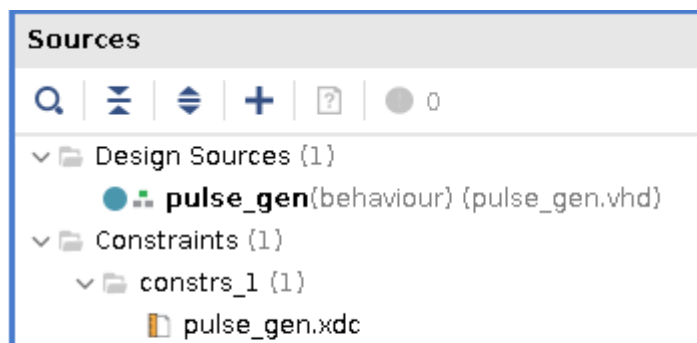
A *constraint file* enables you to express timing requirements against a clock frequency your design is supposed to perform with. In subsequent practical exercises, you'll discover that this constraint file is also used for I/O Planning (i.e which IO ports your design makes use of). These files exhibit a `xdc` extension.

You'll now create a constraint file. To achieve this, click on '+' in Sources area and select *add or Create constraints* → create a file named '`pulse_gen.xdc`'



Your design now features a new constraint file named '`pulse_gen.xdc`'.

Instead of trying to fill it on yourself, we'll make use of the *Synthesis* → **Constraints Wizard**



pluse_gen unconstrained paths

Sources **Netlist** x ? _ □ ✕

pulse_gen

> Nets (17)

▼ Leaf Cells (15)

- GND (GND)
- MCLK_IBUF_BUFG_inst (BUFG)
- MCLK_IBUF_inst (IBUF)
- P_i_1 (LUT1)

Project Summary x pulse_gen_synth.pre.tcl x pulse_gen.vhd x test_pulse_gen.vhd x pulse_gen.xdc x **Schematic** x ? _ □ ✕

13 Cells 3 I/O Ports 17 Nets

Path Properties ? _ □ ✕

Path 1

Data Path

General Properties **Report**

Tcl Console Messages Log Reports Design Runs **Timing** x ? _ □ ✕

Unconstrained Paths - NONE - NONE - Setup

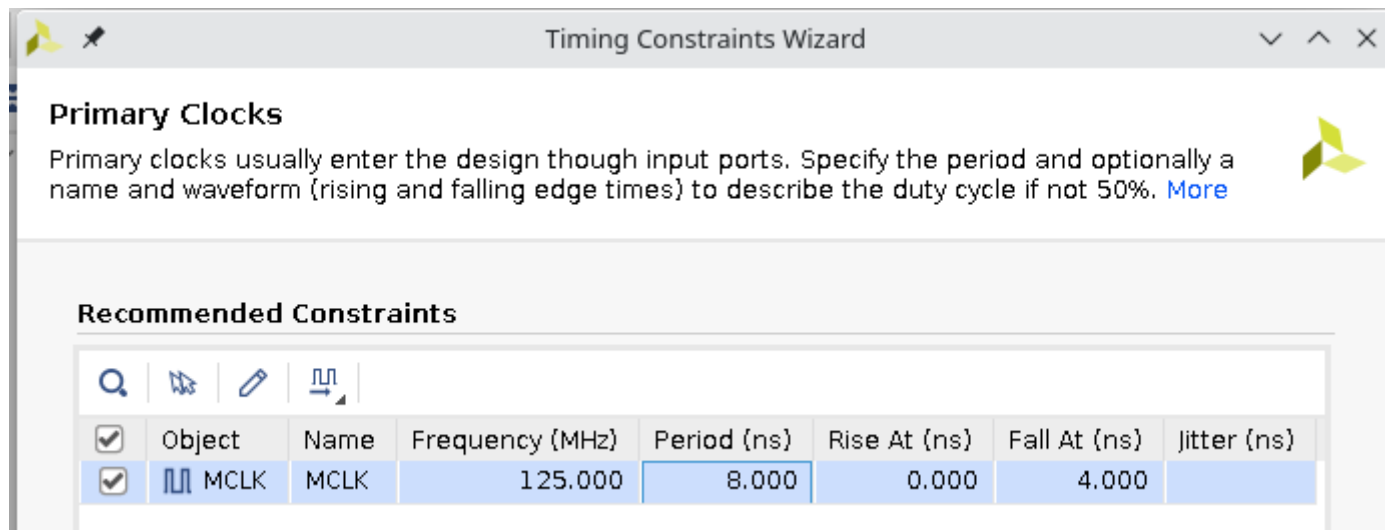
Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination
Path 1	∞	2	2	1	P_reg/C	P	4.076	3.276	0.800	∞		
Path 2	∞	2	3	4	RST	P_reg/R	2.682	1.106	1.576	∞	input port clock	
Path 3	∞	2	3	4	RST	ppulse.cpt_reg[0]/D	1.906	1.106	0.800	∞	input port clock	
Path 4	∞	2	3	4	RST	ppulse.cpt_reg[1]/D	1.906	1.106	0.800	∞	input port clock	
Path 5	∞	2	3	4	RST	ppulse.cpt_reg[2]/D	1.900	1.100	0.800	∞	input port clock	
Path 6	∞	2	2	3	ppulse.cpt_reg[2]/C	P_reg/D	1.510	0.751	0.759	∞		

Timing Summary - timing_1

Synthesis Constraints Wizard

The first time you click on this feature, it will ask you to select a target (i.e which XDC file) $\Rightarrow$ pulse_gen.xdc
 The second time, the wizard will start :)

In this application case, we'll only focus on the MCLK constraints. Set MCLK as being a 125MHz clock then skip to finish.

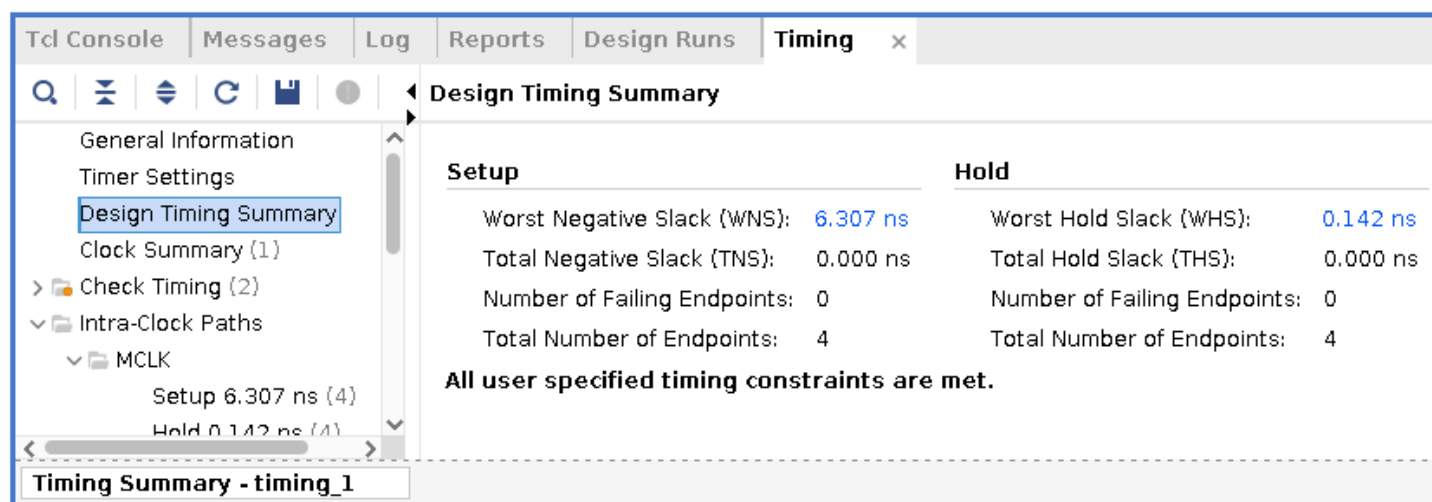


This will lead to a constraint rule added to your pulse_gen.xdc :

```
# Master Clock timing constraint
create_clock -period 8.000 -name MCLK -waveform {0.000 4.000} [get_ports MCLK]
```

Note: this rule DOES NOT DEFINE a clock, instead, it specifies a clk our system is supposed to comply with.

Now you restart Synthesis $\rightarrow$ Report Timing summary



>>> Slack OUGHT to be positive <<<

Setup slack: Required time - Arrival time $\rightarrow$ delay before next CLK front that will memorize the data

Hold slack: Arrival time - Required time $\rightarrow$ delay the data change after the CLK front.

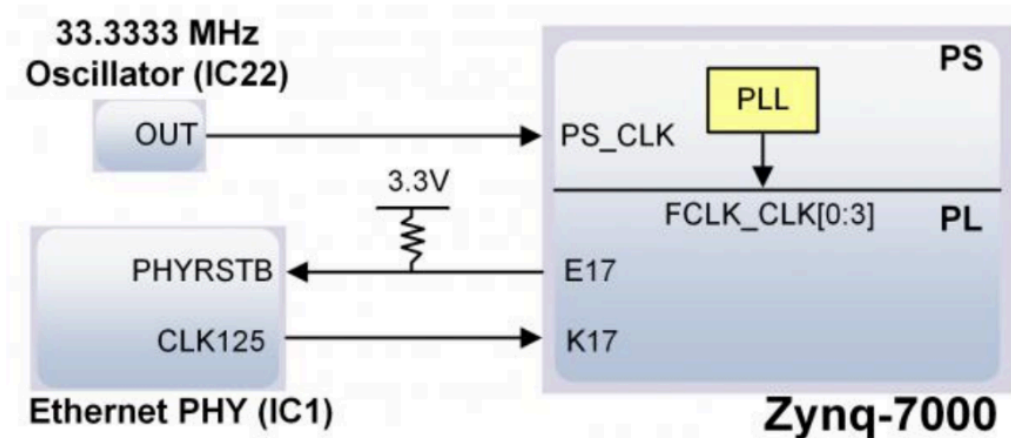
Introduction to slack and others timing considerations in FPGAs

https://secil.univ-tlse3.fr/teaching/francois/UE-SSM/Max-Planck-institute_Timing_fpga_sep24.pdf

Implementation Constraints Wizard

Why did we choose 125MHz for MCLK in the previous example ?

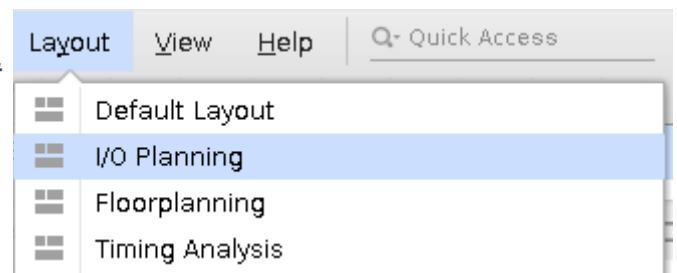
That's because the Zybo board features an independent PL_CLK at 125MHz coming from the onboard Ethernet IC.



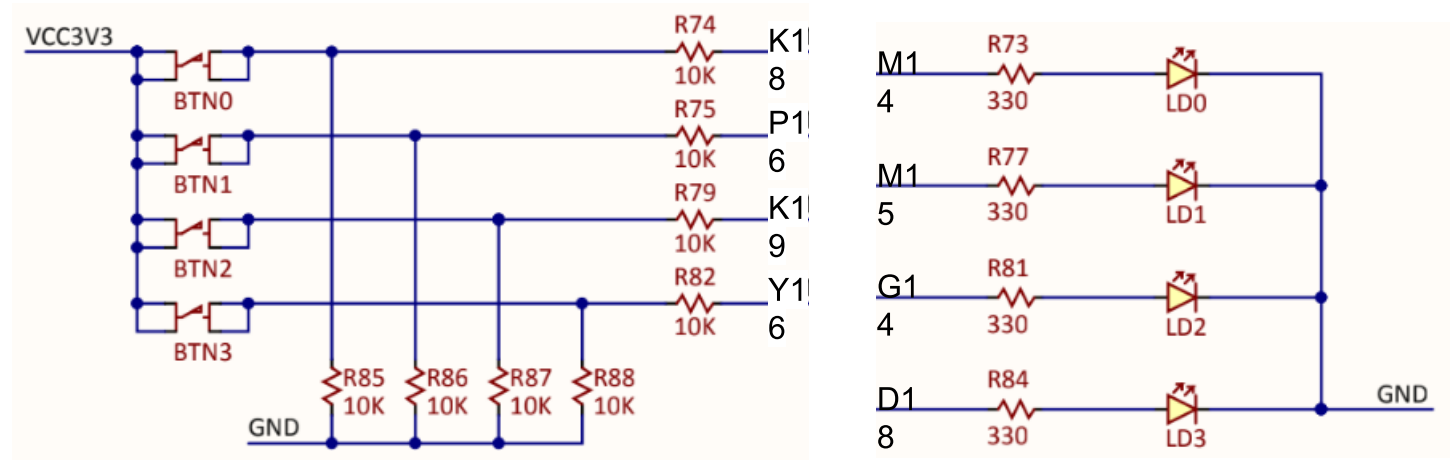
As shown in the upper figure, you can see that the **K17** pin is used as the PL_CLK input. Additionally, **E17** pin will be able to reset the Ethernet IC hence suspending the 125MHz clock.

Up To now, we've been through the synthesis process that translates your VHDL code to instantiated physical elements of our FPGA. Next step is to map IO pads to our design $\Rightarrow$ IO planning

Implementation $\rightarrow$ Run Implementation , then select layout $\rightarrow$ IO planning in the Vivado upper tool bar.



Below is a detailed overview of buttons and leds available on the Zybo board (switches not shown).



Now you'll affect Zybo GPIO to our pulse_gen ports according to the following setup:

Tcl ConsoleMessagesLogReportsDesign RunsTimingPowerMethodologyDRCPackage PinsI/O Ports x

NameDirectionBoard Part PinBoard Part InterfaceNeg Diff PairPackage PinFixedBankI/O Std

All ports (3)

Scalar ports (3)

MCLKIN

K17

35

LVC MOS33*

P

OUT

M14

35

LVC MOS33*

RST

IN

R18

34

LVC MOS33*

When finished, save as **CTRL + S** → Vivado will ask you whether it ought to save these constraints in the default constraints file (i.e pulse_gen.xdc) → yes !

In the end, your pulse_gen.xdc file ought to looks like this:

```
# Master Clock timing constraint
create_clock -period 8.000 -name MCLK -waveform {0.000 4.000} [get_ports MCLK]

set_property PACKAGE_PIN K17 [get_ports MCLK]
set_property IOSTANDARD LVCMOS33 [get_ports MCLK]

set_property PACKAGE_PIN M14 [get_ports P]
set_property IOSTANDARD LVCMOS33 [get_ports P]

set_property PACKAGE_PIN R18 [get_ports RST]
set_property IOSTANDARD LVCMOS33 [get_ports RST]
```

... a more elegant way is to make use of the 'dict' feature: pulse_gen.xdc

```
# Master Clock timing constraint
set_property -dict { PACKAGE_PIN K17 IOSTANDARD LVCMOS33 } [get_ports MCLK];
create_clock -period 8.000 -name MCLK -waveform {0.000 4.000} [get_ports MCLK];

# Reset (warning pull down resistors on board)
set_property -dict { PACKAGE_PIN R18 IOSTANDARD LVCMOS33 } [get_ports RST];

# P output
set_property -dict { PACKAGE_PIN M14 IOSTANDARD LVCMOS33 } [get_ports P];
```

It's now time to restart Synthesis + Implementation :)

Full timings simulation

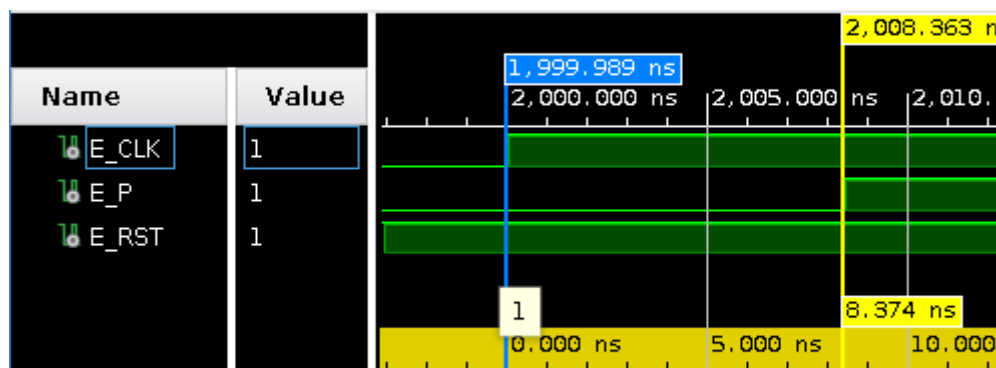
This time, you'll select:

Simulation

→ Run Simulation

→ Post-Implementation timing simulation

What do you observe ??



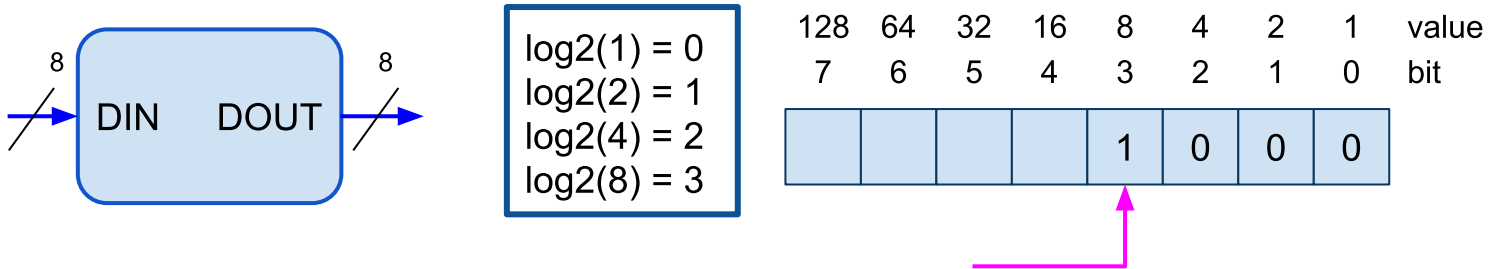
[master2] Bitstream generation and hardware download

Ultimate step that will generate the .bit file. This file will get downloaded into the FPGA's config RAM that will organise all resources as specified in synthesis and implementation steps. This file will get downloaded through JTAG (usb prog port) (hint: change the pulse duration to be able to see it!)

TP4 - Synthesizable Log2 function

This time, we'd like you to write a **synthesizable** Log2 component that will feature the following generic parameter(s):

- **BUS_WIDTH** : size of input and output buses with 8 as default value



log2 @ cpu_package

Once you retrieved the 'cpu_package.0.vhd' file, you'll find definition for several log2 (overloaded) functions:

```
-- fonction log2
--         calcule le logarithme base2 d'un entier naturel, ou plus exactement
--         renvoie le nombre de bits nécessaires pour coder une valeur
function log2 (I: in natural) return natural;
function log2 (vI: in std_logic_vector) return std_logic_vector;
```

Your first task will be to implement a synthesizable version of the log2 function in the package body part of the cpu_package file.

Run a behavioral simulation by means of the 'test_log2.vhd' file.

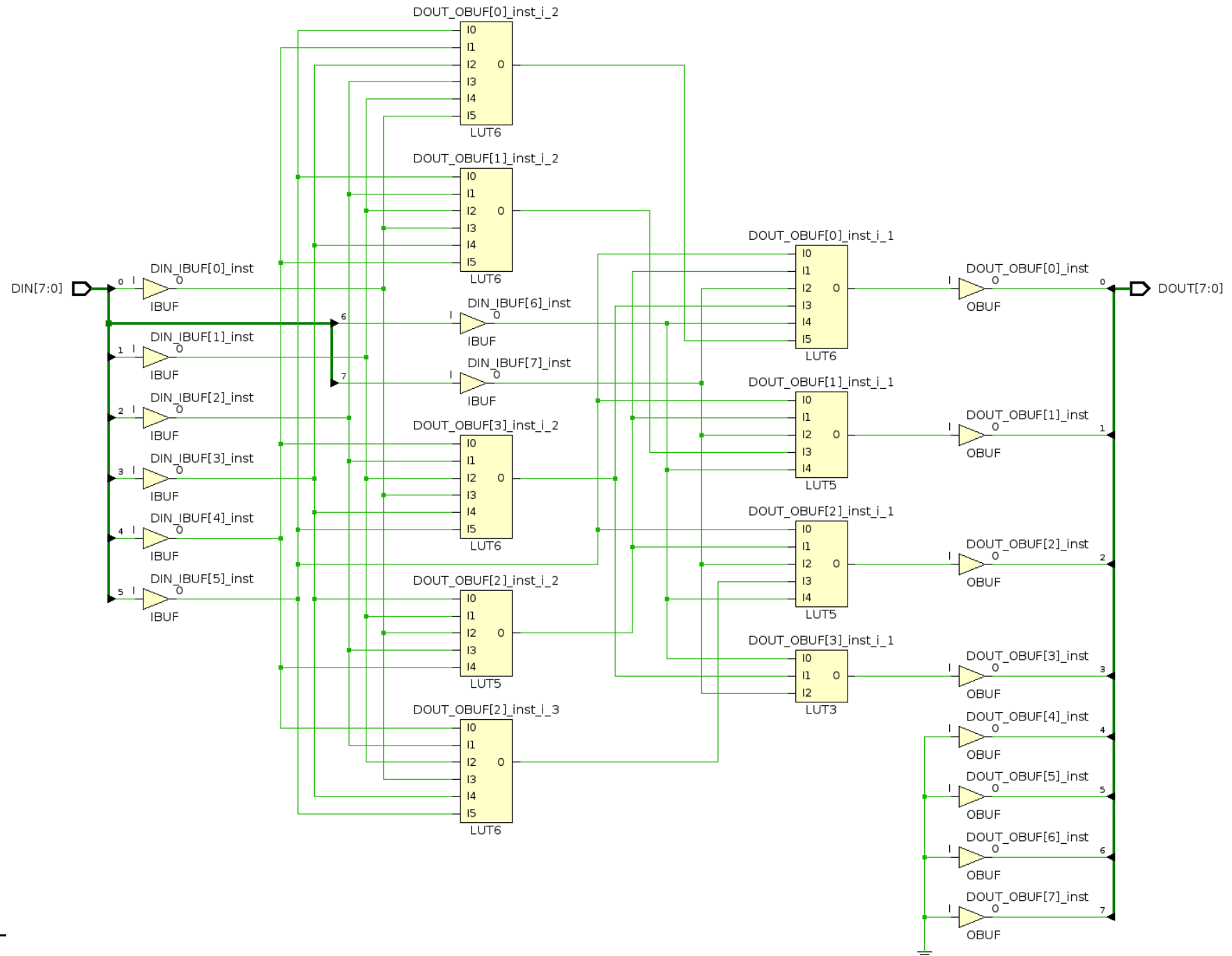
log2 hardware component

As you noticed, your log2 function exhibits **unconstrained** std_logic_vector both as input and output. Synthesis can manage this situation by dynamically selecting the proper bus size through component instantiation.

⇒ complete the 'log2_hw.vhd' file with the generic parameter **BUS_WIDTH**; your architecture will make use of your previously written log2 function.

Undertake a **post-synthesis timing** simulation.

Next step: have a closer look at the schematic of the resulting synthesis: what do you notice on output ??

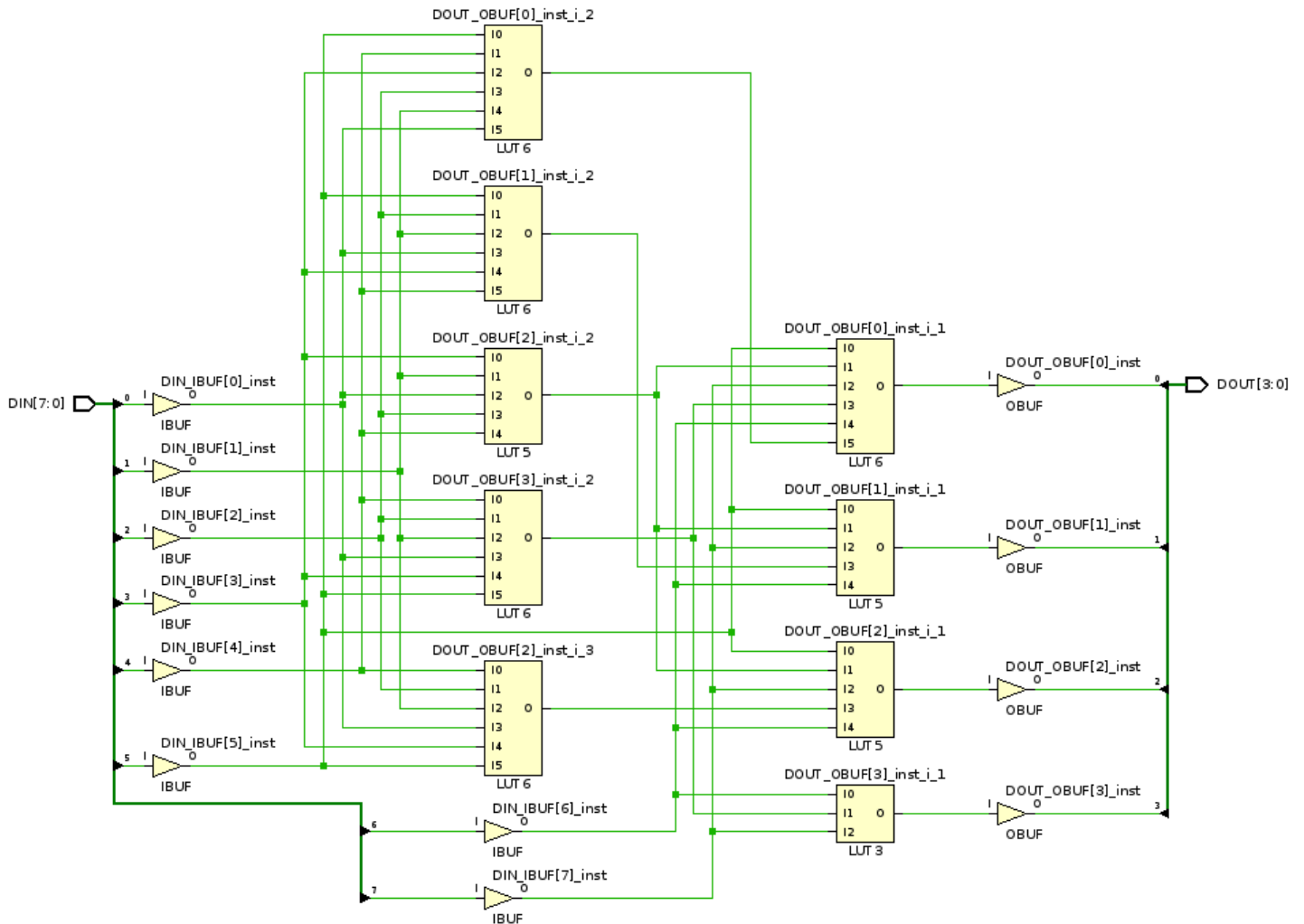


[master2] optimized log2 component

In order to overcome the issue related to the wrong size of the log2 component's output, you'll need to:

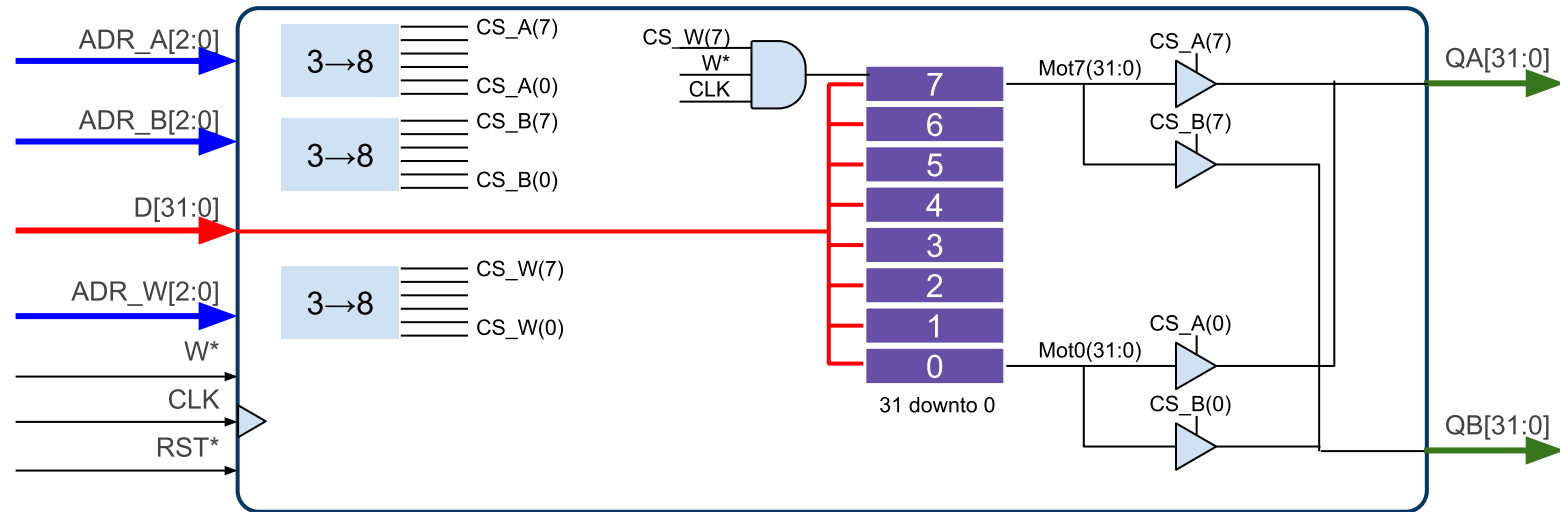
- reshape output and subsequents to `std_logic_vector(log2(BUS_WIDTH)-1 downto 0)`;
- import your `log2` function from `cpu_package` as a new `myLog2` function to settle in the declarative part of your log2 component's architecture.

Undertake a **post-synthesis timing** simulation and have a look at the generated schematic that ought to look like this later:



TP5 - Registers bank for Risc processor

We'd like you to create a bank of 8 registers 32 bits featuring dual read access. One notable point is that **reading register 0 always sends back value 0**.



Signals featuring "*" means that they are active low. ADR_A and ADR_B addresses buses lead to asynchronous reading on ports QA and QB respectively.

RST* is synchronous and leads to all registers filled with 0. Write operations through ADR_W and active W* signal will be undertaken on the rising edge of CLK.

generic parameters

- **DBUS_WIDTH** internal words and data buses size, default to **32**
- **ABUS_WIDTH** size of all addresses buses, default to **5**

Notes

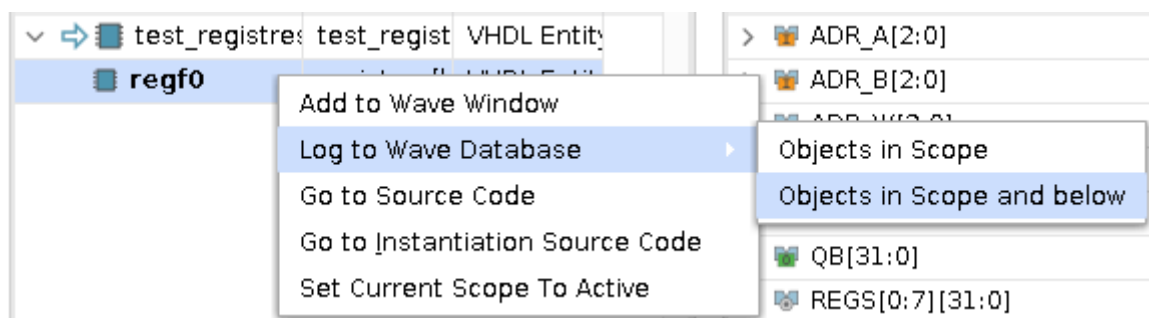
To declare a 32bits wide 8 registers bank in the declarative part of your architecture:

```
type FILE_REGS is array (integer range 0 to (2**5)-1) of std_logic_vector (31 downto 0);
signal REGS : FILE_REGS;
```

Behavioural simulation

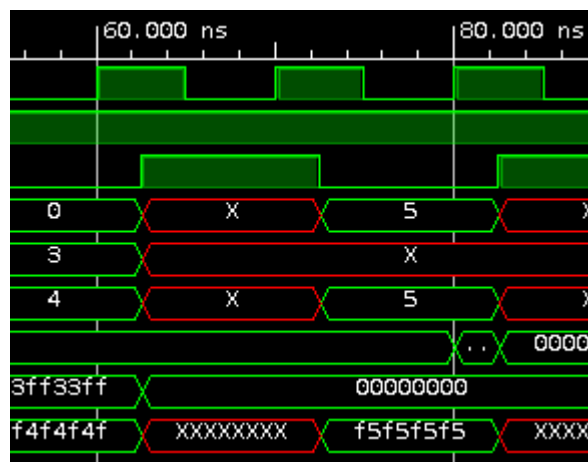
You'll complete the design of our registers bank from the 'registres.0.vhd' file and then, add the simulation source 'test_registres.0.vhd'.

Note: to gain access to ALL signals in objects and below, you'll need to tell the simulator to undertake a recursive inclusion



Around 80ns in the waveform window, you notice an **issue** that occurs whenever the processor is trying to **read and write** the same register at the same time !

To overcome this **feature bug**, we want you to implement the **bypass mechanism**.



Registers Bypass design and simulation

We need to detect the condition which lead to the previously mentioned issue:

if write and ADDR_A or ADDR_B equal ADDR_W then QA or QB <= D;

This time, you'll need to **remove** the 'registres.0.vhd' file with the new one 'registres.1.vhd'. In this file, we want you to rewrite in the concurrent area of your architecture the two existing processes named P_READQA and P_READQB.

Check the operability of your **bypass** by means of a **behavioural simulation**.

Synthesis with constraints

This step will try to tackle the following objectives:

- determining the **maximum frequency** a processor can access our bank of registers,
- use **BlockRAMs** instead of traditional FF

Note: remember to set generic parameters for synthesis as what has been done in [top-level synthesis and generic parameters](#)

Note: in a similar way to what was done in [Synthesis Constraints Wizard](#), define the same 125MHz clock.

Ref Name	Used	Functional Category	Site Type	Used	Fixed	Prohibited	Available	Util%
FDRE	224	Flop & Latch	Block RAM Tile	0	0	0	60	0.00
LUT5	128	LUT	RAMB36/FIFO*	0	0	0	60	0.00
LUT6	66	LUT	RAMB18	0	0	0	120	0.00
OBUF	64	IO						
MUXF7	64	MuxFx						
IBUF	44	IO						
LUT4	7	LUT						
LUT3	2	LUT						
LUT1	1	LUT						
BUFG	1	Clock						

We're not using any **BRAM** primitives :(

To found out the maximum affordable frequency of our design, we need to discover the longest path in our design ⇒ *Synthesis* → *Report Timing summary*

Tcl Console

Messages

Log

Reports

Design Runs

Timing

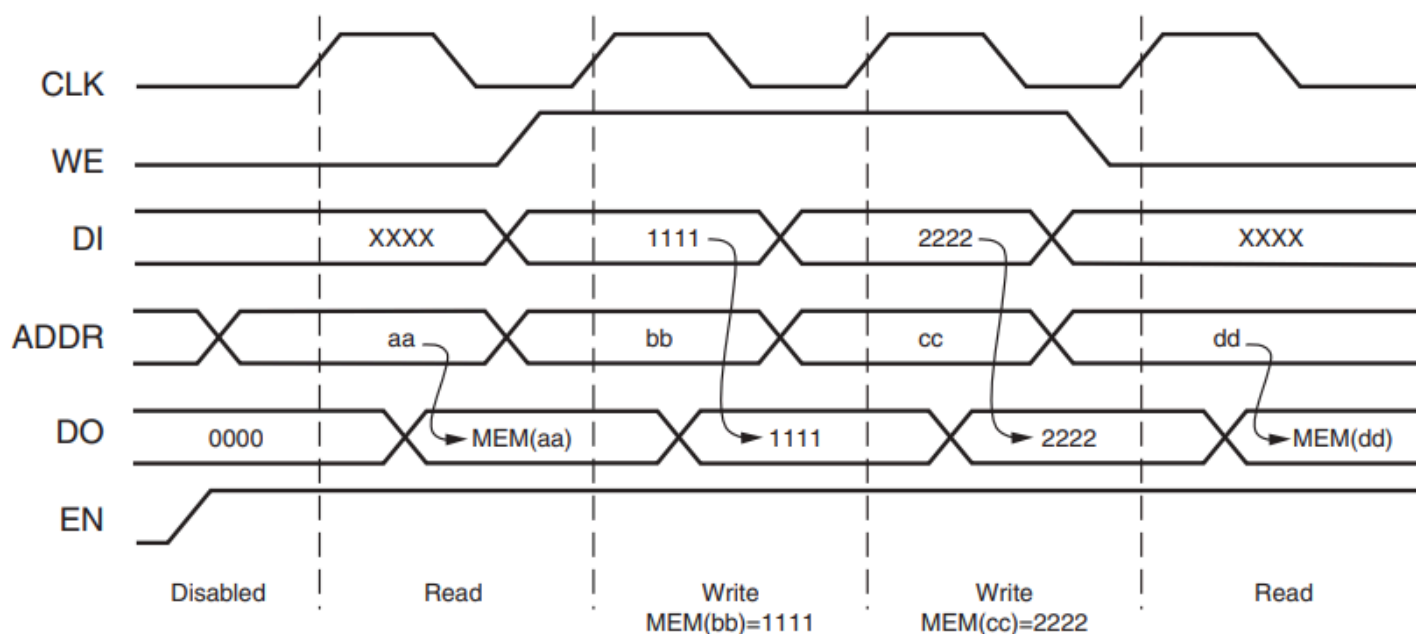
Debug

Then, as you may already have guessed ... the longest path is when you're undertaking both read and write operations on the same register involving all of the bypass logic
 ⇒ 150MHz will be our most achievable operational frequency.

[master2] BRAM inference and maximum reachable frequency

For comparison purposes, we want you to design our bank of registers by means of Block RAM primitives. You'll drive synthesis and post-synthesis timing simulation to find out the max sustainable frequency.

Below is **ug473** excerpt showing **WRITE_FIRST** (i.e transparent mode) policy. Note that pipeline register output s not used

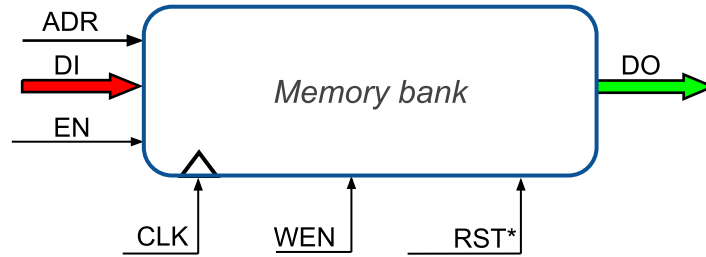


To achieve this, best is to start with [TP6 - Memory bank for RISC processor](#)

TP6 - Memory bank for RISC processor

We propose you to carry out our future data & instructions L1 caches as a generic memory bank featuring the following parameters:

- **DBUS_WIDTH** internal words and data buses size, default to **32**
- **MEM_SIZE** nb words within our memory bank, default to **8**
- **FILENAME** file whose content will be our memory bank defaults values, default to *empty string*.



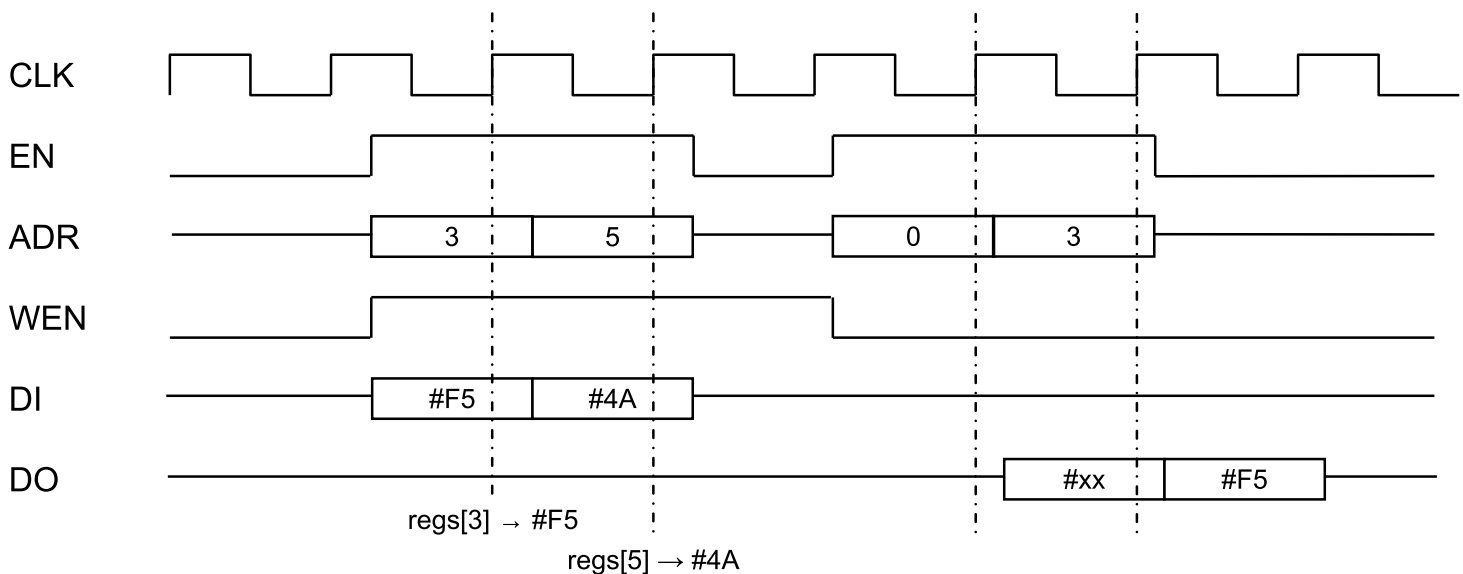
Functional principle

RST* will be **synchronous**. It either enables loading a file's content whose name has been given as the generic parameter **FILENAME**, or all memory elements will get assigned value **0**.

EN signal validates memory access whose address is set on the ADR bus while WEN signal will tell this operation is a write (i.e read otherwise). On the next rising edge of CLK, data will be set on the DO bus (read) or written within the corresponding word.

Memory mapping: 1@ → 1 word

EN disabled ⇒ **DO sets to high impedance Z**.



Simulation

Undertake a behavioural simulation of your memory component by means of the 'memory.3.vhd' and 'test_memory.3.vhd' test bench. You'll also need your 'cpu_package.0.vhd' as source file (log2 function) along with the 'rom_file.0.vhd' that will get used to pre-fill your memory on reset.

Note: 'rom_file.0.txt' ought to be added as source file (it will be tagged as TEXT file).

WARNING: in 'test_memory.3.vhd', you ought to specify the absolute filename to the 'rom_file.0.txt' file:

```

.....
constant FILENAME : string := "/home/<user>/tpvhdl/rom_file.0.txt";
.....
    
```

Once functional behaviour is achieved, start **design synthesis** and then carry out a **post-synthesis timing simulation** to determine the maximum sustainable frequency.

Hint: don't forget to add the following pre-synthesis constraints like what was done in [top-level synthesis and generic parameters](#)

- memory_synth.pre.tcl

```

# set generic parameter for synthesis
set_property generic {DBUS_WIDTH=32 MEM_SIZE=32 FILENAME="rom_file.0.txt"} [current_fileset]
    
```

Note: below some size threshold, Vivado may decide to infer RAM as distributed RAM (i.e LUT)

>>> How many BRAM did you infer ??? <<<

Block RAM inference

As you may have noticed, you'll need your previously written **log2** function to express the size of the ADR bus. Starting from the memory.3.vhd file, you infer Block RAMs from the Xilinx architecture.

Hint: have a look at ug901 'RAM HDL Coding Techniques' to know how to infer BRAM at synthesis without any architecture specific component instantiation.

⇒ you discover that you need to remove the RST line and disable the 'Z' feature when EN='0'

Restart design synthesis and observe the output results:

Site Type	Used	Fixed	Prohibited	Available	Util%
Slice LUTs*	0	0	0	17600	0.00
LUT as Logic	0	0	0	17600	0.00
LUT as Memory	0	0	0	6000	0.00
Slice Registers	0	0	0	35200	0.00
Register as Flip Flop	0	0	0	35200	0.00
Register as Latch	0	0	0	35200	0.00
F7 Muxes	0	0	0	8800	0.00
F8 Muxes	0	0	0	4400	0.00

Site Type	Used	Fixed	Prohibited	Available	Util%
Block RAM Tile	0.5	0	0	60	0.83
RAMB36/FIFO*	0	0	0	60	0.00
RAMB18	1	0	0	120	0.83
RAMB18E1 only	1				

Tcl ConsoleMessagesLogReportsDesign Runs x

Q

≡

≡

⏮

⏪

▶

⏩

+

%

Name

Constraints

Status

LUT

FF

BRAM

URAM

DSP

✓ synth_1

constrs_1

synth_design Complete!

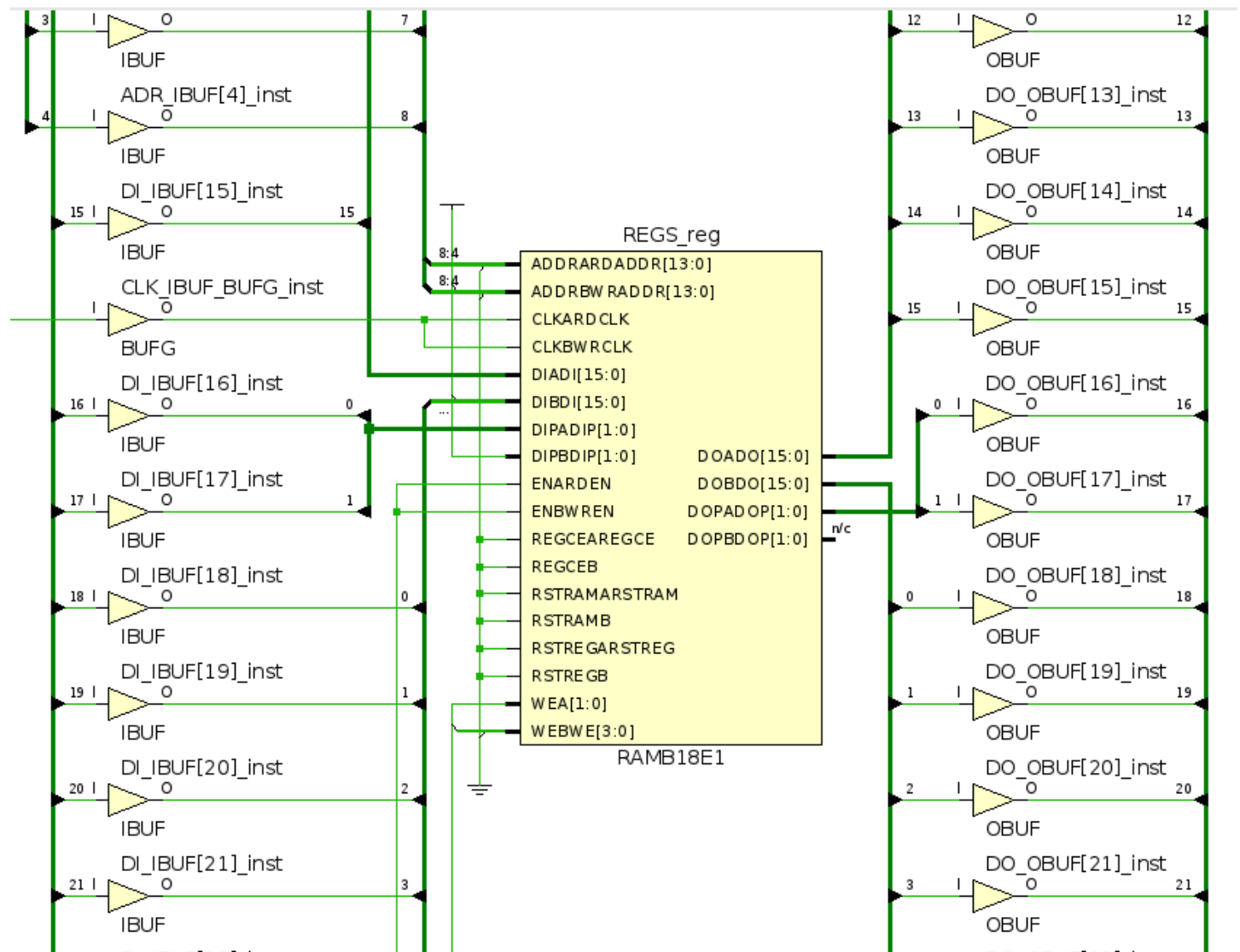
0

0

0.5

0

0



Ok, good you really got so far :)

Unfortunately, whatever simulator you may use, VHDL timing simulation does not exist for Xilinx primitives (only verilog and system verilog) ... $\Rightarrow$ it's time to test in real life :)

[master2] bitstream & JTAG debug

Same steps as [\[master2\] Bitstream generation and hardware download](#) then open the **Hardware Debugger**

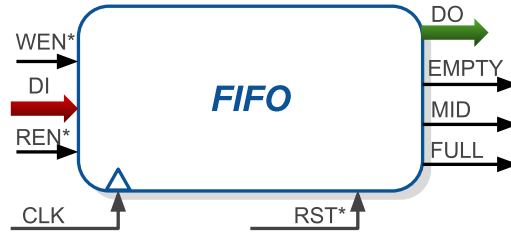
to be continued ...

TP7 - dual ports FIFO

On se propose de réaliser une FIFO* synchrone double port selon les caractéristiques génériques suivantes :

- **DBUS_WIDTH** Taille d'un mot du banc avec 32 bits par défaut.
- **ABUS_WIDTH** Profondeur d'utilisation avec 3 bits d'adresse par défaut - soit $2^{**}3$ mots -.

*First In First Out



Principe de fonctionnement

Le signal RST*, synchrone, remet à 0 les pointeurs lecture et écriture **mais n'efface pas** le contenu des registres. Il positionne également la sortie **DO <= 'Z'**.

Tant que WEN* est actif, la FIFO continue de se remplir de façon circulaire à chaque **front montant** d'horloge écrasant ainsi les données les plus anciennes.

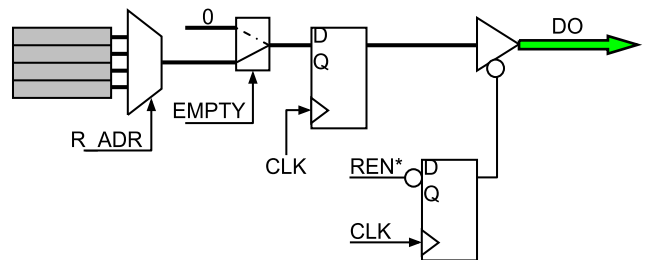
Tant que REN* est actif, la FIFO continue de présenter une donnée sur le bus à chaque **front montant** d'horloge jusqu'à être vide, auquel cas elle présentera la valeur 0.

Le bus de sortie Q devant être raccordé à d'autres unités, la sortie passe en haute impédance quand REN* inactif.

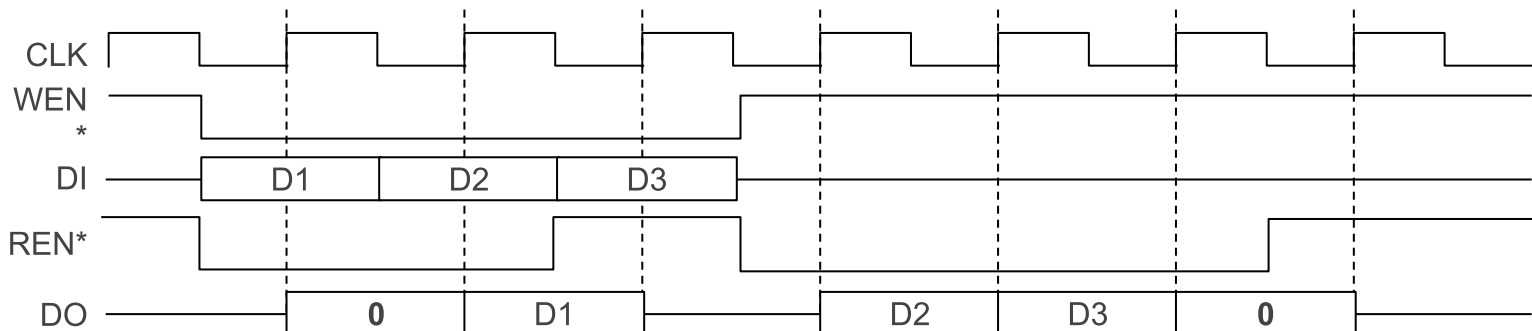
Les signaux EMPTY, MID et FULL représentent respectivement l'état vide, au moins à moitié plein* et plein. Ces derniers seront mis à jour sur front montant d'horloge.

*Le signal MID='1' quand remplissage FIFO $\geq 50\%$.

Les signaux WEN* et REN* peuvent être actifs simultanément.



Chronogramme



Implémentation

Vous complétez le fichier `fifo_0.vhd` comprenant entité et architecture.

La lecture / écriture étant circulaire, vous ferez attention aux conditions limites pour l'établissement des indicateurs de remplissage. Ensuite, afin d'accélérer les transferts, les pointeurs devront déjà être positionnés sur le futur emplacement de lecture ou d'écriture, l'incrément se faisant après l'opération proprement dite. Enfin, **le pointeur de lecture référence toujours la donnée la plus ancienne**.

Cas particuliers

Lecture & Ecriture & FIFO vide $\Rightarrow Q=0$ & mémorisation de la donnée en entrée.

FIFO pleine & Ecriture $\Rightarrow$ mémorisation de la donnée et incrément du pointeur de lecture.

FIFO pleine & Lecture & Ecriture $\Rightarrow Q=\text{donnée la plus ancienne}$ et mémorisation de la nouvelle.

Notes

- Le type **buffer** associé à un signal permet de relire un port configuré en sortie.

Behavioural simulation

Vous complétez le fichier `test_fifo.0.vhd` avec une instance du composant `fifo` possédant **4** mots mémoire de **4** bits.

Synthesis

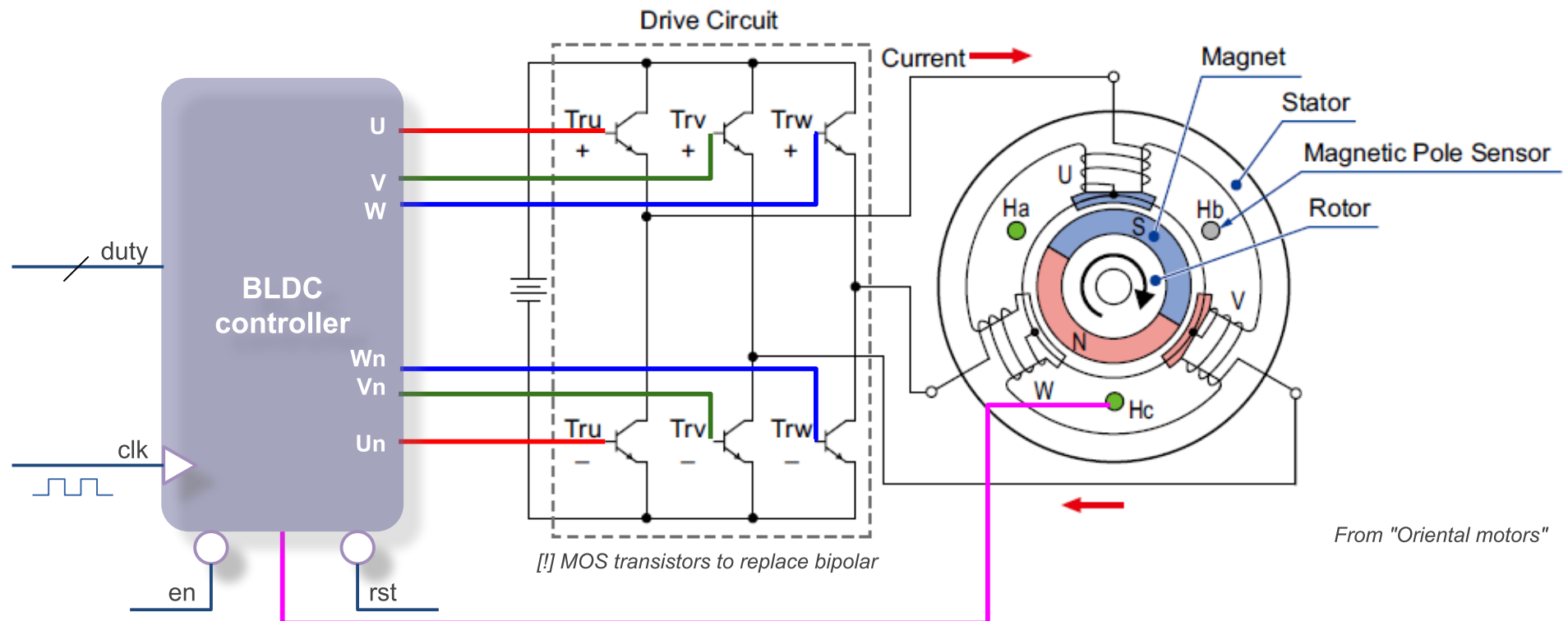
You will, as usual, undertake a synthesis of your design.

However, having already been using **BRAM** primitives, we'd like you to implement our FIFO based on these primitives. Indeed, according to [Xilinx ug473 documentation](#), BRAMs embed all logic bases to implement a FIFO. You are welcomed to tamper on the initial features of our FIFO, the main goal being to reduce the overall usage of LUT and FF vs BRAM elements.

>>> the less LUTs and FFs you use, the better will be your design (and rating) !! <<<

TP8 - BLDC controller

BLDC stands for "brushless direct current" and is related to a type of motor making use of permanent magnets on their rotor. These high reliability motors are intended for heavy duty purposes and can be found in drones, servers ... and hard drives !



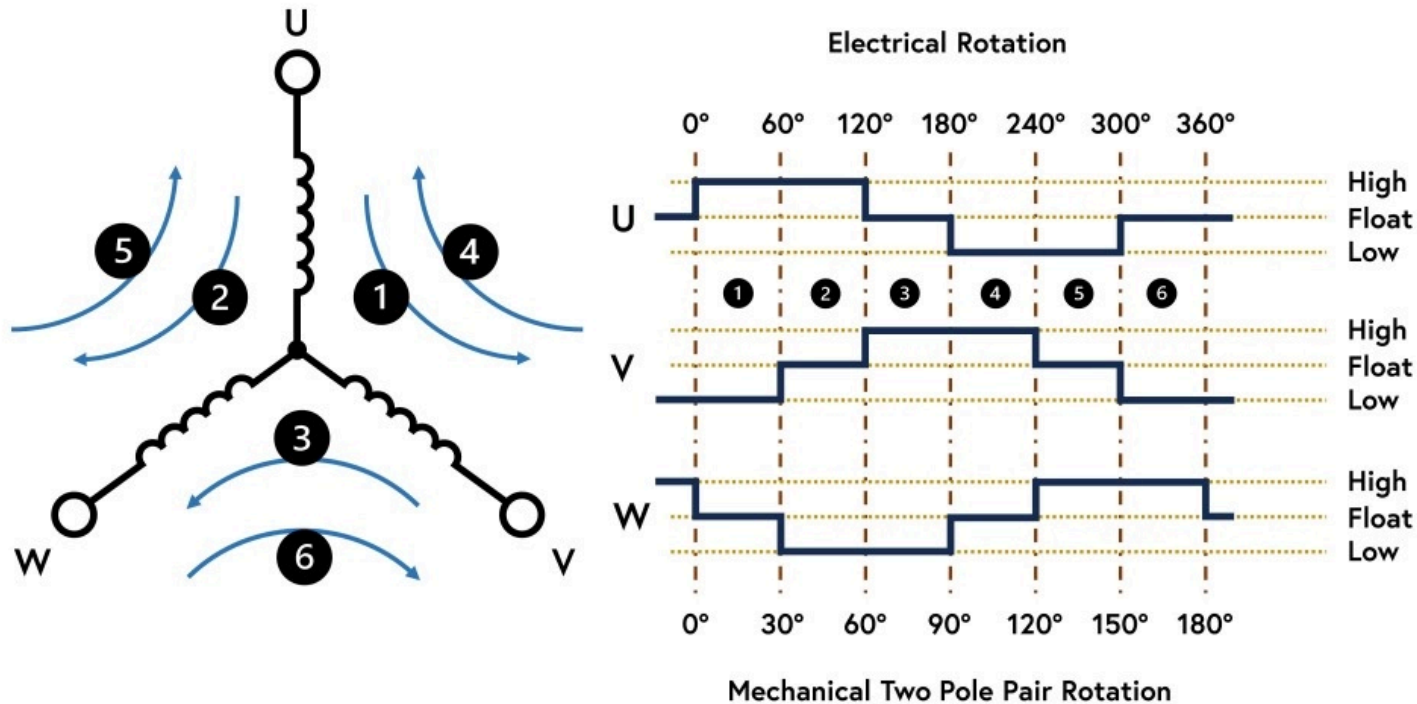
Unlike DC motors, a BLDC motor requires 3 phases sequenced in a precise timing.

As shown above, your "*BLDC controller*" circuit will be in charge of driving the six transistors of the "*driver*" part of the system. It is worth mentioning that "*electrical speed*" may differ from "*mechanical speed*" (e.g startup phase with mechanical load), hence we'll make use of a feedback signal. This signal will either come from a hall effect sensor located within the stator ... or (and probably easier to use) a simple optical signal that detects the position of the disk in the hard drive BLDC use case (see photos below).

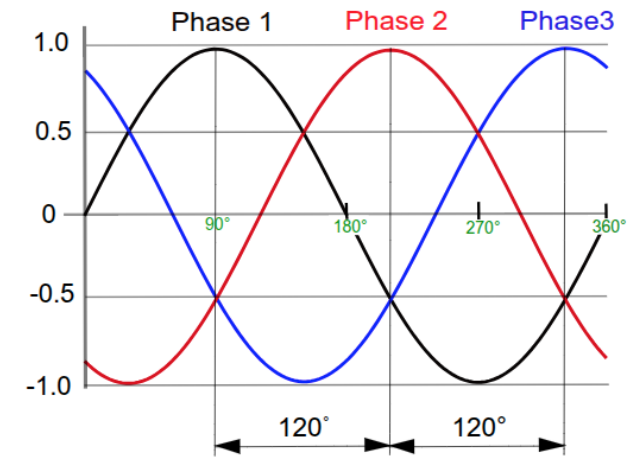
Here, you can see the 3 phases that are available everywhere in the electrical distribution network.

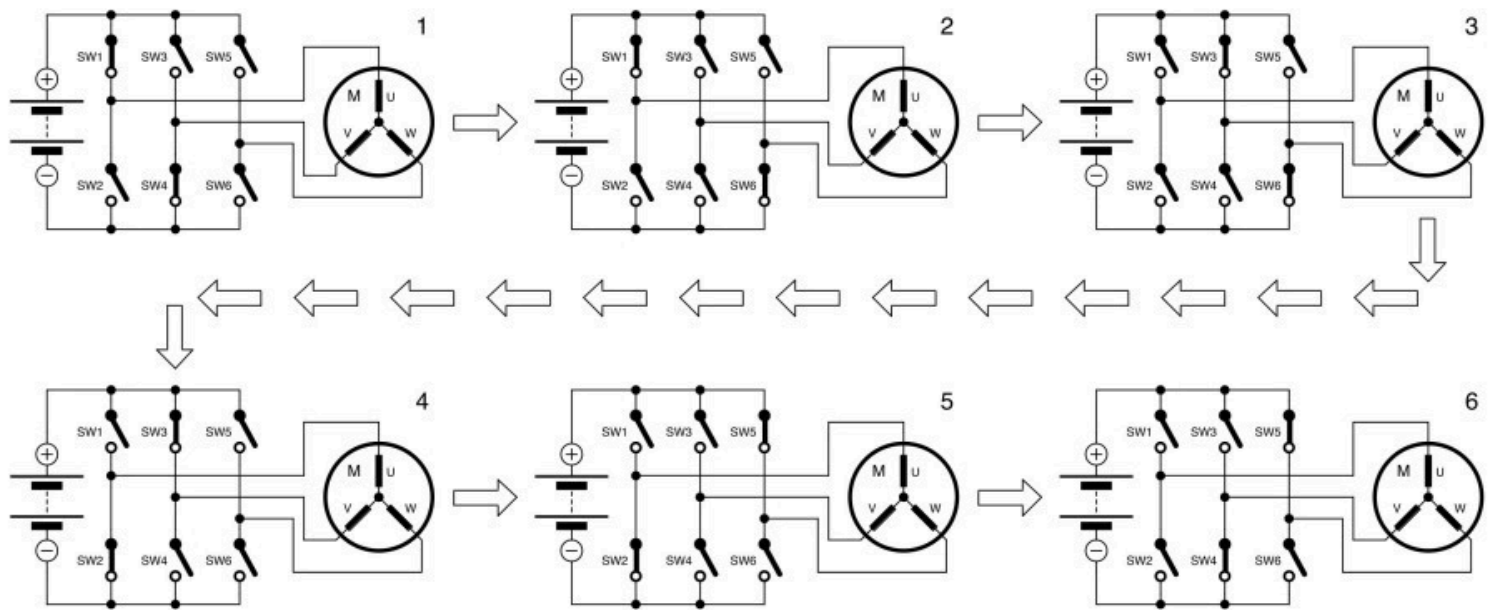
Wikipedia https://fr.wikipedia.org/wiki/Courant_triphas%C3%A9#/media/Fichier:3_phase_AC_waveform.svg

In order to ease understanding



Excerpt from Elektor magazine: "BLDC Guide du débutant"





Excerpt from Elektor magazine: "BLDC Guide du débutant"

TO BE CONTINUED

Speed control

Of course, in a digital system, you can't easily generate analog outputs ... thus we'll make use of PWM signals. Integration of these PWM signals by the motor windings will produce a sinusoidal electric current.

TO BE CONTINUED

You'll make use of PWM control to adjust the speed of the brushless motor. Additionally, there will be a ramp UP and ramp DOWN effect to avoid high pulses of energy, exemple:

- motor is at stop and the speed control is set at its maximum ... then you'll smoothly increase the speed up to the specified value.

BLDC controller

By means of a FPGA, we'd like you to implement the logical part of the controller as a component. Unfortunately, this kind of controller requires a LOT of parameters ... but to ensure your success as a first step undertaking such a device, we'll restrain ourselves to the following generic parameters:

- **MAX_CPT** → defines a whole phase cycle expressed as number of ticks of the main clock

TO BE CONTINUED

- CLK Frequency : la fréquence d'horloge fixée à 1 000 000 Hz.
- PHASE Frequency : fixée à 50 Hz.
- DUTY SIZE : taille du vecteur de la duty. Dans notre cas, il est fixé à 8 bits permettant des

valeurs de Duty entre 0 et 255.

— MIN DUTY PERCENT : entre 0 et 100% : ici fixée à 50 sous demande de l'enseignant.

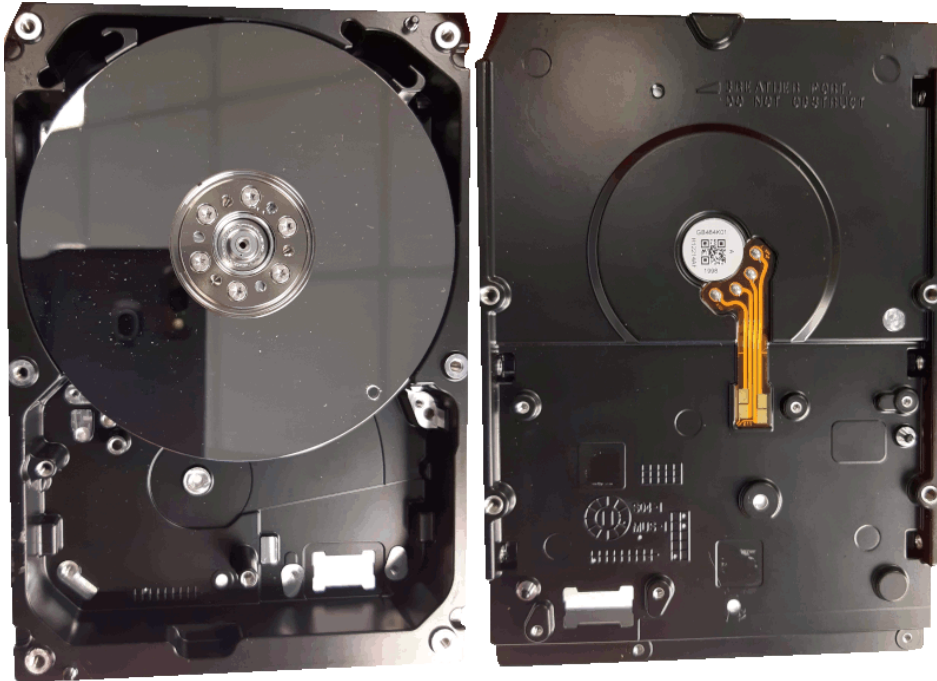
TO BE CONTINUED

As an example, let's say our main clock has a 1MHz frequency. On the other end, our motor requires a 50Hz phase cycle $\Rightarrow$ MAX_CPT will get set at (1MHz/50) at instantiation (or synthesis) time.

to be continued

Use case

In order to avoid destroying one of our electrical vehicles (!), we'll make use of specifically prepared hard drives for such an experiment.



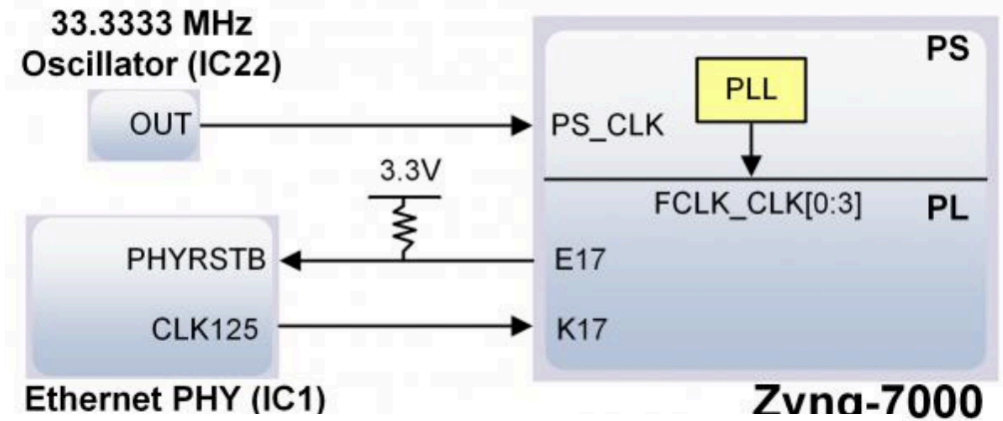
to be continued

Links

[Elektor] BLDC newbies guide

<https://www.elektormagazine.fr/articles/contr%C3%B4le-des-moteurs-bldc-guide-du-d%C3%A9butant>

Master 2



Links

VHDL base files

<https://secil.univ-tlse3.fr/teaching/francois/UE-VHDL/M2/>

Zybo-Z7 reference manual

<https://secil.univ-tlse3.fr/teaching/francois/UE-M2-VHDL/ZYBO-Z7-reference-manual-B.pdf>

Zybo-Z7 schematic

https://secil.univ-tlse3.fr/teaching/francois/UE-M2-VHDL/ZYBO-Z7_schematic-D1.pdf

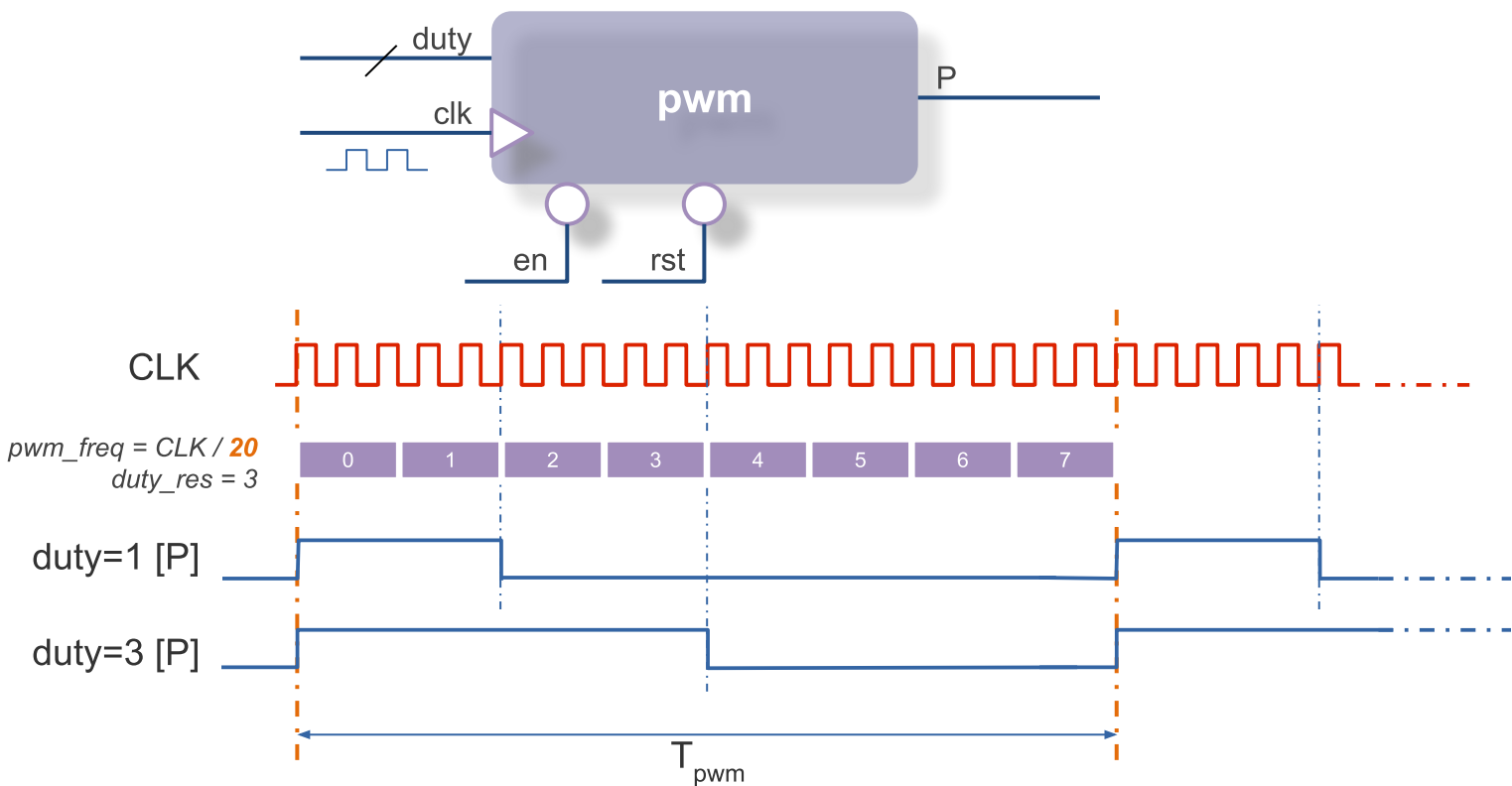
[M2] PWM

As a first step, we'll implement a PWM module that will get connected to a 125MHz clock generator located on our Zybo-Z7. The outcome will be a 1s variable pulse width driving a led.

To help you starting, you'll need the following files:

<https://secil.univ-tlse3.fr/teaching/francois/UE-VHDL/M2/pwm.vhd>

https://secil.univ-tlse3.fr/teaching/francois/UE-VHDL/M2/test_pwm.vhd



The `duty=0` corner case: since our PWM features an EN pin, we'll consider `duty=0` as an active slot.

Behavioural simulation

You'll need to follow the [TP1 - Pulse generator](#) as a guideline:

- select Zybo-Z7 20 board
- add `pwm.vhd` component file + `test_pwm.vhd` simulation file
- complete these files
- launch behavioural simulation

Note: to launch the simulation, don't forget to re-enable generic map along with the behaviour architecture.

Synthesis and Implementation

Now you'll follow the [TP2 - Pulse generator synthesis](#) as a guideline:

Add the 'pwm_synth.pre.tcl' script to the synthesis of your design:

- pwm_synth.pre.tcl

```
# set generic parameter for synthesis
set_property generic {SYS_CLK=125000000 PWM_FREQ=1 DUTY_RES=4} [current_fileset]

# set generic parameters intended to post-synthesis simulation pwm_clk = sys_clk / 20
#set_property generic {SYS_CLK=125000000 PWM_FREQ=6250000 DUTY_RES=4} [current_fileset]
```

Synthesis output will exhibits:

```
-----
Starting RTL Elaboration : Time (s): cpu = 00:00:03 ; elapsed = 00:00:04 . Memory (MB):
peak = 2810.660 ; gain = 0.000 ; free physical = 16041 ; free virtual = 37790
-----
INFO: [Synth 8-638] synthesizing module 'pwm' [/home/devel/teaching-vhdl/VHDL/pwm.vhd:40]
Parameter sys_clk bound to: 125000000 - type: integer
Parameter pwm_freq bound to: 1 - type: integer
Parameter duty_res bound to: 4 - type: integer
INFO: [Synth 8-256] done synthesizing module 'pwm' (0#1)
[/home/devel/teaching-vhdl/VHDL/pwm.vhd:40]
```

Note: SYS_CLK=125E06 will be considered a string ... without issue in the upcoming steps 😊

Post-synthesis simulation

Please pay attention to the fact that being a top-level design, our 'pwm' module won't be able to make use of VHDL **generic** parameters.

Hence, we need to remove the *generic map* part of the 'test_pwm.vhd' file. Please pay attention to the 'structure' architecture now in use.

- test_pwm.vhd

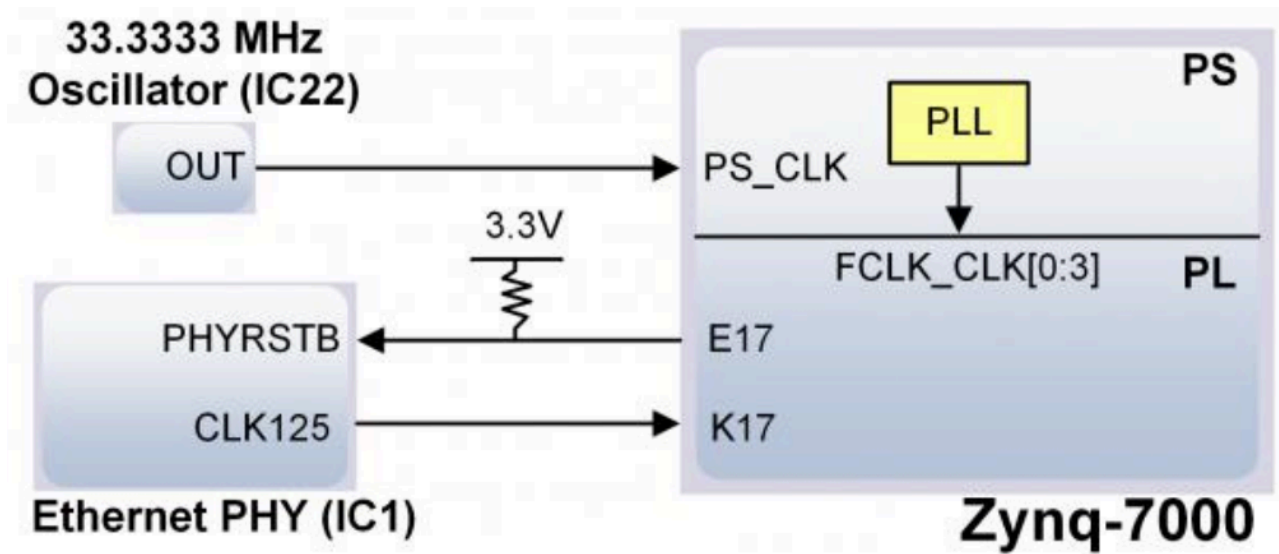
```
-----
-- instantiation et mapping du composant registres
--pwm0 : entity work.pwm(behaviour) --behavioural simulation
--      generic map (sys_clk => sys_freq,
--                  pwm_freq => pwm_freq,
--                  duty_res => duty_res)
pwm0 : entity work.pwm(structure) --post-synthesis functional simulation
      port map (CLK => E_CLK,
                RST => E_RST,
                EN => E_EN,
                DUTY => E_DUTY,
                P => E_P);
-----
```

In addition, you may consider using **alternate generic values** for post-synthesis simulation, see pwm_synth.pre.tcl.

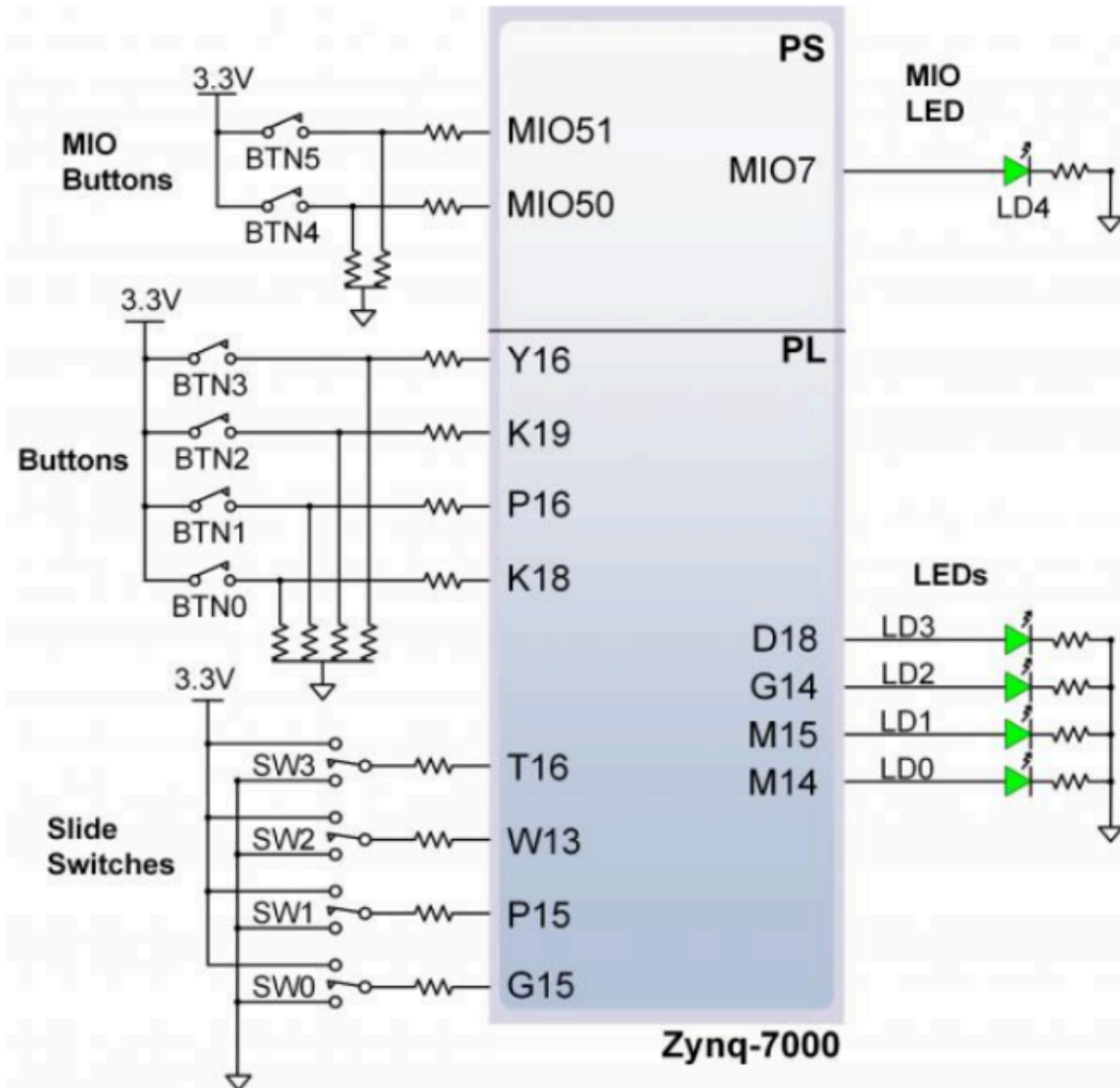
IO planning (implementation)

It's now time to start the **Implementation** (i.e IO planning):

- connect the onboard 125MHz clock to our CLK input



- connect the 4 slides switches SW[3..0] as the DUTY input bus,
- connect the BTN0 as the EN input (active low ---look at the pull-down resistors!)
- connect the 4 leds as the P output
- RST signal will get tied to '1' RST signal will get tied to BTN1 (pull-down resistor!)



See [TP3 - Pulse generator constraints](#) as a guideline:

Launch *Synthesis* → *Open Synthesized design* → *Constraint Wizard* to create the `pwm.xdc` constraint file:

- `pwm.xdc`

```
# Master Clock timing constraint
create_clock -period 8.000 -name CLK -waveform {0.000 4.000} [get_ports CLK]
set_property PACKAGE_PIN K17 [get_ports CLK]
set_property IOSTANDARD LVCMOS33 [get_ports CLK]

set_property PACKAGE_PIN K18 [get_ports EN]
set_property IOSTANDARD LVCMOS33 [get_ports EN]
set_property PACKAGE_PIN P16 [get_ports RST]
set_property IOSTANDARD LVCMOS33 [get_ports RST]
set_property PACKAGE_PIN G15 [get_ports {DUTY[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {DUTY[0]}]
```

```

set_property PACKAGE_PIN P15 [get_ports {DUTY[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {DUTY[1]}]
set_property PACKAGE_PIN W13 [get_ports {DUTY[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {DUTY[2]}]
set_property PACKAGE_PIN T16 [get_ports {DUTY[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {DUTY[3]}]

set_property PACKAGE_PIN D18 [get_ports P]
set_property IOSTANDARD LVCMOS33 [get_ports P]
    
```

Note: best is to first create this XDC file through the Wizard ... then to modify it on your own

TODO: set RST as '1'

TODO: add debug core to our design

Bitstream generation

Select 'impl_1' (i.e implementation), click the green RUN icon, then continue with *Bitstream generation*

Tcl Console

Messages

Log

Reports

Design Runs

×

🔍

⌵

⌶

⏮

⏪

▶

⏩

⏭

🔍

Name	Constraints	Status	LUT	FF	BRAM	URAM	DSP
✓ synth_1	constrs_1	synth_design Complete!	56	54	0	0	0
✓ impl_1	constrs_1	write_bitstream Complete!	48	54	0	0	0

Hardware Manager

This is the main application responsible for establishing a dialog with the hardware ... that will enable you to upload or readback a bitstream.

Open Hardware Manager → Open Target → auto connect

PROGRAM AND DEBUG

- Generate Bitstream
- Open Hardware Manager**
 - Open Target
 - Program Device
 - Add Configuration Memory D

Hardware

Name	Status
localhost (1)	Connected
xilinx_tcf/Digilent/210351B481D8	Open
arm_dap_0 (0)	N/A
xc7z020_1 (1)	Programmed
XADC (System Monitor)	

[optional] Bitstream readback

In order to backup a readable bitstream, use the following command in the TCL console:

```
readback_hw_device [current_hw_device] -bin_file zyboZ7-20_original.bin
```

<https://docs.xilinx.com/r/en-US/ug908-vivado-programming-debugging/Bitstream-Verify-and-Readback-for-FPGAs-and-MPSoCs>

Download bitstream

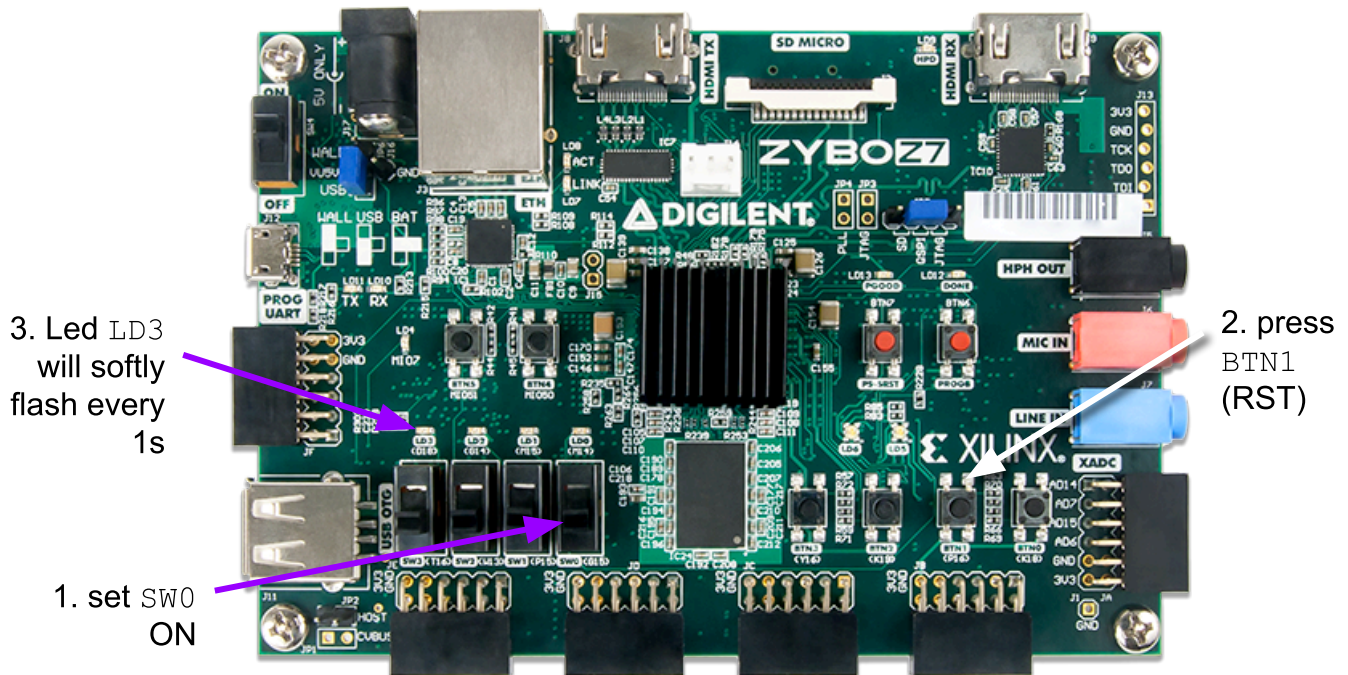
Now you just need to click on 'Program device' and it will select the latest generated bit stream that will get downloaded to the config RAM of your FPGA.

HARDWARE MANAGER - localhost/xilinx_tcf/Digilent/210351B481DEA

There are no debug cores. [Program device](#) [Refresh device](#)

Hardware	
Name	Status
localhost (1)	Connected
xilinx_tcf/Digilent/210351B481DEA	Open
arm_dap_0 (0)	N/A
xc7z020_1 (1)	Programmed
XADC (System Monitor)	

Tests 😊

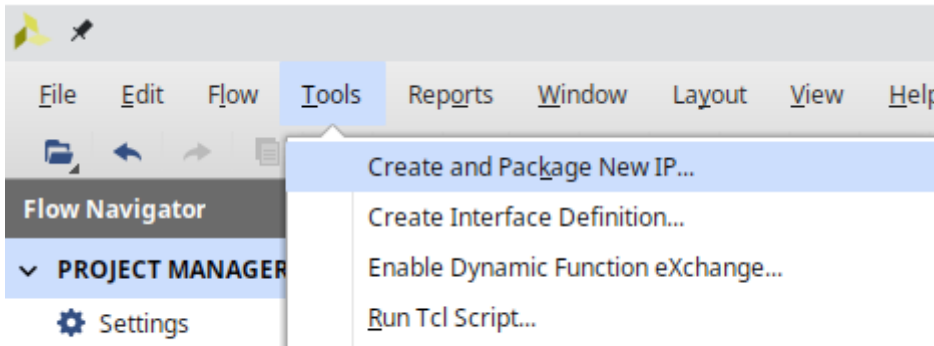


[M2] my AXI-enabled PWM IP²

We'll now turn our PWM to an AXI-enabled PWM IP. This leverages the needs for large systems integration involving the PS part (i.e on board ARM9 processors).

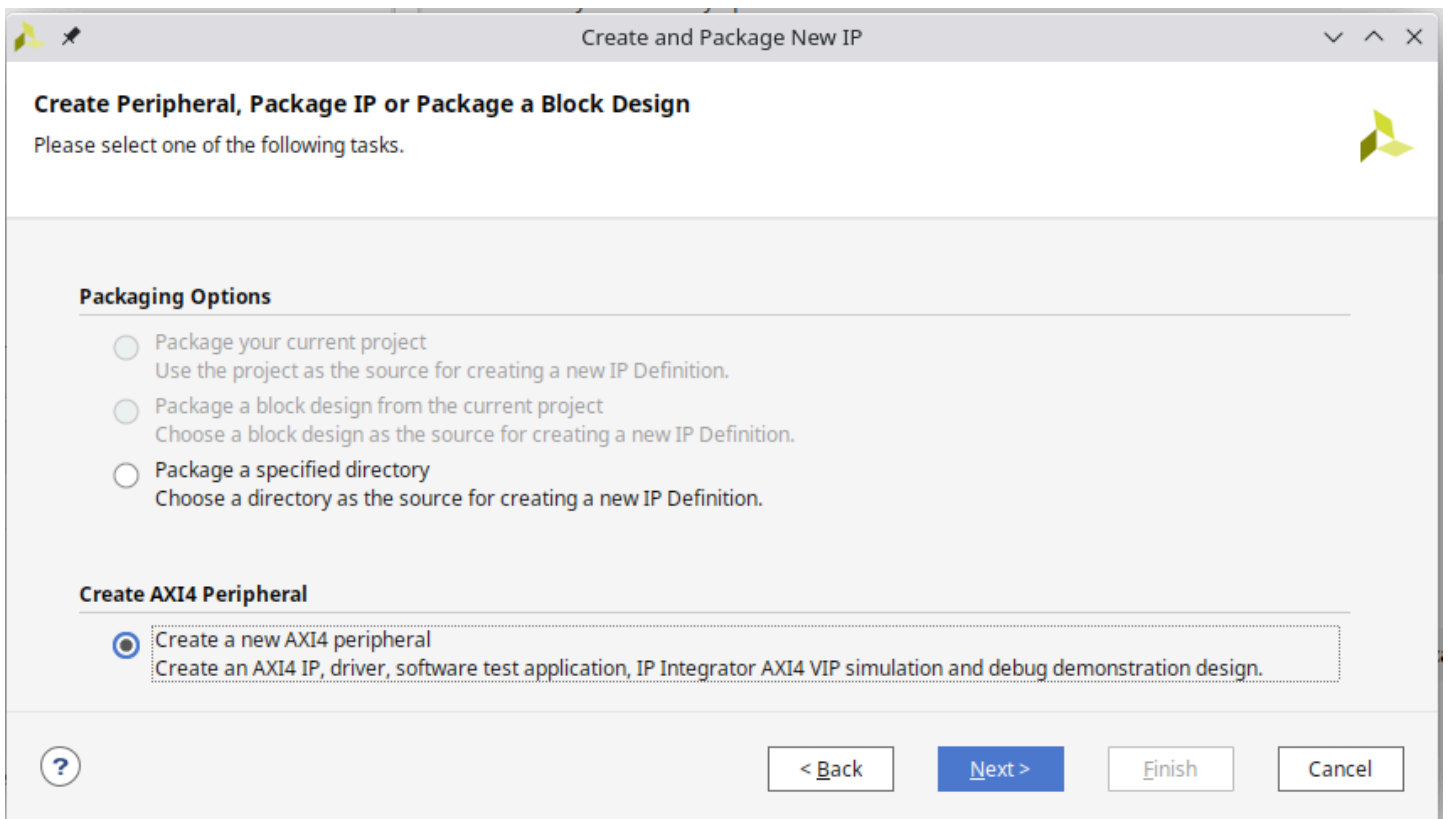
To reach this goal, we'll start creating a new VHDL project without specifying any HDL source code.

Then, we'll start the creation of an AXI component that exhibits the following features:



- AXI4-lite SLAVE
- 32 bits bus width
- 4 registers

Select → *Create a new AXI4 peripheral*



² To help yourself creating an IP, search over the net for "Xilinx IP integrator" then select "Vivado Design Suite Tutorial: Creating and Packaging Custom IP"

Create and Package New IP

Peripheral Details

Specify name, version and description for the new peripheral

Name: myAXIpwm

Version: 1.0

Display name: myAXIpwm_v1.0

Description: AXIpwm

IP location: /home/devel/ip_repo

☐ Overwrite existing

< Back Next > Finish Cancel

Create and Package New IP

Add Interfaces

Add AXI4 interfaces supported by your peripheral

☐ Enable Interrupt Support

Interfaces

- S00_AXI

S00_AXI
myAXIpwm_v1.0

Name: S00_AXI

Interface Type: Lite

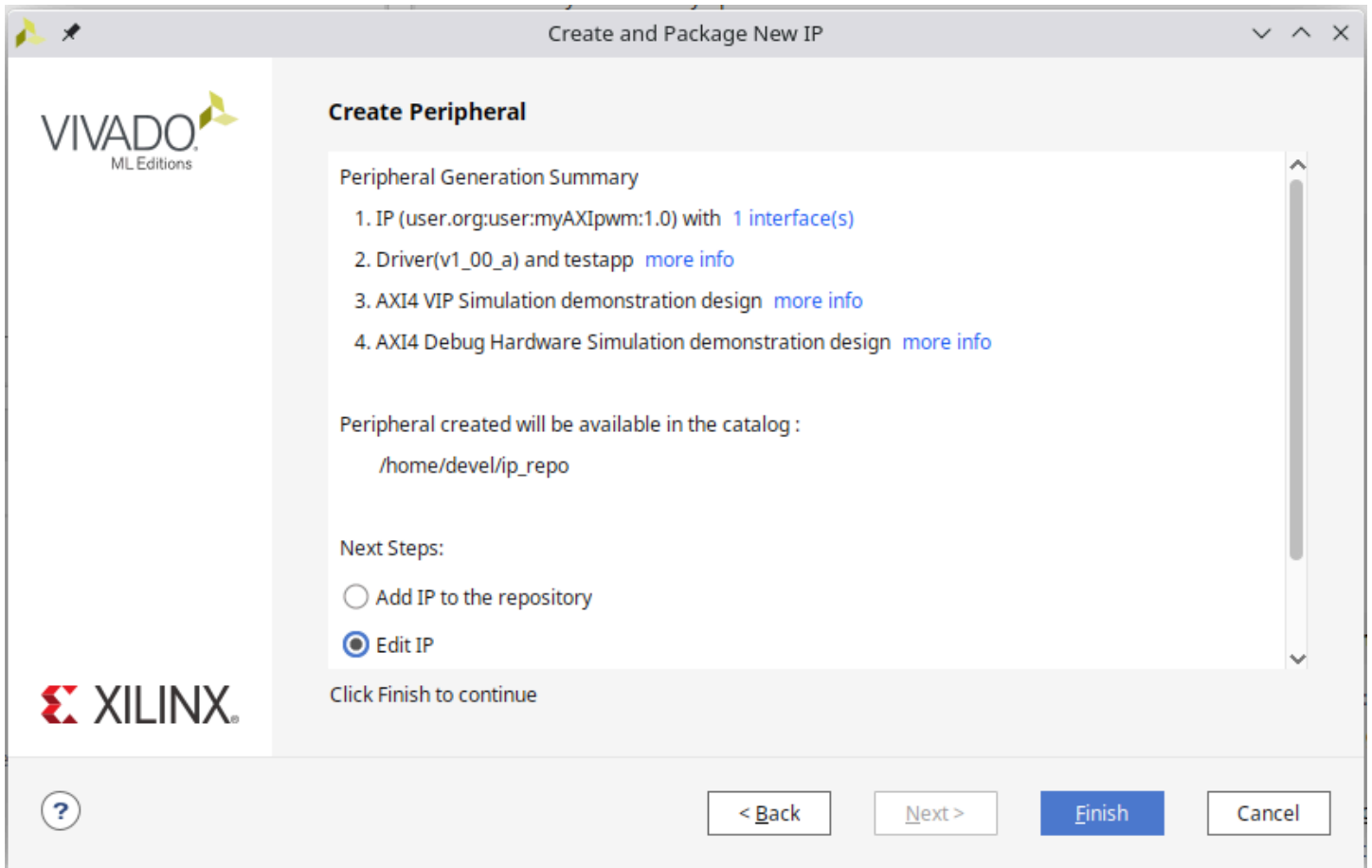
Interface Mode: Slave

Data Width (Bits): 32

Memory Size (Bytes): 64

Number of Registers: 4 [4..512]

? < Back Next > Finish Cancel



Now you can go on with [IP customization](#)



This will dynamically create a new project `<home>/ip_repo/edit_myAXI_pwm_v1_0.xpr`

Links

Digilent Zybo's AXI IP + Vitis baremetal driver

<https://digilent.com/reference/programmable-logic/guides/getting-started-with-ipi>

AXI IP creation

<https://indico.ictp.it/event/a14283/session/62/contribution/252/material/slides/2.pdf>

[Xilinx] Vivado Design Suite Tutorial: Creating and Packaging Custom IP (UG1119)

HOW TO CREATE an AXI4-FULL CUSTOM IP with AXI4-LITE and UART INTERFACES in VIVADO

<https://www.mehmetburakaykenar.com/how-to-create-an-axi4-full-custom-ip-with-axi4-lite-and-uart-interfaces-in-vivado/192/>

Fixing Xilinx's Broken AXI-lite Design in VHDL

<https://zipcpu.com/blog/2021/05/22/vhdlaxil.html>

[sample] https://github.com/ZipCPU/wb2axip/blob/master/bench/formal/xlnxdemo_2020_2.vhd

Create Custom AXI Cores Part 1: Straight to the Finish Line

<https://www.hackster.io/cospan/create-custom-axi-cores-part-1-straight-to-the-finish-line-a70e5e>

Designing a Custom AXI IP on Vitis

<https://www.hackster.io/pablotrujillojuan/designing-a-custom-axi-ip-on-vitis-a0ad06>

Editing an IP from main project



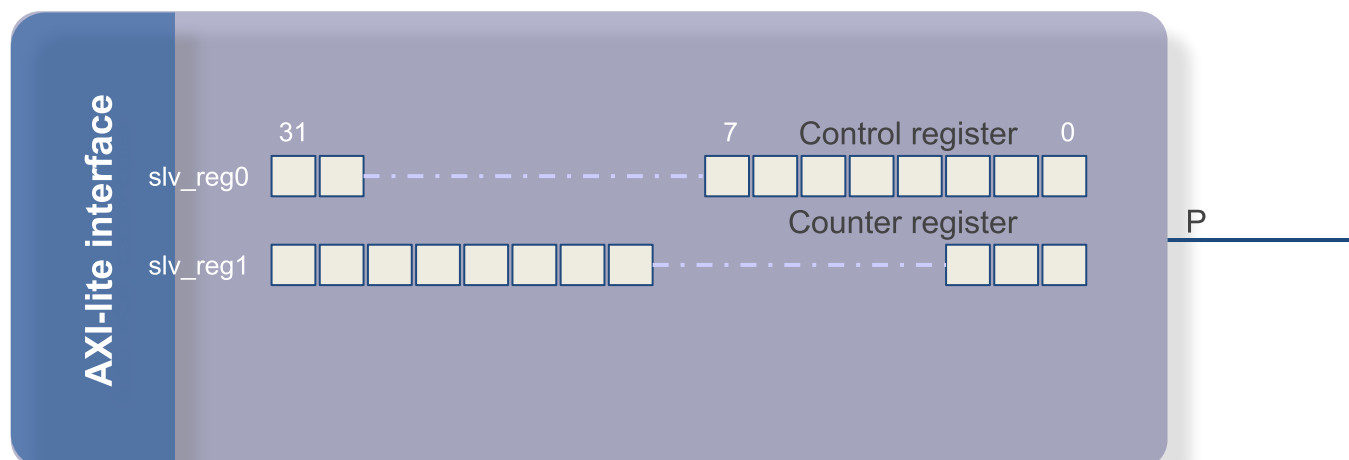
Editing an IP means it will **launch a new project** embedded within your current project.

To start editing your IP from your main project, click on *IP catalog* then select your IP + right click then select *Edit in IP Packager*

The screenshot shows the Vitis Project Manager interface. On the left, the 'Flow Navigator' pane has 'PROJECT MANAGER' expanded, with 'IP Catalog' circled in red. The main 'PROJECT MANAGER' pane shows a tree view of the project 'zyboZ7-AXI-pwm'. The 'Cores' tab is active, displaying a list of cores. A search bar is present. The core 'myAXIpwm_v1.0' is selected, and a right-click context menu is open over it. The menu options include 'Properties...', 'IP Settings...', 'Add Repository...', 'Refresh All Repositories', 'Customize IP...', 'Edit in IP Packager' (highlighted in blue), 'Disable IP', and 'Delete IP' (marked with a red X). A pink arrow points from the 'Edit in IP Packager' option to the 'myAXIpwm_v1.0' core in the tree.

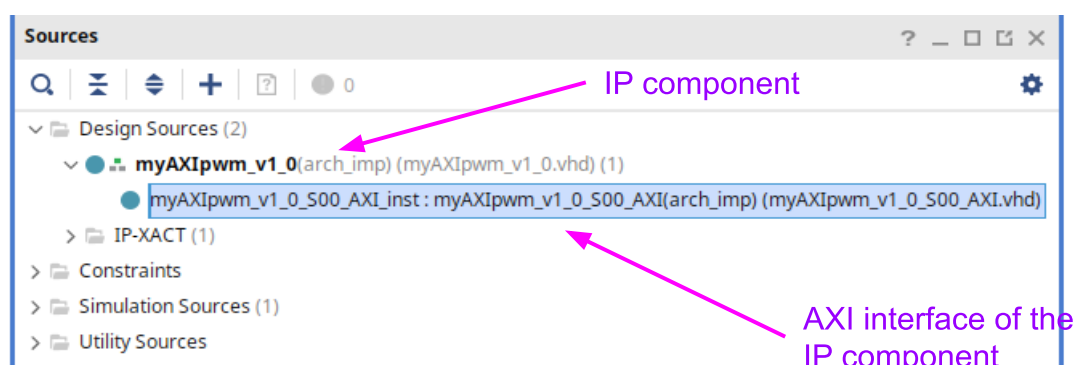
IP customization

Roughly speaking, IP customization is a packaging of our previously designed PWM component.

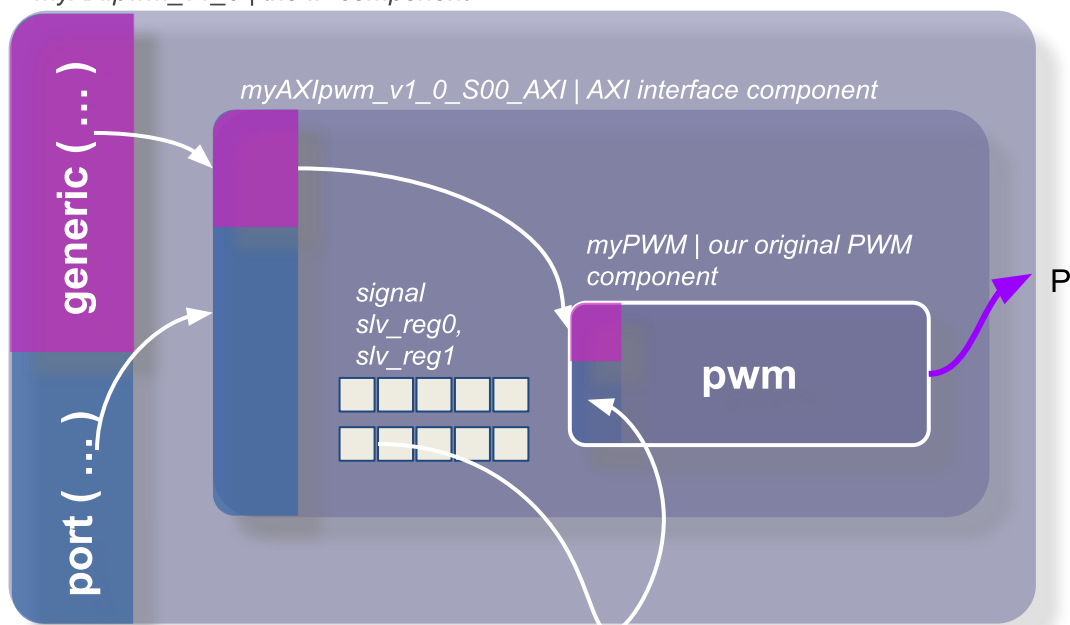


In the main window of the newly created project, you can see two files:

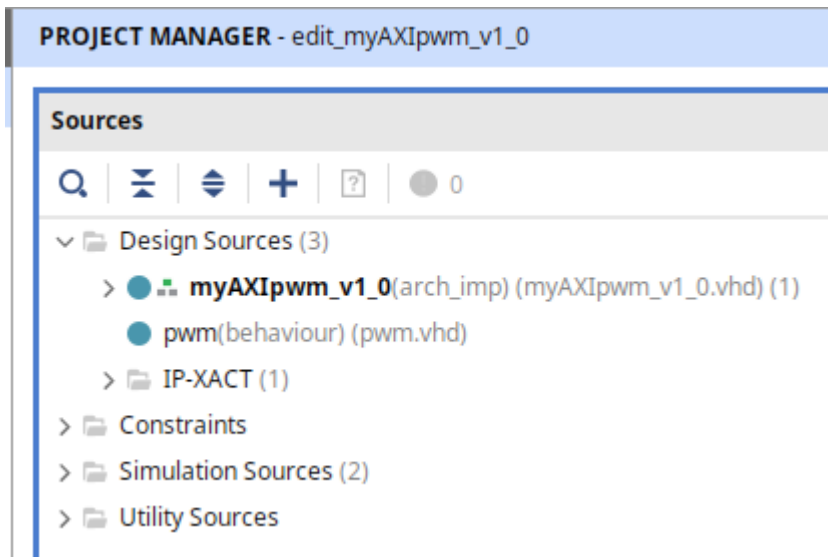
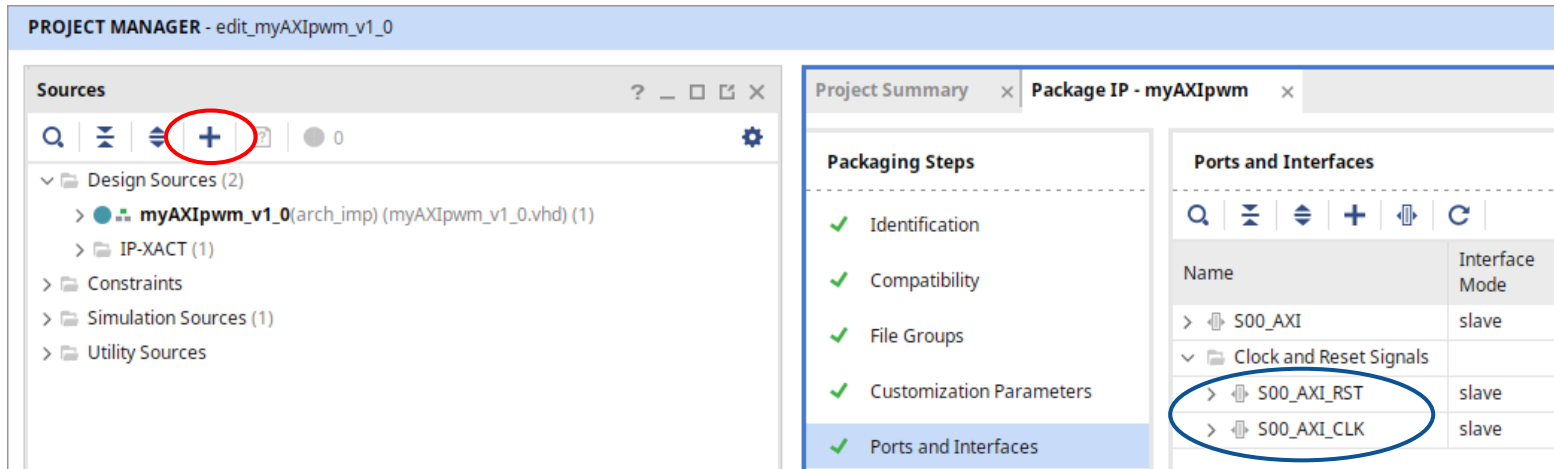
- the IP component `myAXIpwm_v1_0`
- `myAXIpwm_v1_0_S00_AXI` the AXI slave interface **component**, part of the IP component itself



`myAXIpwm_v1_0` | the IP component



Now we'll add the source code of your previously designed PWM component:

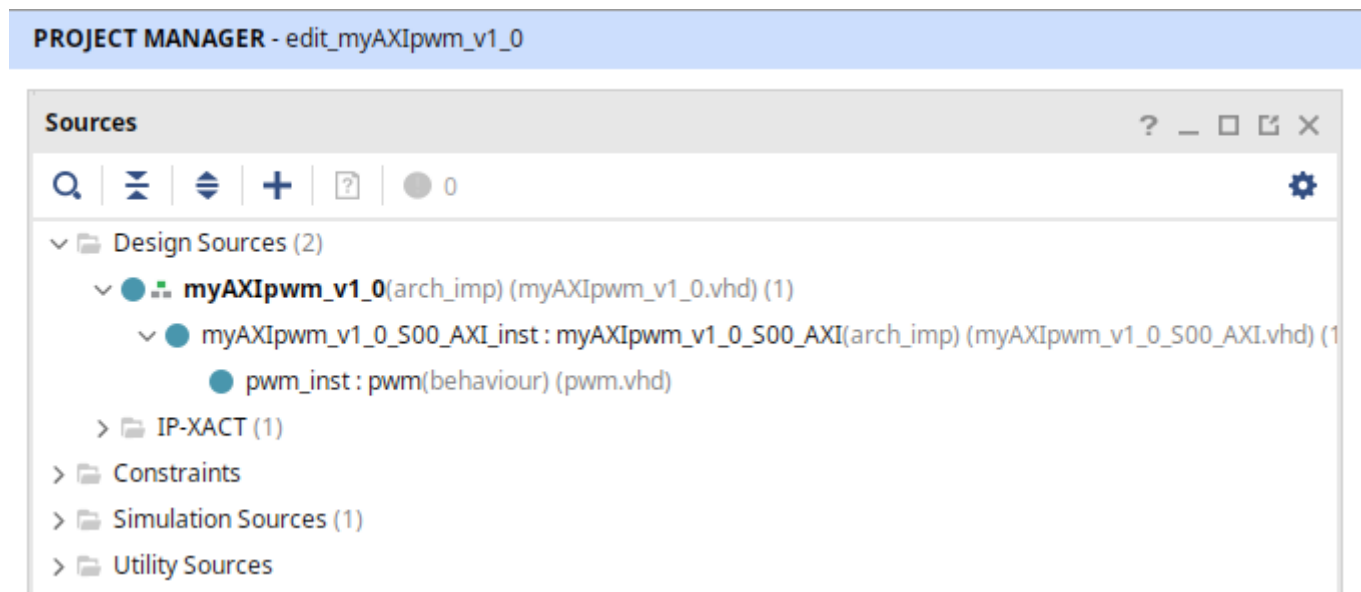


It's now time to integrate an instance of our original PWM within the AXI_S00 instance and to map ports + generic parameters ... and extend AXI_S00_inst accordingly.

Finally, extend both generic parameters and ports of the top-level IP and map them to the AXI_S00 instance.

Add the various generic parameters as editable with a proper range of values.

In the end, you'll see your pwm_inst integrated within the AXI interface instance



The screenshot shows the Vivado IP Customization GUI for the 'myAXIpwm' component. The top tabs include 'Project Summary', 'Package IP - myAXIpwm', 'myAXIpwm_v1_0.vhd', 'myAXIpwm_v1_0_S00_AXI.vhd', and 'pwm.vhd'. The left sidebar shows 'Packaging Steps' with 'Customization GUI' selected. The main area is divided into 'Layout' and 'Preview'.

Layout:

- ☐ Window
 - ⚙️ Component Name
 - ▼ Page 0
 - ⚙️ Sys Clk
 - ⚙️ Pwm Freq
 - ⚙️ Duty Res
 - ⚙️ C S00 AXI DATA WIDTH
 - ⚙️ C S00 AXI ADDR WIDTH
 - ⚙️ C S00 AXI BASEADDR
 - ⚙️ C S00 AXI HIGHADDR
- ☐ Hidden Parameters

Preview:

how disabled ports

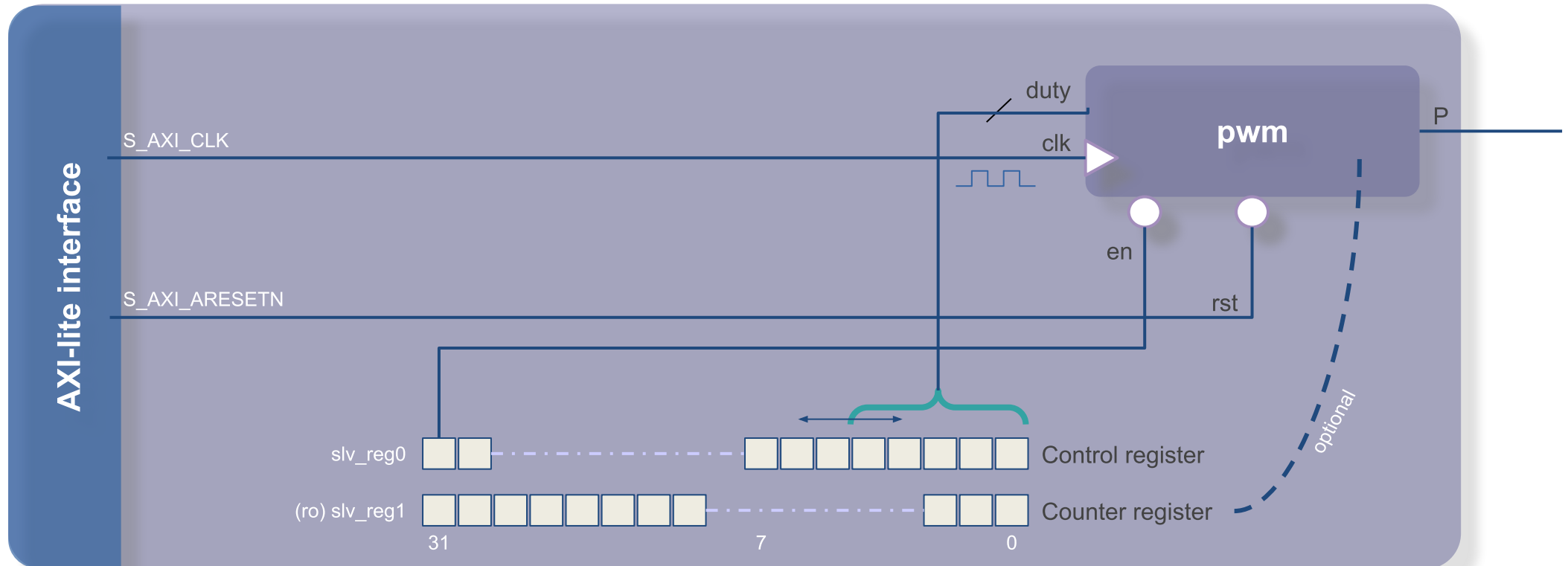
Component Name: myAXIpwm_0

Sys Clk	1000000
Pwm Freq	100000
Duty Res	8
C S00 AXI DATA WIDTH	32
C S00 AXI ADDR WIDTH	4
C S00 AXI BASEADDR	0xFFFFFFFF
C S00 AXI HIGHADDR	0x00000000

Diagram showing the S00_AXI block with ports: s00_axi_aclk, s00_axi_aresetn, and P.

In order to help yourself: have a look to VHDL 'AXI' labelled files already available for download (see [Links](#))

Here are the details of the mapping between our AXI registers and our PWM component.
It's now time to complete the modification of our AXI interface.



[optional] AXI simulation with the AXI VIP

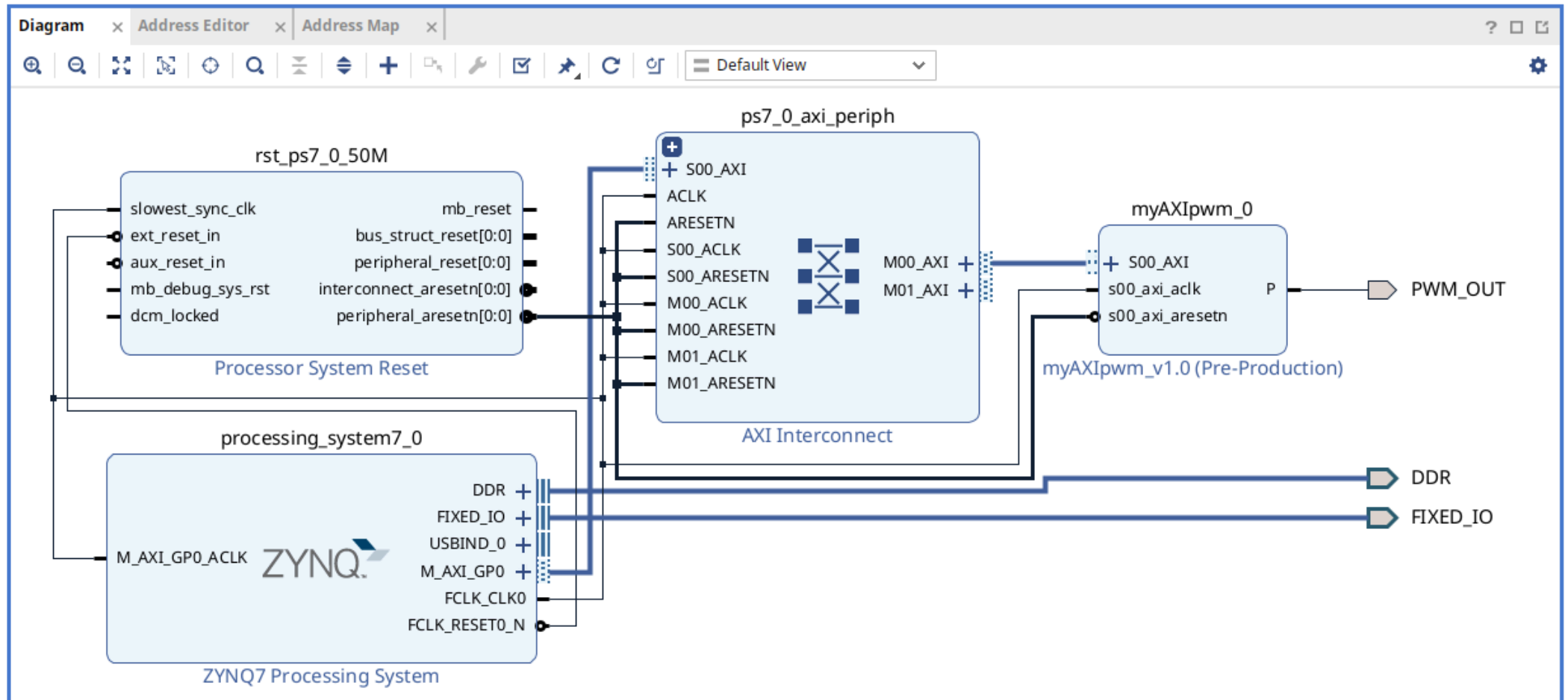
In order to enable a simple testing of your AXI-enable PWM IP, we'll make use of the AXI verification IP
https://support.xilinx.com/s/article/1058302?language=en_US

to be continued

Block design

It's now time to create a *Block Design* featuring the PS part (i.e Dual ARM9 CPU) and your AXI-enabled IP.

In the end, you'll need to *Create Port* (right-click in design) that will be named 'PWM_OUT': connect it with your PWM output.



Address Path Properties			
/myAXIpwm_0/S00_AXI			
Name	Base Address	Range	High Address
Source			
/processing_system7_0			
/processing_system7_0/M_AXI_GP0	0x4000_0000	1 G	0x7FFF_FFFF
Apertures			
/processing_system7_0/M_AXI_GP0	0x4000_0000	1 G	0x7FFF_FFFF
Connections			
/ps7_0_axi_periph/s00_couplers/auto_pc			
/ps7_0_axi_periph/s00_couplers/auto_pc/S_AXI	0x0	4 G	0xFFFF_FFFF
/ps7_0_axi_periph/s00_couplers/auto_pc/M_AXI	0x0	4 G	0xFFFF_FFFF
/ps7_0_axi_periph/xbar			
/ps7_0_axi_periph/xbar/S00_AXI	0x0	4 G	0xFFFF_FFFF
/ps7_0_axi_periph/xbar/M00_AXI	0x0	4 G	0xFFFF_FFFF
Destination			
/myAXIpwm_0			
/myAXIpwm_0/S00_AXI	0x0	16	0xF

Your PWM registers are set to 0x43C0_0000 (linux mmap later)
Using the *Address Editor*, you can change some parameters like the `range` value.

In the Block Design view, click to configure your myAXIpwm_0:

- sys_clk (50MHz),
- pwm_clk (1Hz),
- duty_res (4)

It's now time to *Validate your design ... then Generate Block Design*

Add a constraint file to map the PWM_OUT to an onboard LED.

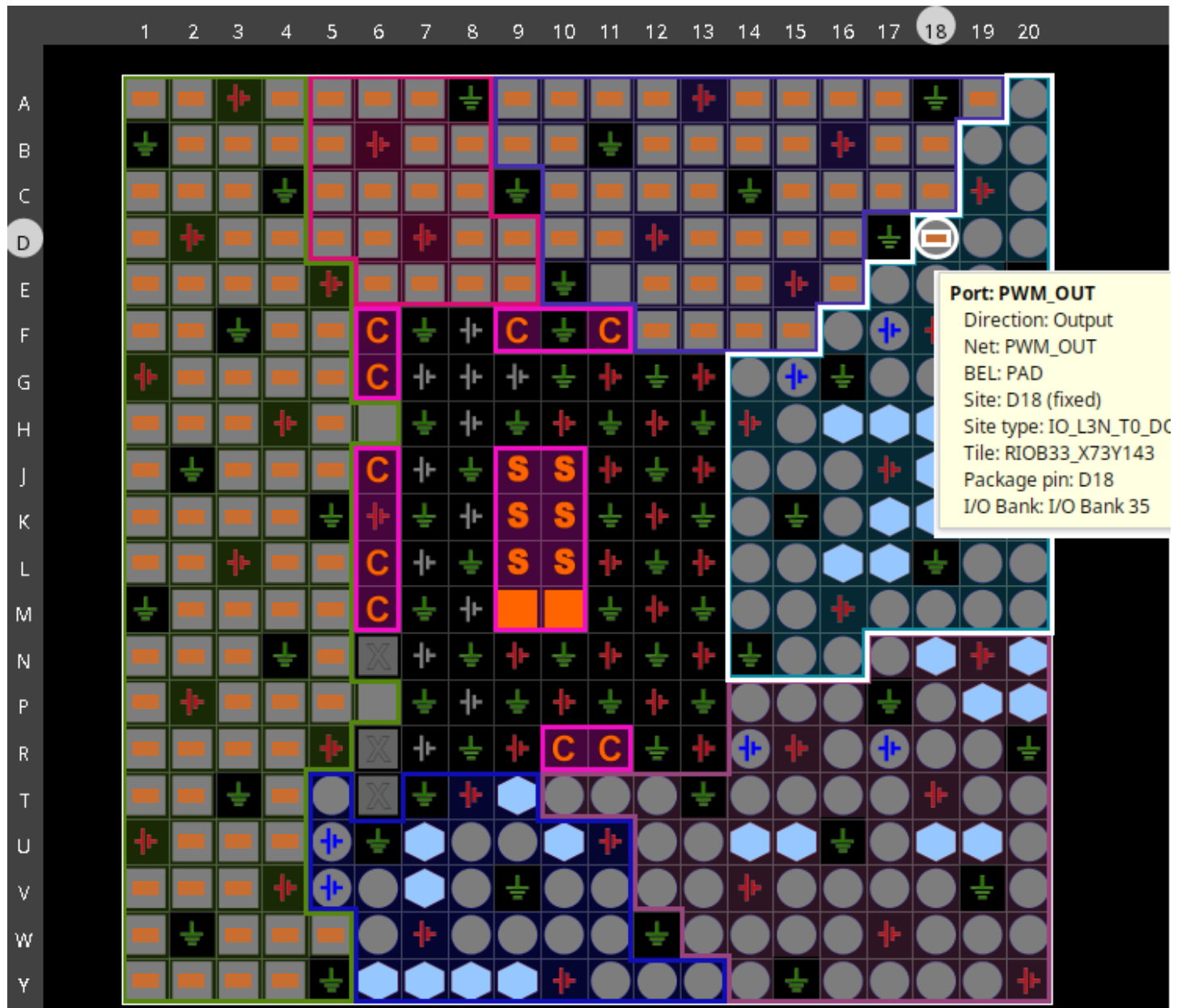
At the *Project Manager*1, *Design Source* right click on myAXIpwmm → add HDL wrapper

... then launch *Generate bitfile* (i.e it will undertake both IP synthesis, whole design synthesis + implementation too)

Diagram x Address Editor x Address Map x design_1.vhd x

☒ Assigned (1)
 ☒ Unassigned (0)
 ☒ Excluded (0)
 ☒ Incomplete (1)
 Hide All

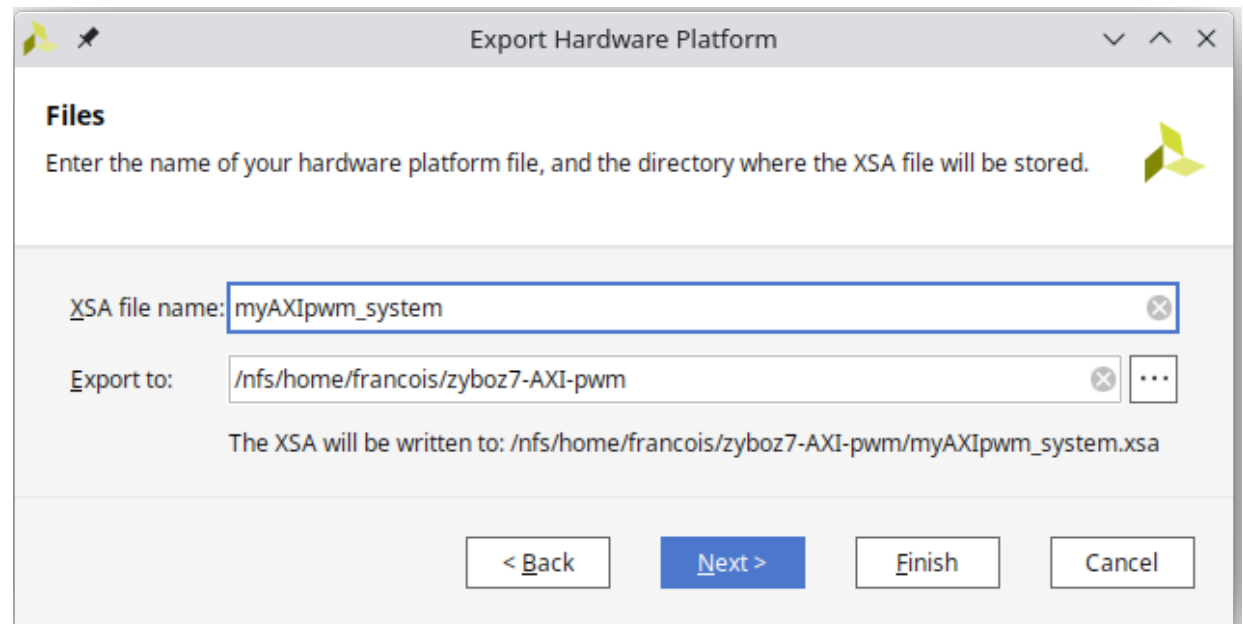
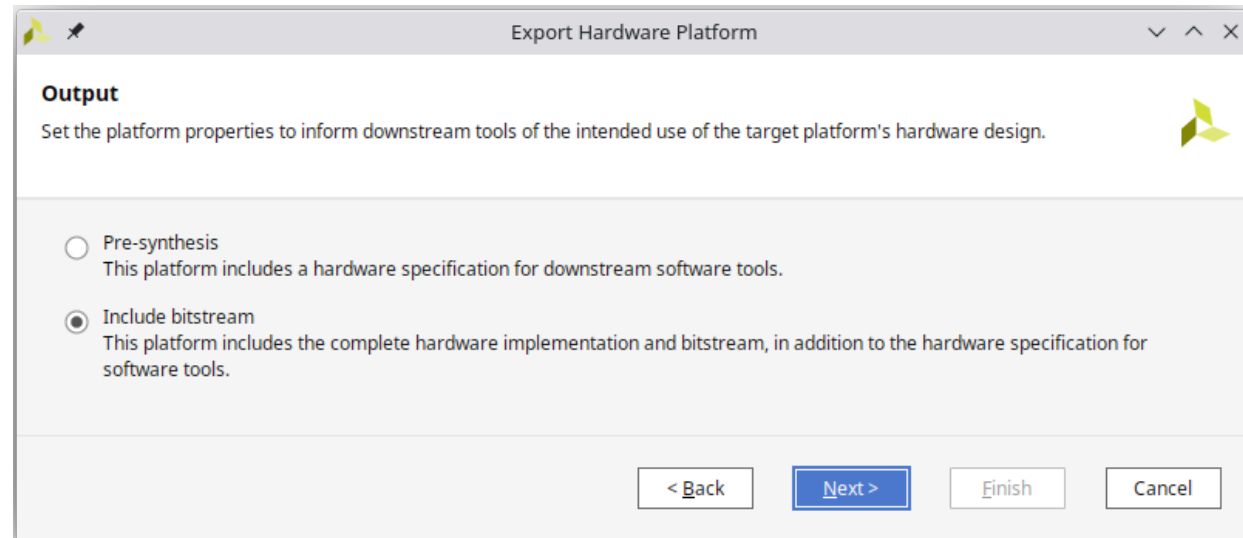
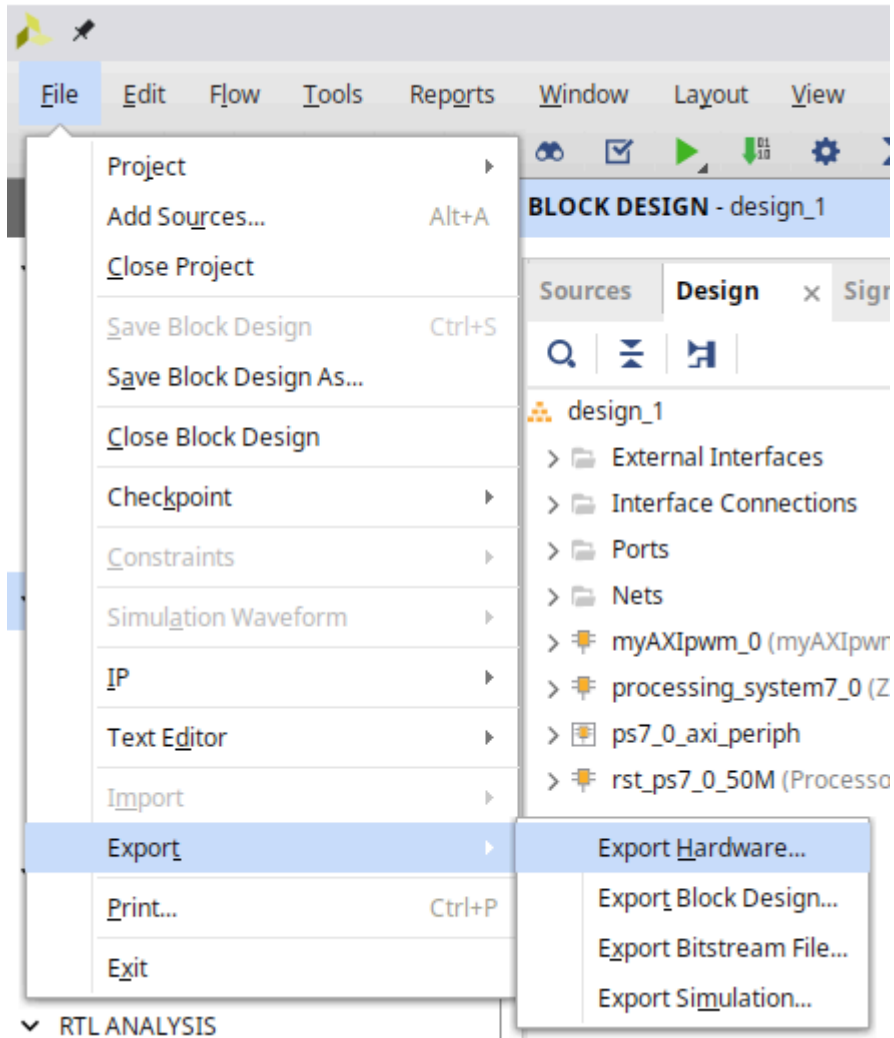
Name	Interface	Slave Segment	Master Base Address	Range	Master High Address
Network 0					
/processing_system7_0					
/processing_system7_0/Data (32 address bits : 0x40000000 [1G])					
/myAXIpwm_0/S00_AXI	S00_AXI	S00_AXI_reg	0x43C0_0000	128	0x43C0_007F
Incomplete Paths (1)					



Bitstream generation and export hardware

As previously undertaken, see [Bitstream generation](#) to generate a '.bit' file.

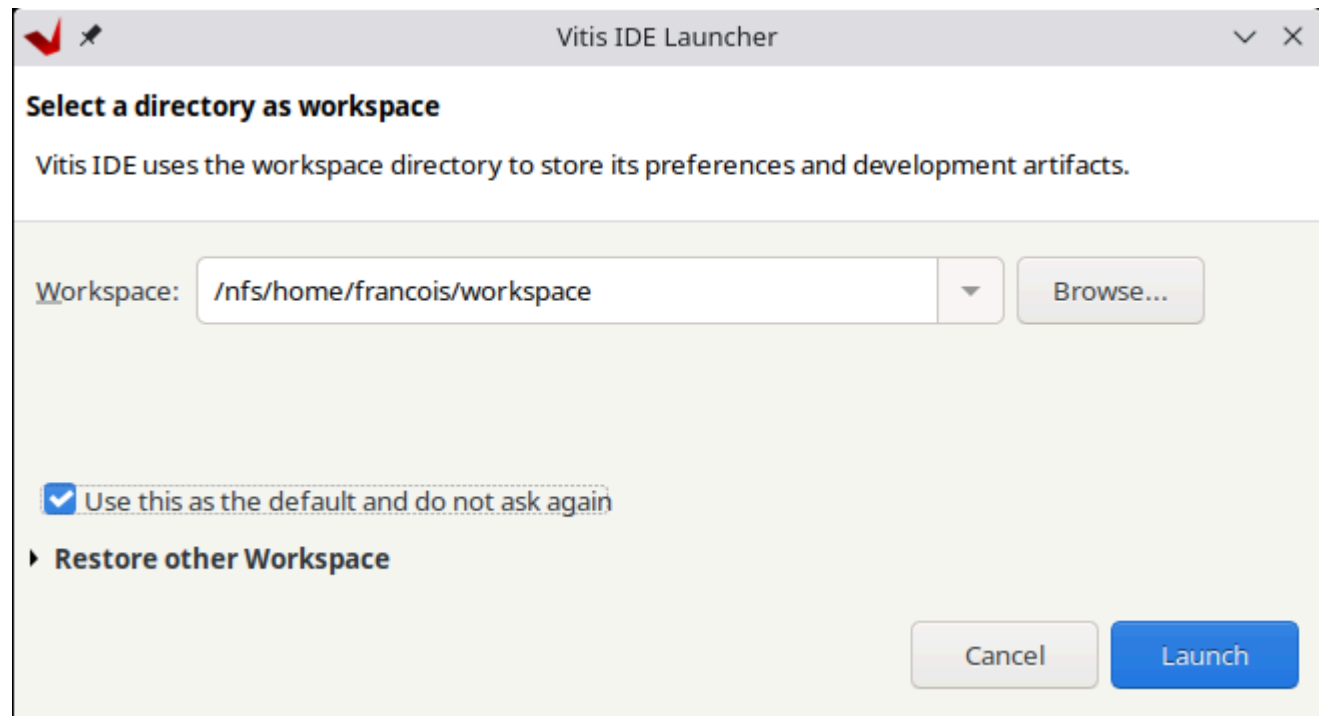
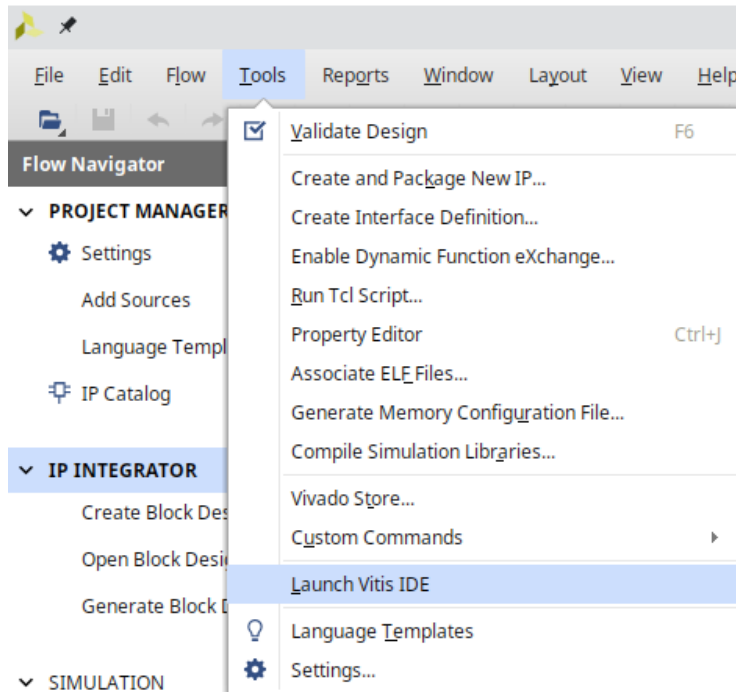
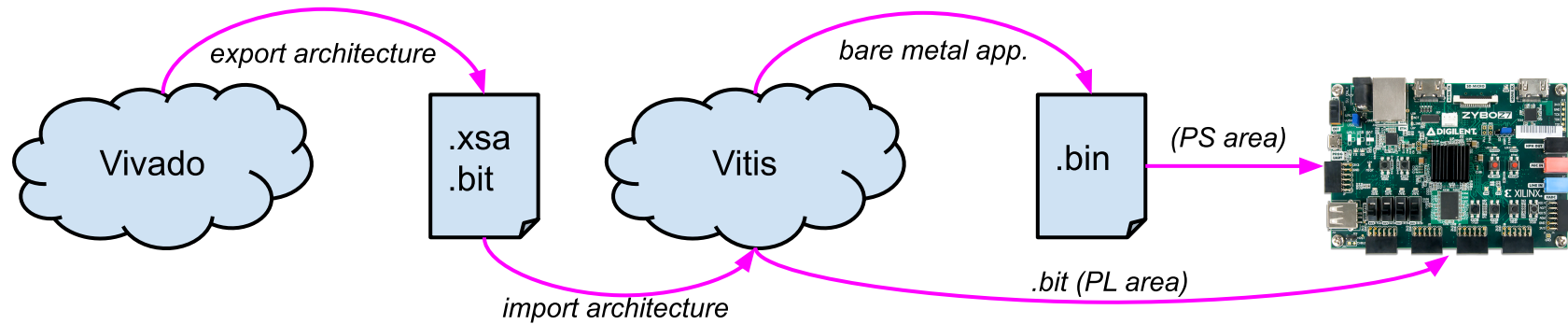
Next step is to Export a hardware description of our global system in order to develop a simple, bare metal, application that will interact with our PWM IP.



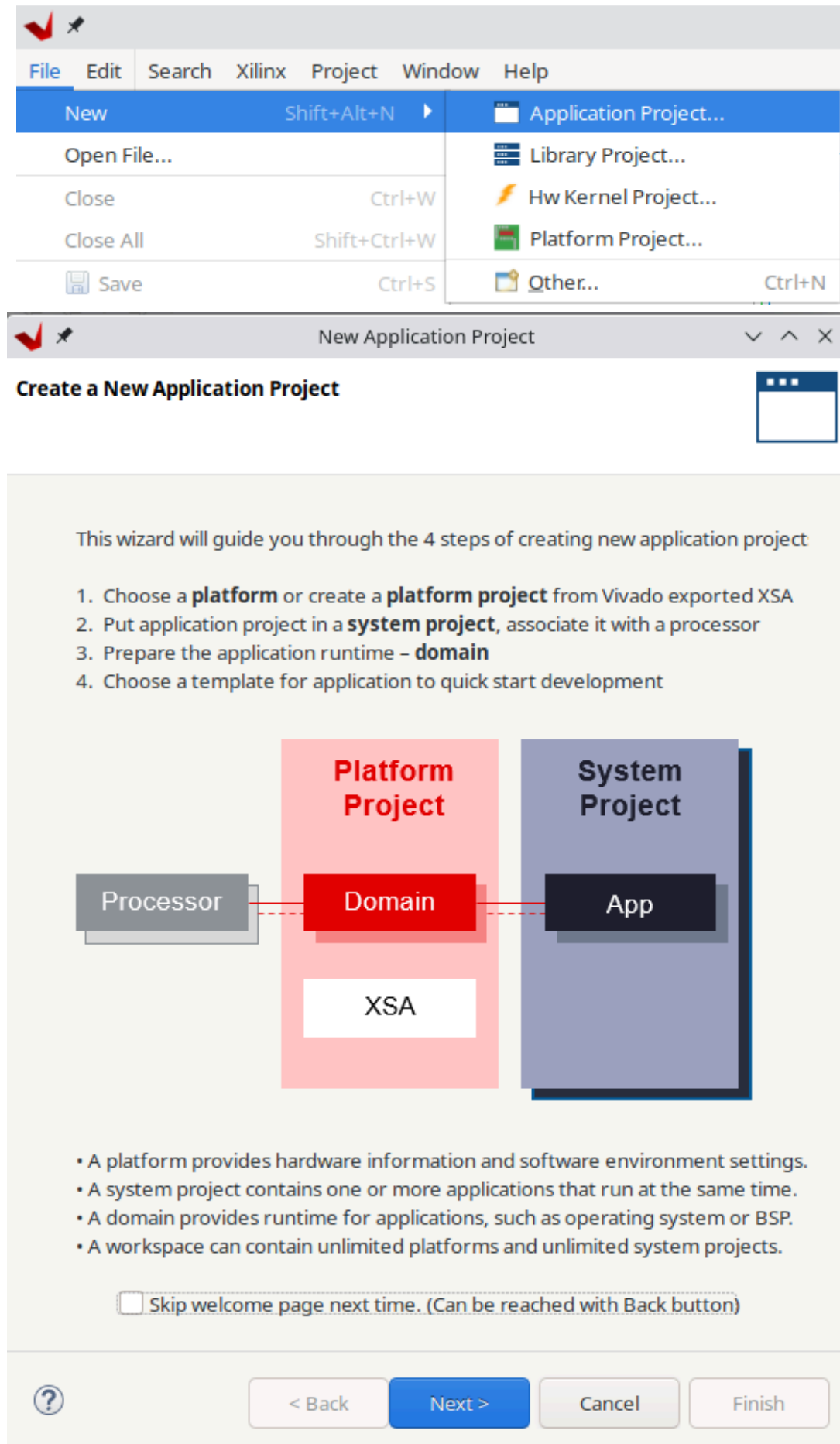
Launch Vitis IDE

At this point, you have a '.xsa' file that describes our whole system featuring the *myPWM* IP block.

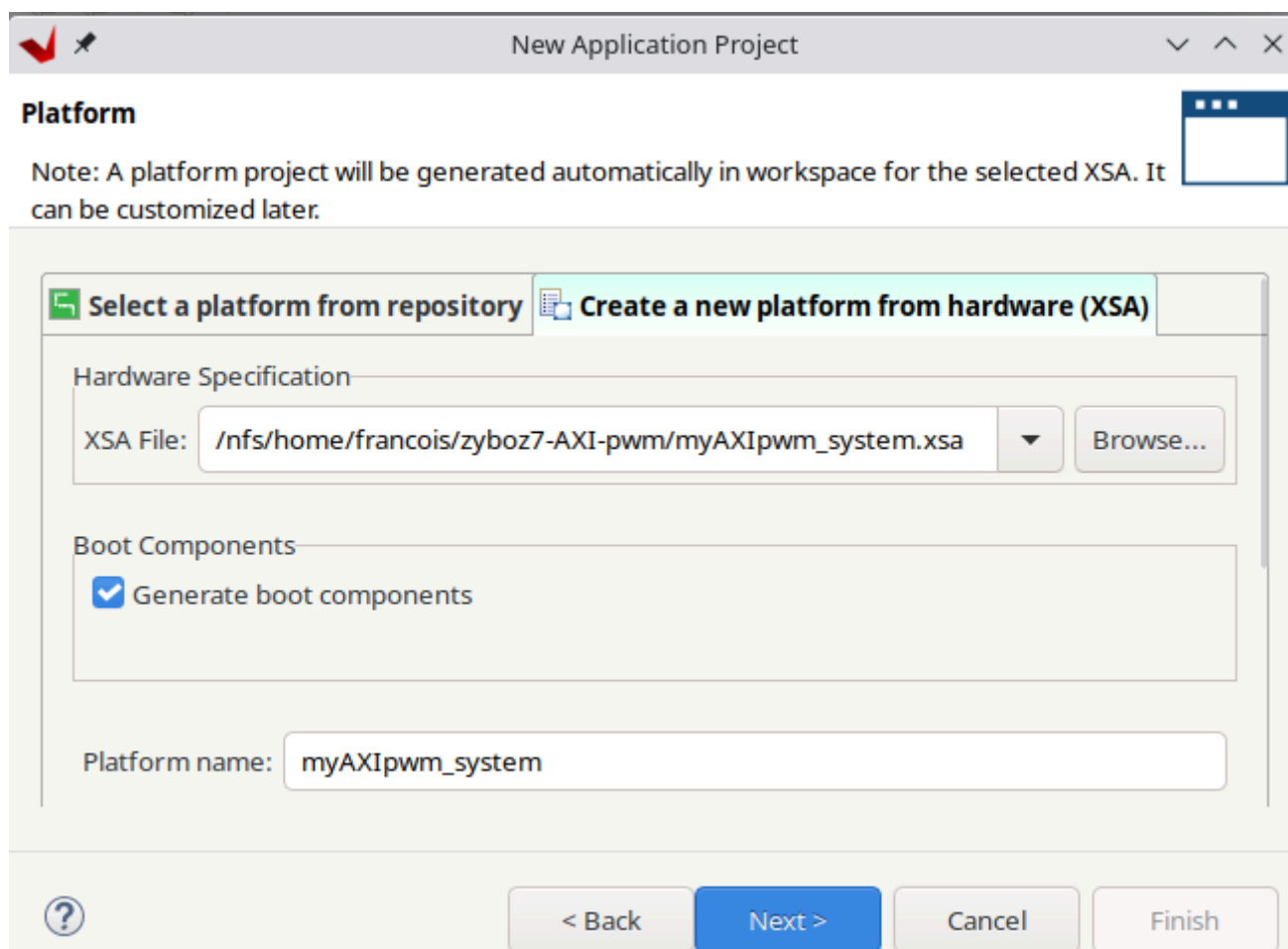
By means of the **Vitis IDE**, we'll import the hardware design to develop a bare metal app. able to interact with our PWM IP on its AXI bus.



Create an 'application project'



You'll then get prompted to create a new platform from your previously generated XSA file.



Platform

Note: A platform project will be generated automatically in workspace for the selected XSA. It can be customized later.

Select a platform from repository **Create a new platform from hardware (XSA)**

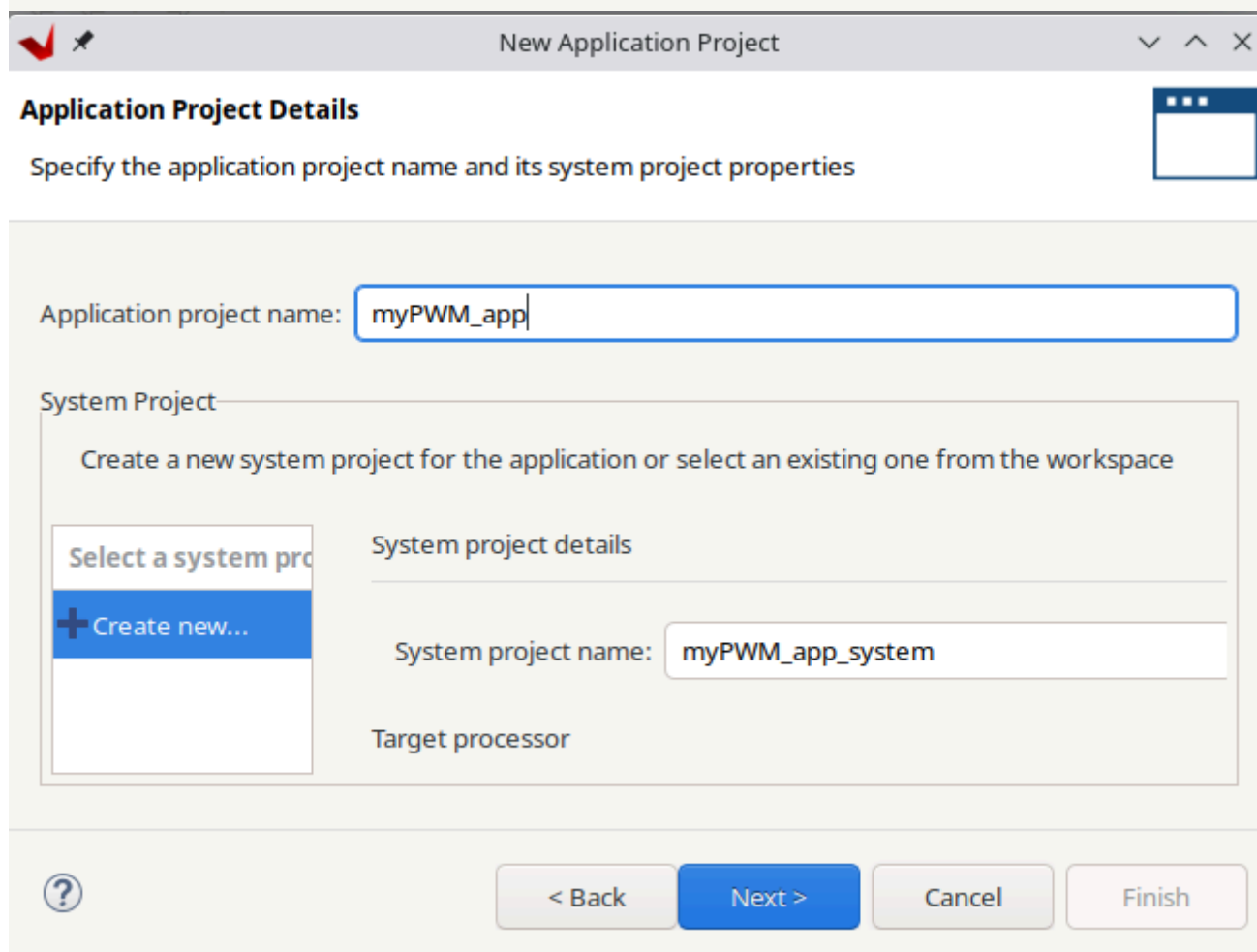
Hardware Specification

XSA File:

Boot Components

☒ Generate boot components

Platform name:



Application Project Details

Specify the application project name and its system project properties

Application project name:

System Project

Create a new system project for the application or select an existing one from the workspace

Select a system project

System project details

System project name:

Target processor

*

Once your application project name has been defined, you'll select 'standalone' OS (i.e bare metal ---no OS) allong with 'Empty C++ app'

Domain

Select a domain for your project or create a new domain

Select the domain that the application would link to or create a new domain

Note: New domain created by this wizard will have all the requirements of the application template selected in the next step

Select a domain

+ Create new...

Domain details

Name: standalone_ps7_cortexa9_0

Display Name: standalone_ps7_cortexa9_0

Operating System: standalone

Processor: ps7_cortexa9_0

Architecture: 32-bit

< Back Next > Cancel Finish

Templates

Select a template to create your project.

Available Templates:

Find:

▼ Embedded software development templates

Dhrystone

Empty Application (C++)

Empty Application(C)

Hello World

lwIP Echo Server

lwIP TCP Perf Client

lwIP TCP Perf Server

lwIP UDP Perf Client

lwIP UDP Perf Server

Memory Tests

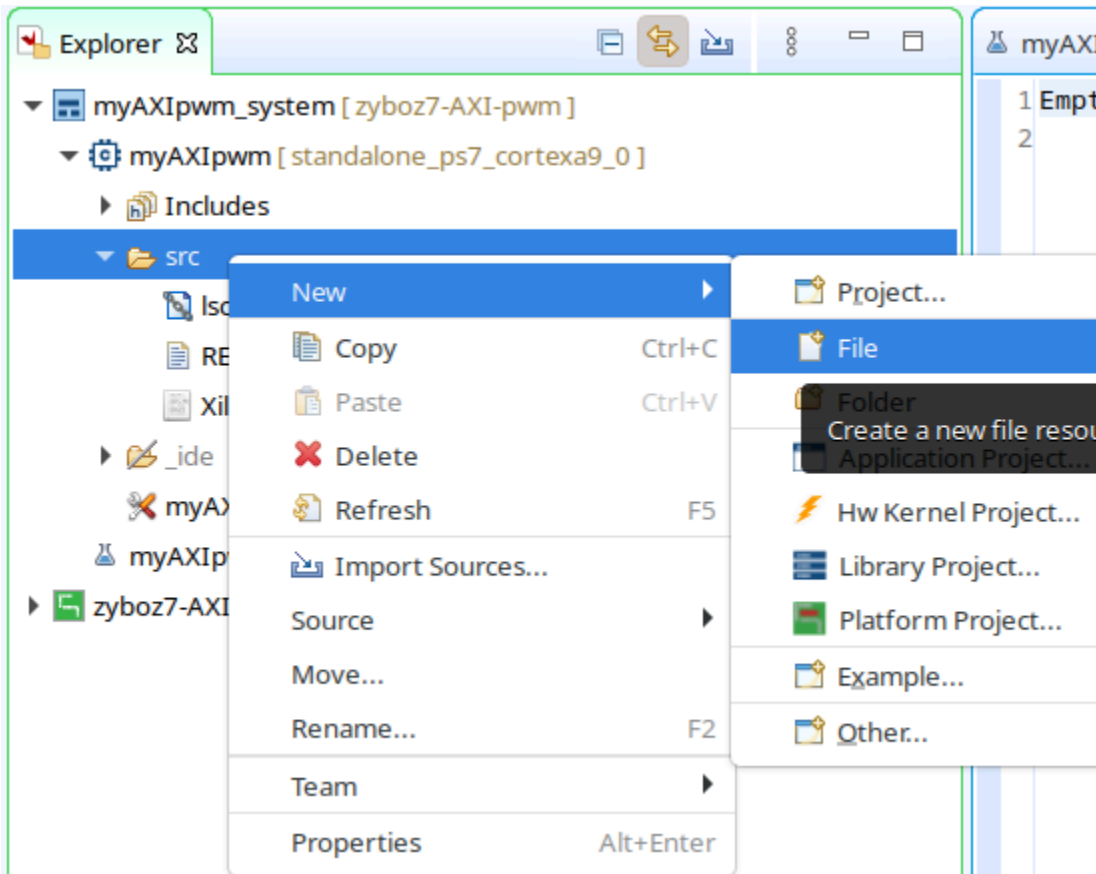
OpenAMP echo test

Empty Application (C++)

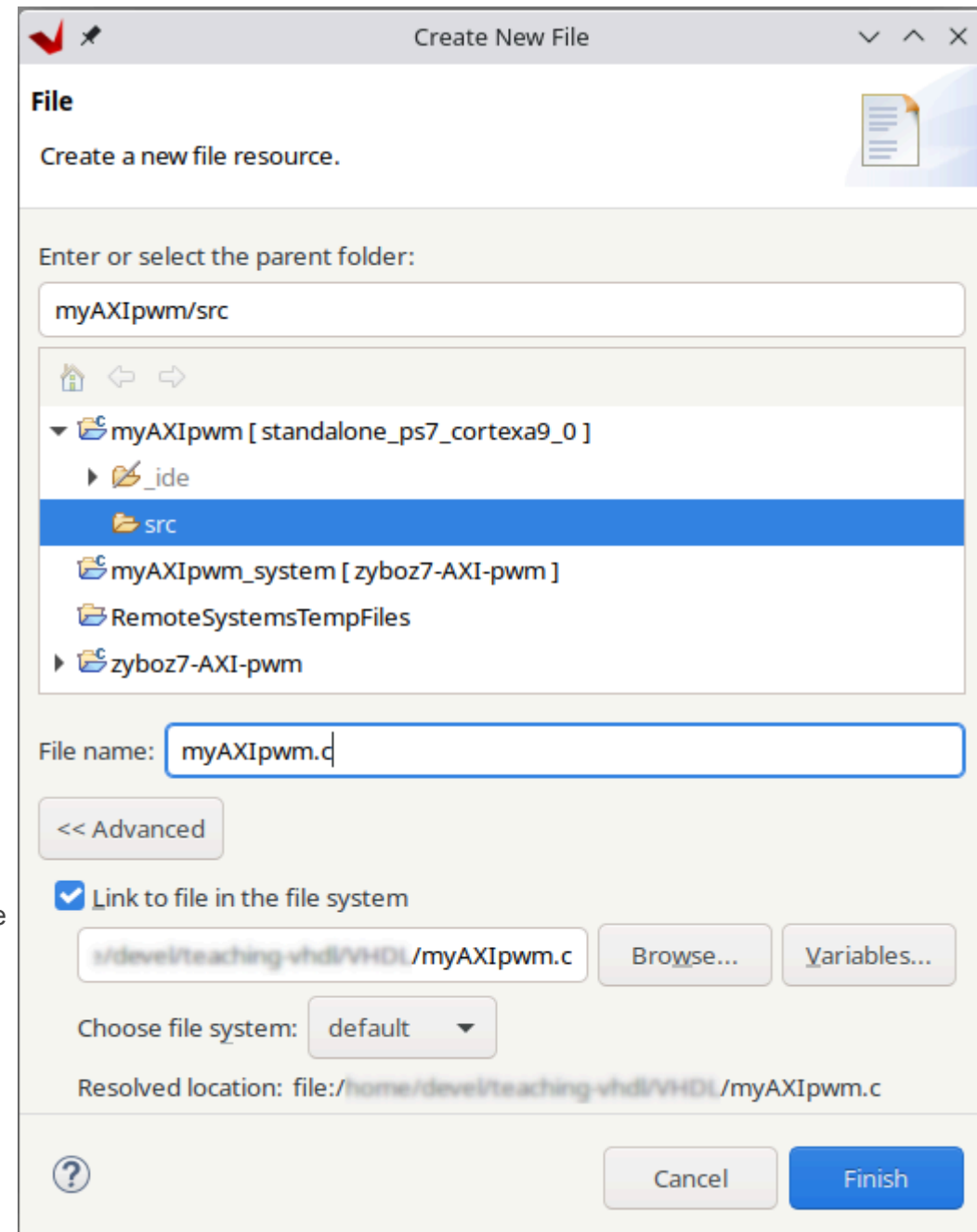
An empty C++ Application

< Back Next > Cancel Finish

... then right click on the `src` folder to add a new file ...

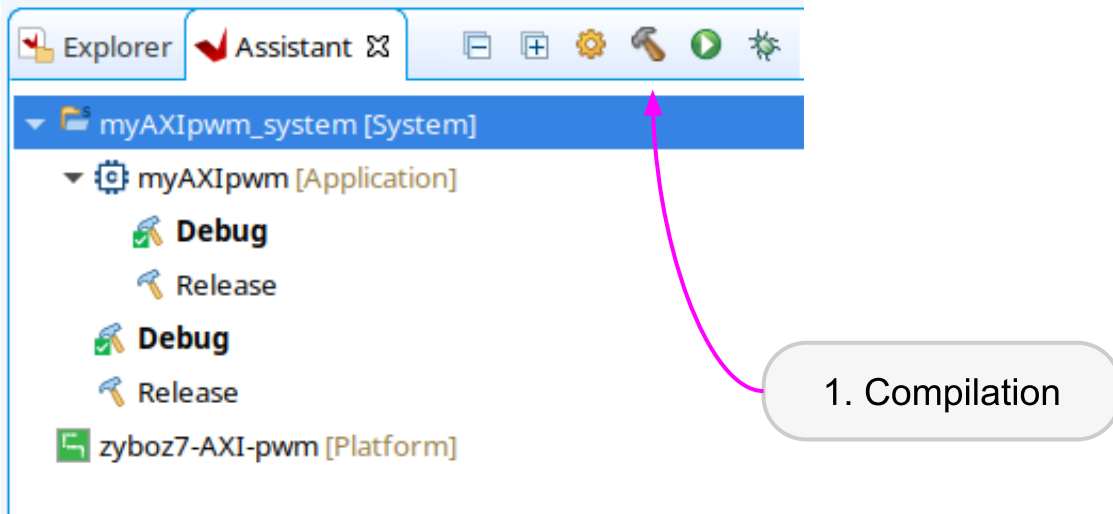


... click on **Advanced** to link to the existing `myAXIpwm.c` file you'll find from the [Practical exercises base files](#)



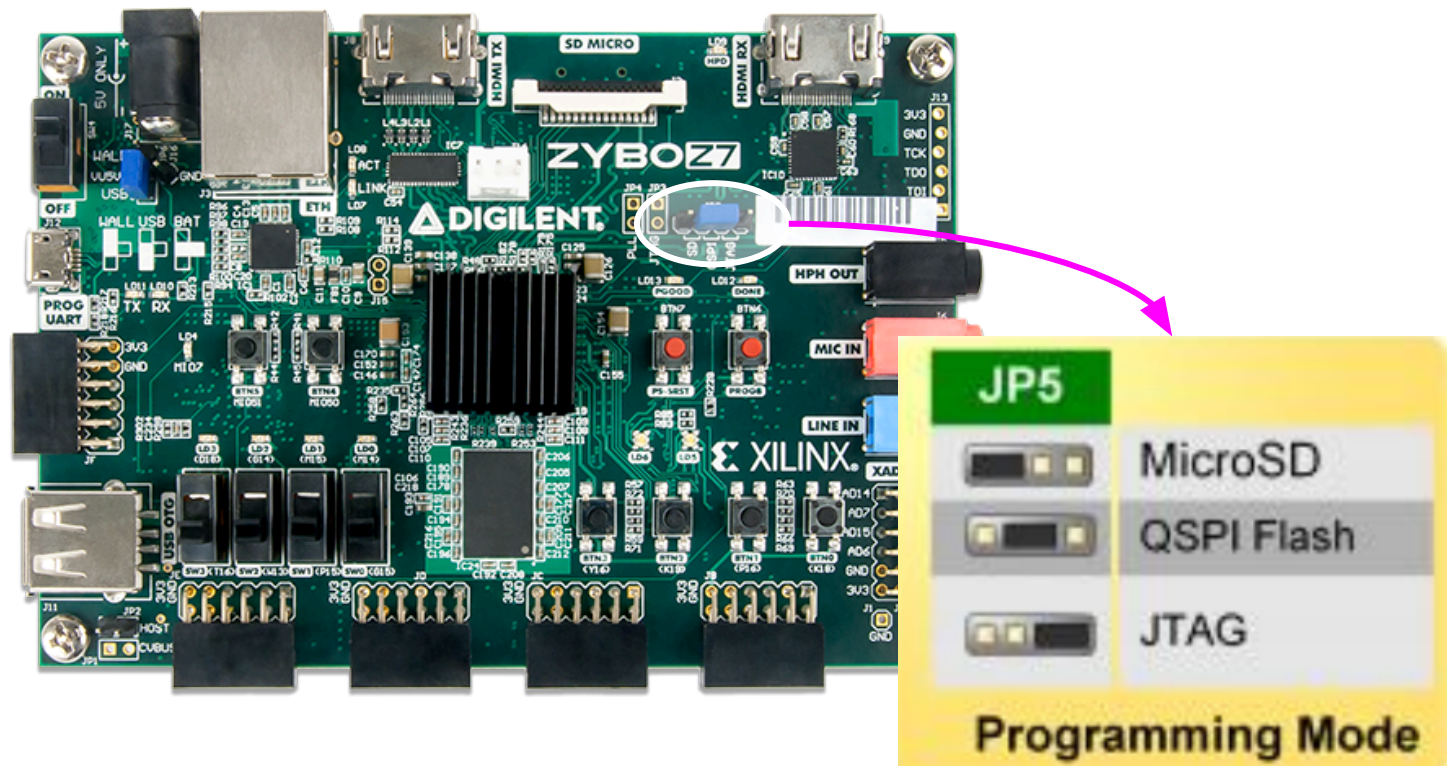
Compilation

Select 'myAXIpwm_system' (it's important to select the system level), then click on build to launch the compilation ... you may need to undertake some corrections within your C++ source code that will interact with your IP block.

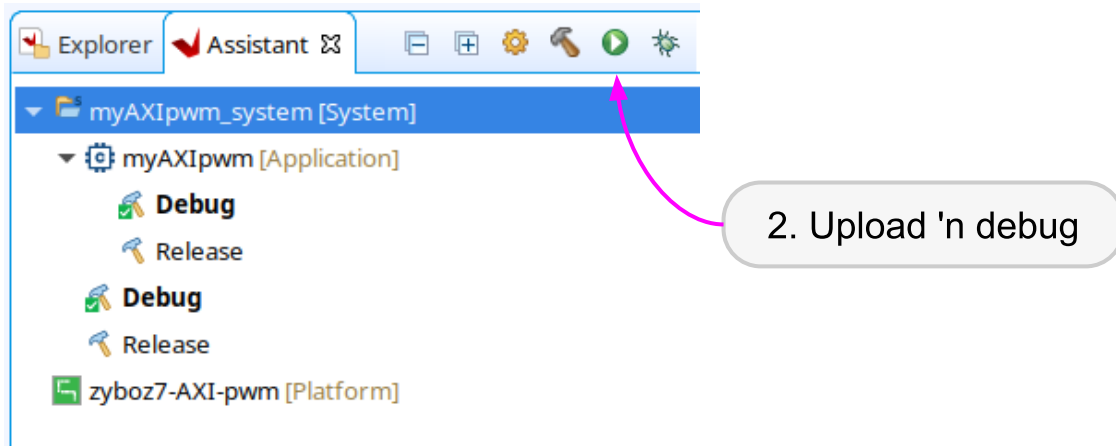


Zybo board | JTAG mode

Before testing your whole project on your hardware, set **JP5** to **JTAG** mode before powering on the Zybo board.



... then upload both FPGA configuration along with your standalone app. by clicking on the RUN icon within the assistant frame



You can follow the whole upload within the XSCT console

XSCT console

```
attempting to launch hw_server

***** Xilinx hw_server v2022.1.0
**** Build date : Apr 10 2022 at 06:24:21
** Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.

INFO: hw_server application started
INFO: Use Ctrl-C to exit hw_server application

INFO: To connect to this hw_server instance use url: TCP:127.0.0.1:3121

initializing
 0%    0MB    0.0MB/s  ????:? ETA
28%    1MB    2.2MB/s  ????:? ETA
50%    1MB    1.9MB/s  ????:? ETA
72%    2MB    1.8MB/s  ????:? ETA
93%    3MB    1.8MB/s  ????:? ETA
100%   3MB    1.8MB/s  00:02

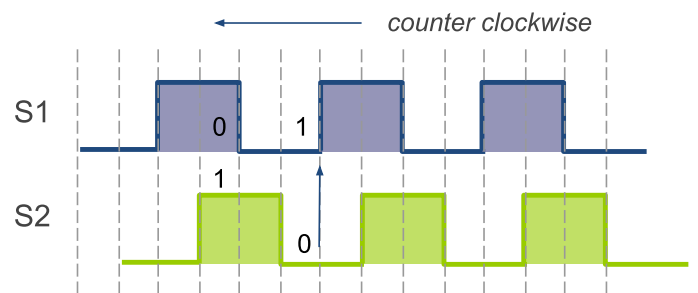
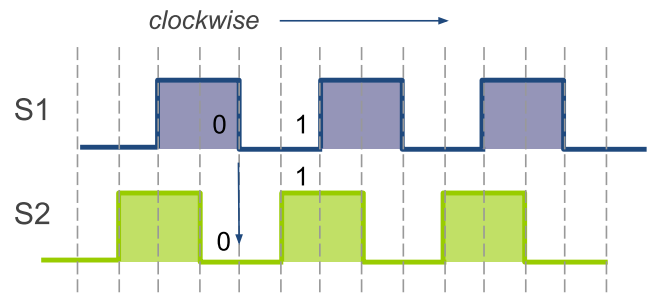
Downloading Program -- /home/devel/workspace/myAXIpwm/Debug/myAXIpwm.elf
  section, .text: 0x00100000 - 0x001035db
  section, .init: 0x001035dc - 0x001035e7
  section, .fini: 0x001035e8 - 0x001035f3
  section, .rodata: 0x001035f4 - 0x00103798
  section, .data: 0x001037a0 - 0x0010401f
  section, .eh_frame: 0x00104020 - 0x00104023
  section, .mmu_tbl: 0x00108000 - 0x0010bfff
  section, .ARM.exidx: 0x0010c000 - 0x0010c0c7
  section, .init_array: 0x0010c0c8 - 0x0010c0cb
  section, .fini_array: 0x0010c0cc - 0x0010c0cf
  section, .bss: 0x0010c0d0 - 0x0010c133
  section, .heap: 0x0010c134 - 0x0010e13f
  section, .stack: 0x0010e140 - 0x0011193f

 0%    0MB    0.0MB/s  ????:? ETA
100%   0MB    0.5MB/s  00:00
Setting PC to Program Start Address 0x00100000
Successfully downloaded /home/devel/workspace/myAXIpwm/Debug/myAXIpwm.elf
Info: ARM Cortex-A9 MPCore #0 (target 2) Stopped at 0xffffffff28 (Suspended)
xsct% Info: ARM Cortex-A9 MPCore #0 (target 2) Running
```

... if everything goes right, you now have a 1s blinking led 😊

[M2] AXI IP encoder

- IP rotary encoder
an endless rotary knob



TO BE CONTINUED

[M2] PicoRV32 @ Zybo

PicoRV32 <https://github.com/YosysHQ/picorv32>

TO BE CONTINUED

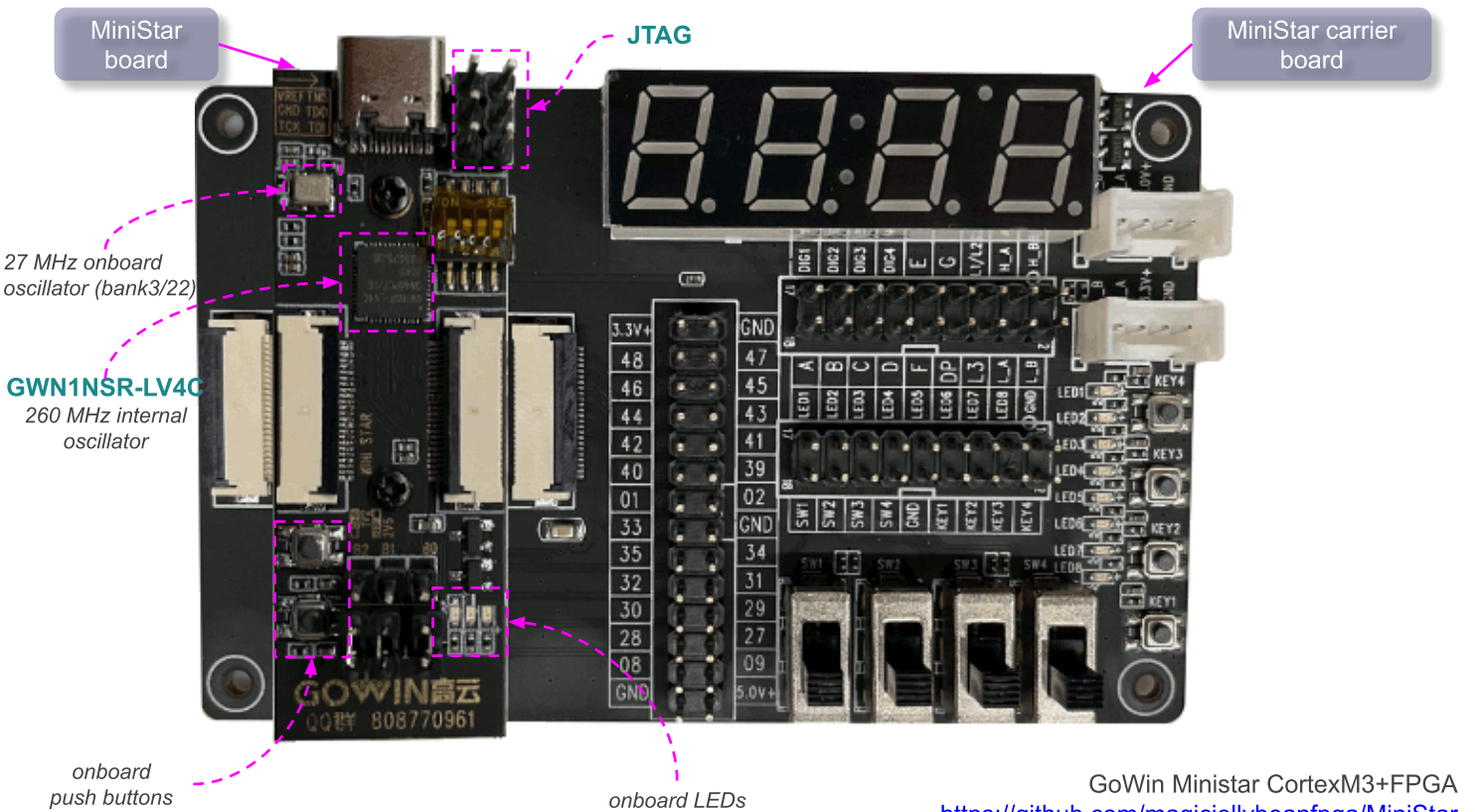
[M2] GoWIN "Hello World" SystemVerilog

The blinking of an LED is to hardware what "Hello World" is to software !

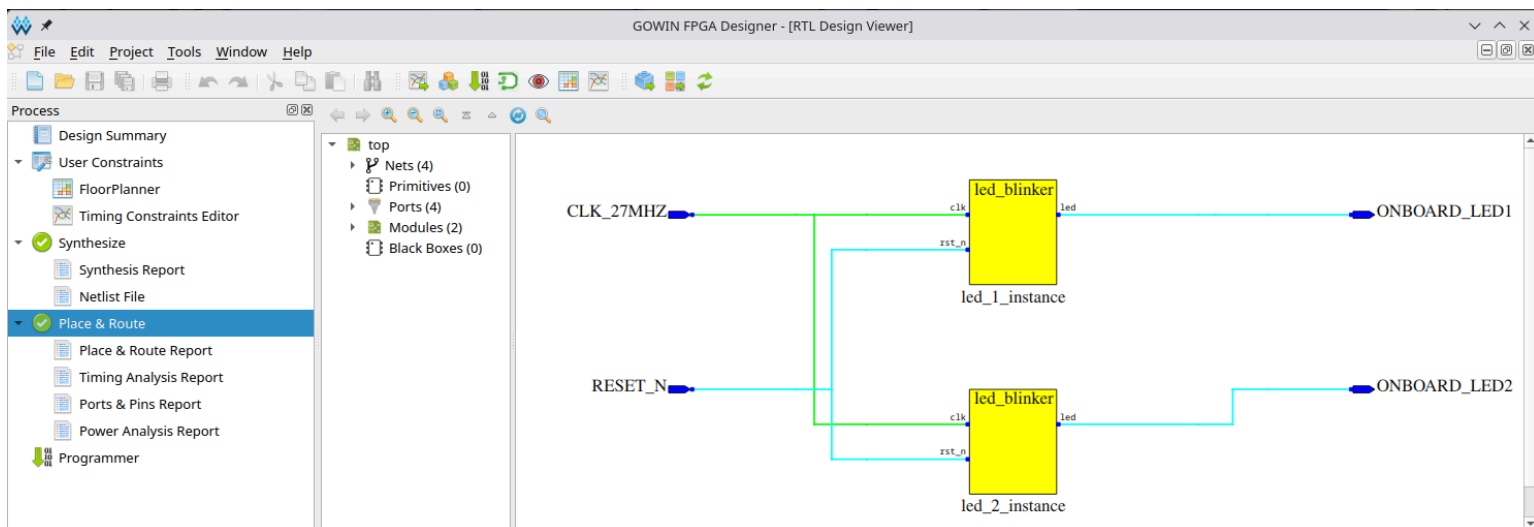
In this quick, hardware only tutorial, we'll see how to make use of available clocks to blink several leds. Especially, we'll develop a SystemVerilog module that will get instantiated for each of the LEDs we want to blink.

Required hardware

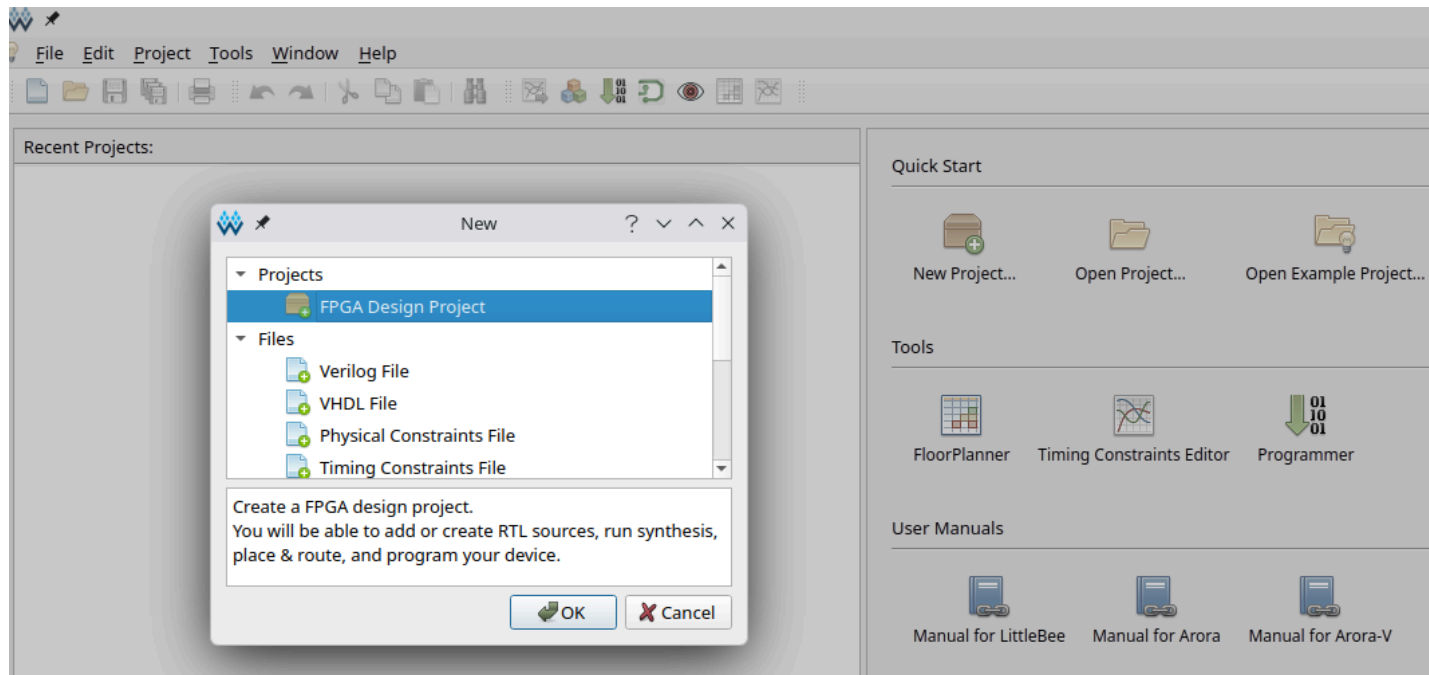
We'll make use of the GoWIN Ministar board (see [course](#) slide "GoWin Ministar dev. kit" ---114)



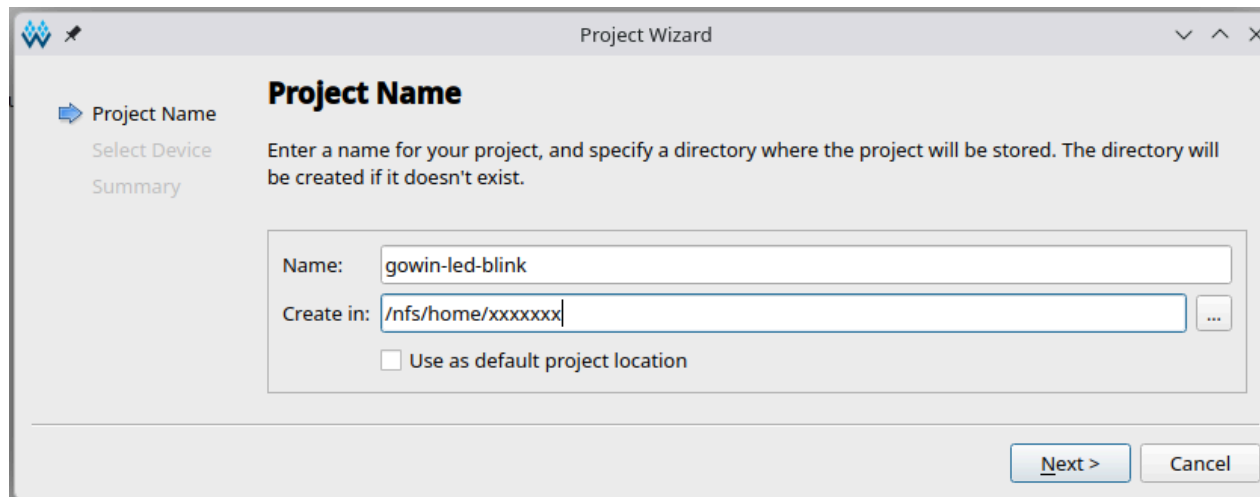
GoWin Ministar CortexM3+FPGA
<https://github.com/magiciellybeanfpga/MiniStar>



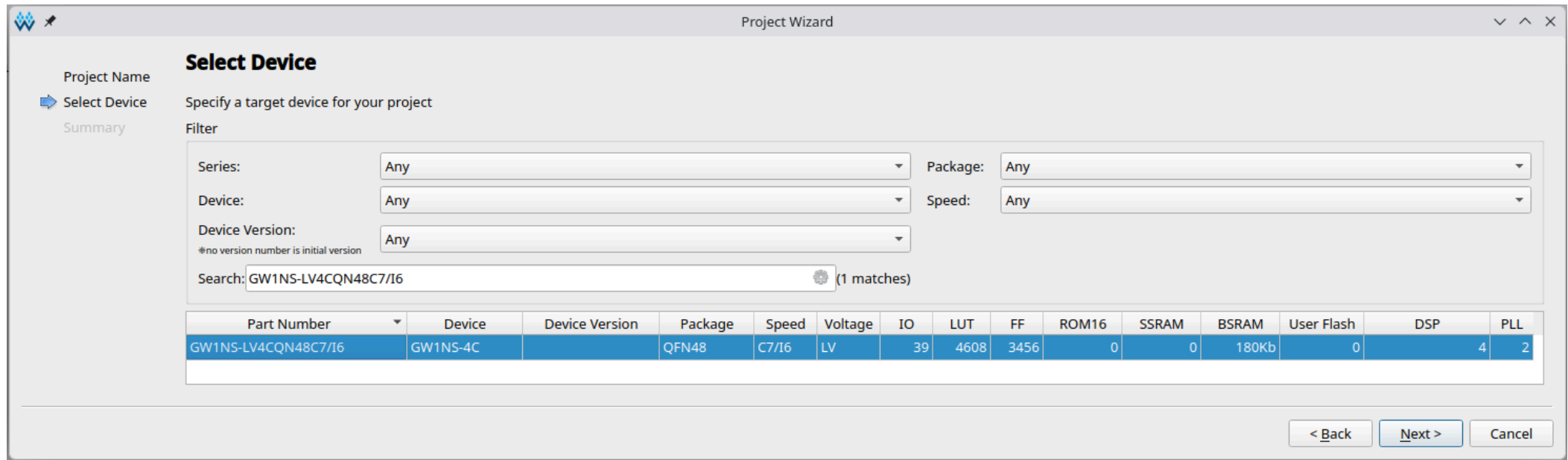
First, launch the GoWIN FPGA IDE 'gowin' ... then *File* → *New* → *FPGA Project Design*



Name and create the project



Search for the target device 'GW1NS-LV4CQN48C7/I6' (the one embedded in the Ministar board) and select it !



Select Device

Specify a target device for your project

Filter

Series: Any Package: Any

Device: Any Speed: Any

Device Version: Any

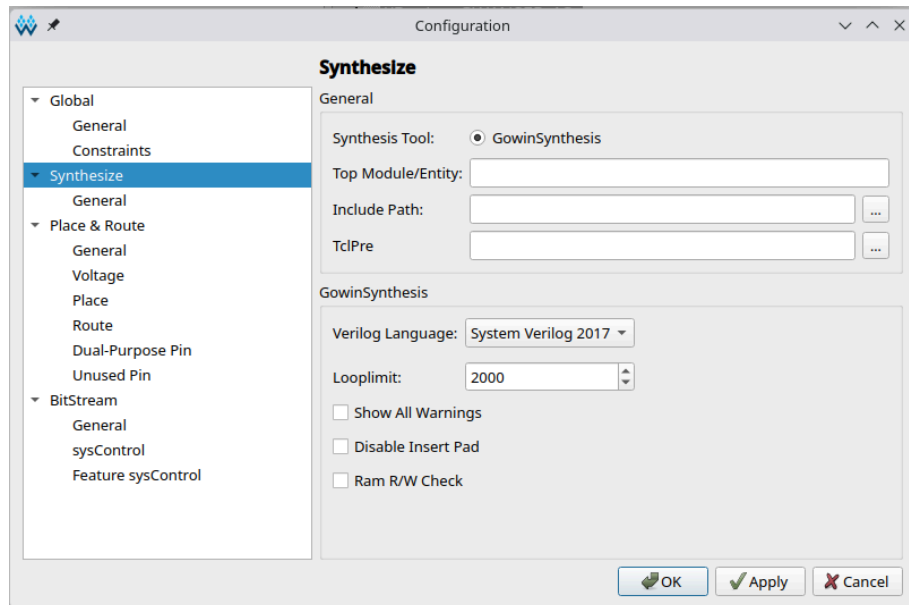
*no version number is initial version

Search: GW1NS-LV4CQN48C7/I6 (1 matches)

Part Number	Device	Device Version	Package	Speed	Voltage	IO	LUT	FF	ROM16	SSRAM	BSRAM	User Flash	DSP	PLL
GW1NS-LV4CQN48C7/I6	GW1NS-4C		QFN48	C7/I6	LV	39	4608	3456	0	0	180Kb	0	4	2

< Back Next > Cancel

... and DON'T FORGET to select SystemVerilog support for synthesis: *Project* → *Configuration*



Synthesize

General

Synthesis Tool: ☒ GowinSynthesis

Top Module/Entity:

Include Path:

TclPre:

GowinSynthesis

Verilog Language: System Verilog 2017

Looplimit: 2000

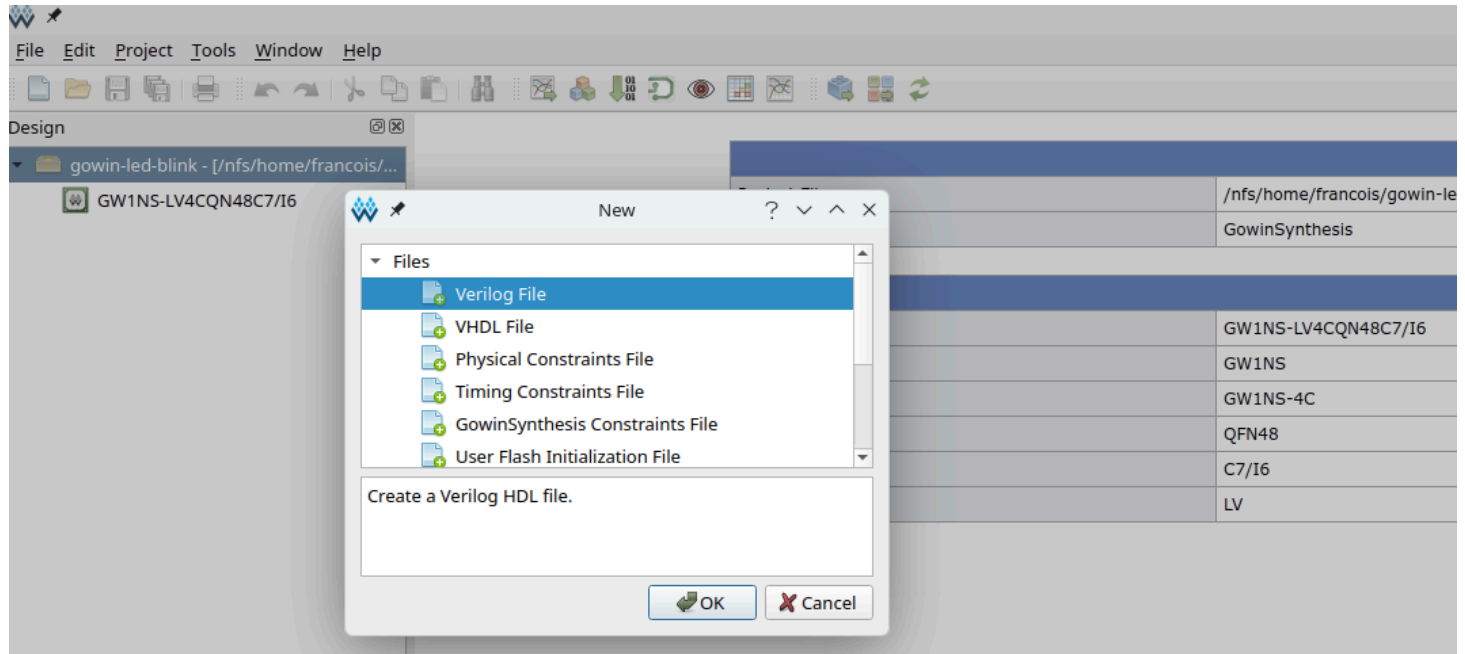
☐ Show All Warnings

☐ Disable Insert Pad

☐ Ram R/W Check

OK Apply Cancel

Okay, it's now time to add your first SystemVerilog file: Right click on the project to create a new Verilog file named 'led_blinker.sv'



Now code a parametrized Verilog module to blink a 1s led according to the specified input frequency

- led_blinker.sv

```

module led_blinker
#(
    parameter int FREQ_MHZ = 27,
    parameter int OUTPUT_HZ = 1,
    parameter bit INVERT = 0
)
(
    input clk, // Horloge principale
    input rst_n, // Signal de réinitialisation actif bas
    output led // Sortie vers la LED
);

// Fréquence de l'horloge en Hz
localparam FREQ_CLK = FREQ_MHZ * 1_000_000;

// Calcul du compteur pour une demi-période (0.5 seconde)
// Nombre de cycles nécessaires pour 0.5s: 27,000,000 * 0.5 = 13,500,000
// On compte de 0 à N-1, donc N-1 = 13,499,999
localparam COUNT_HALF_PERIOD = (FREQ_CLK/OUTPUT_HZ)/2 - 1;

// Déclaration du compteur (24 bits est nécessaire car 2^23 < 13.5M < 2^24)
logic [23:0] counter = 24'd0;

// Registre pour l'état de la LED
logic led_reg = (INVERT) ? 1'b0 : 1'b1;

// Logique séquentielle pour le compteur et la bascule de la LED
always_ff @(posedge clk) begin
    // Réinitialisation synchrone (active bas)
    if (!rst_n) begin
        counter <= 24'd0;
        // 1'b0 ==> LED éteinte
        led_reg <= (INVERT) ? 1'b0 : 1'b1;
    end
    // Opération normale
    else begin

```

```

        // Vérifie si le compteur a atteint la valeur maximale (fin de la demi-période)
        if (counter == COUNT_HALF_PERIOD) begin
            counter <= 24'd0; // Réinitialise le compteur
            led_reg <= ~led_reg; // Bascule l'état de la LED (0 -> 1 ou 1 -> 0)
        end else begin
            counter <= counter + 1'b1; // Incrémente le compteur
        end
    end
end

// Assignment de la sortie finale
assign led = led_reg;

endmodule

```

We're now about to instantiate this module in the top module itself → create a new Verilog file 'top.v'

- top.v

```

// Note: C++ comment style 😊
// All IO from top module will get mapped to pins

module top (
    input logic CLK_27MHZ, // main clock
    input logic RESET_N, // main reset
    output logic ONBOARD_LED1, // Sortie vers la première LED
    output logic ONBOARD_LED2 // Sortie vers la deuxième LED
);

// -----
// Instanciation 1: LED 12
// -----
    led_blinker #(
        .OUTPUT_HZ (3) // change from default HZ output
    ) led_1_instance (
        .clk (CLK_27MHZ), // Connecte l'horloge globale
        .rst_n (RESET_N), // Connecte le signal de réinitialisation
        .led (ONBOARD_LED1) // Connecter la sortie à la LED1
    );

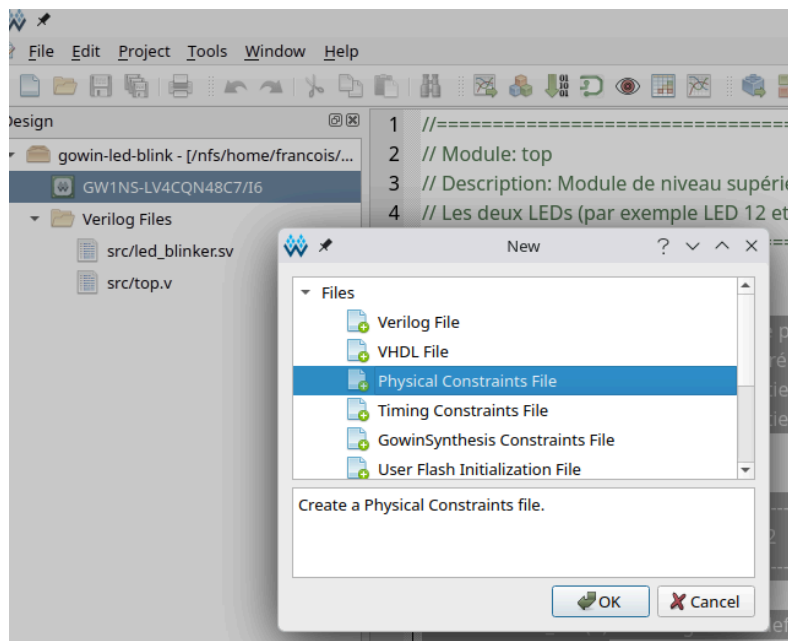
// -----
// Instanciation 2: Contrôle la LED 13
// -----
    led_blinker #(
        .INVERT (1) // Surcharge le paramètre INVERT pour inverser l'état de sortie
    ) led_2_instance (
        .clk (CLK_27MHZ), // Connecte l'horloge globale
        .rst_n (RESET_N), // Connecte le signal de réinitialisation
        .led (ONBOARD_LED2) // Connecter la sortie à la LED2
    );

endmodule

```

Physical Constraints

Now creating constraints file 'led_blinker.cst' ... this way you'll MAP top module IO to real life pins 😊



- led_blinker.cst

```
//File Title: Physical Constraints file
//GOWIN Version: 1.9.12
//Part Number: GW1NS-LV4CQN48C7/I6
//Device: GW1NSER-4C
//Ministar development board

// LED1 (Ministar onboard)
IO_LOC "ONBOARD_LED1" ??;
//IO_PORT "ONBOARD_LED1" IO_TYPE=LVCMS33 PULL_MODE=UP DRIVE=8;

// LED2 (Ministar onboard)
IO_LOC "ONBOARD_LED2" ??;
//IO_PORT "ONBOARD_LED2" IO_TYPE=LVCMS33 PULL_MODE=UP DRIVE=8;

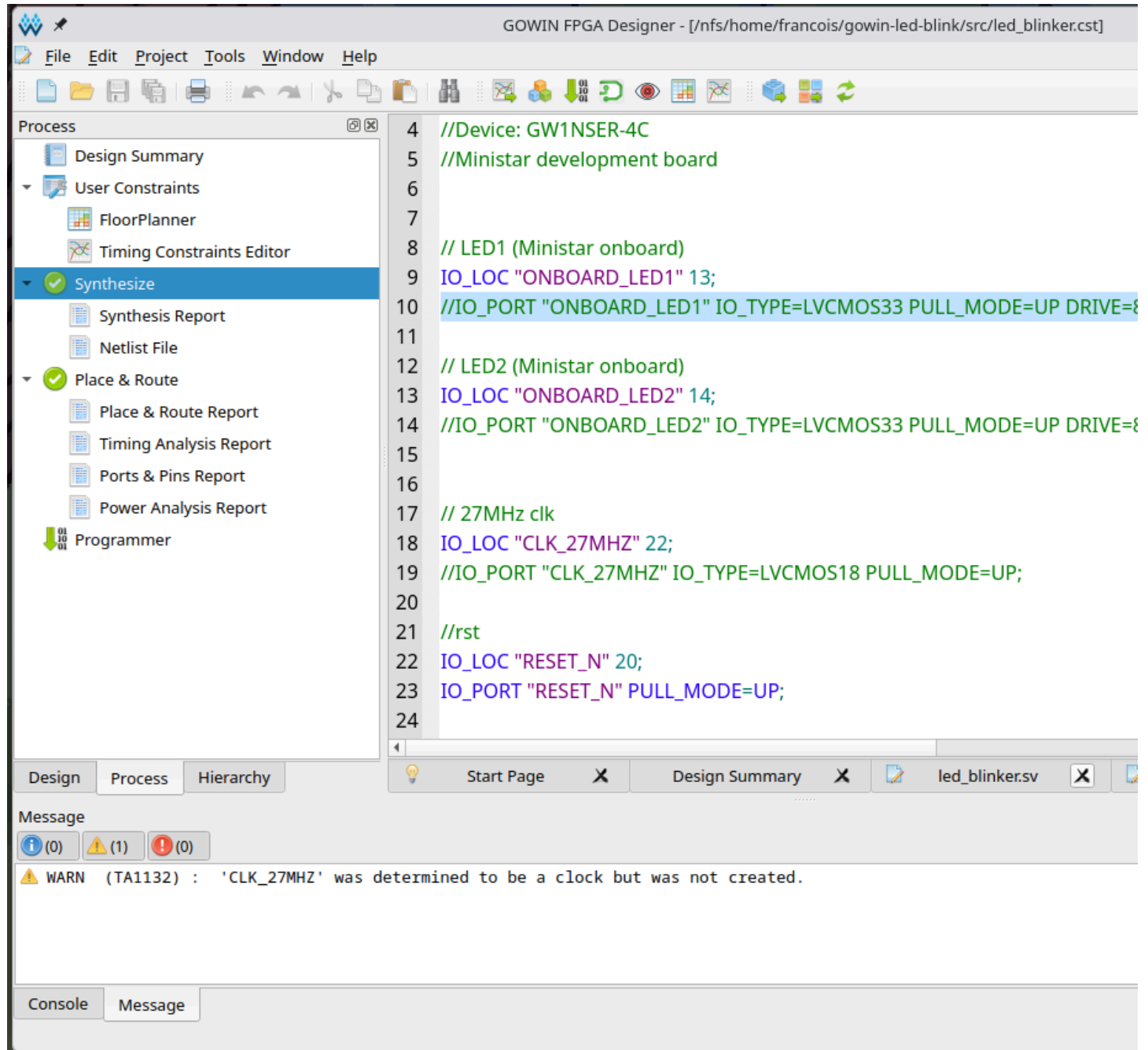
// 27MHz clk
IO_LOC "CLK_27MHZ" ??;
//IO_PORT "CLK_27MHZ" IO_TYPE=LVCMS18 PULL_MODE=UP;

//rst
IO_LOC "RESET_N" ??;
IO_PORT "RESET_N" PULL_MODE=UP;
```

Note: see Ministar schematic for exact pins numbers

Synthesis

Select 'Process' tab and dbl click on Synthesize then Place & Route



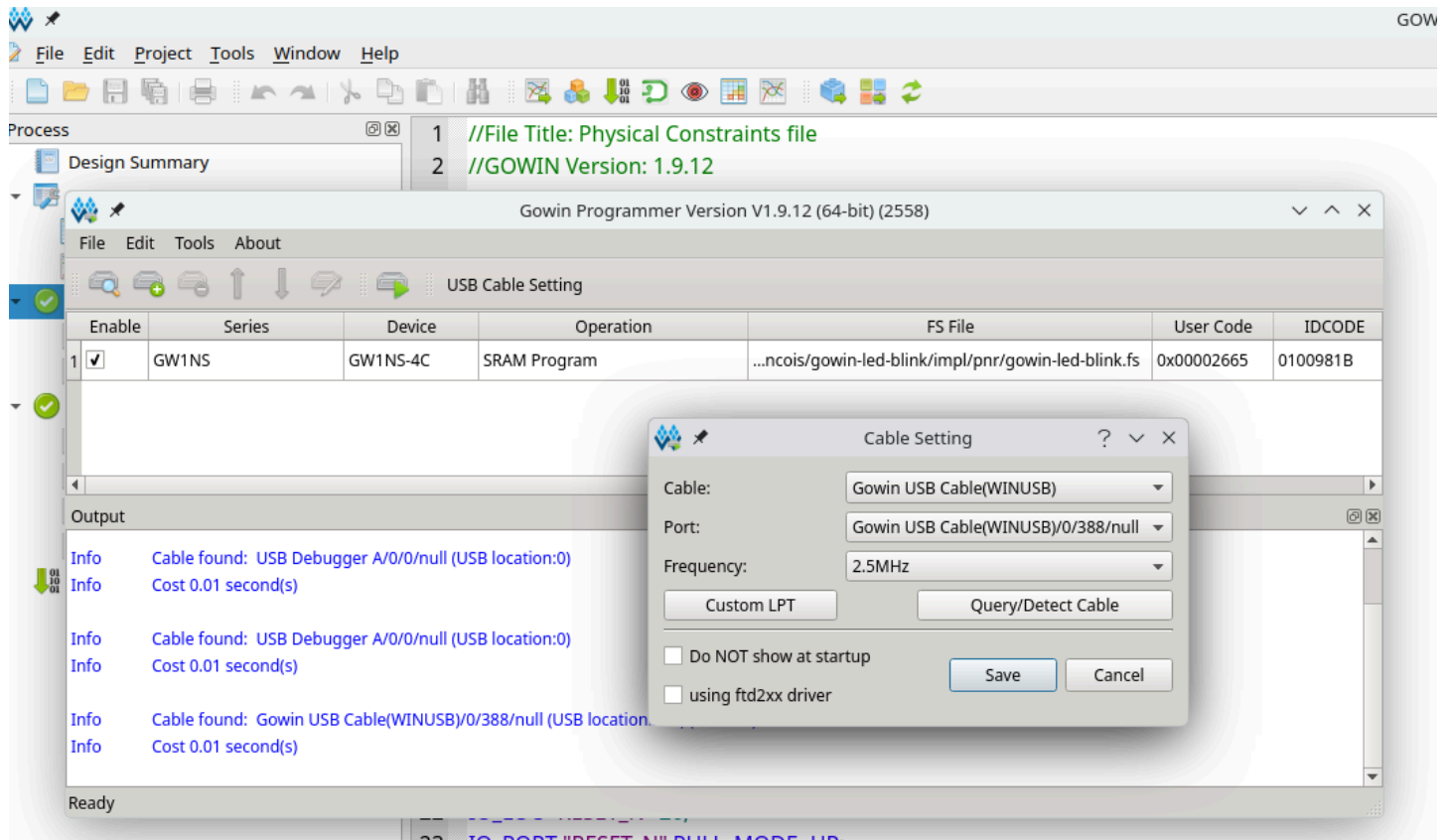
Programmer

If successful, you're ready to upload your hardware to the board:

- connect it by means of a USB cable
- click 'Programmer'

... a new window appears:

- accept *cable settings*
- click upload (green arrow)



[TODO] Hardware debugging

TO BE CONTINUED

Have a look to https://nand2mario.github.io/posts/2024/tang_tips/

|--> print.v ? serial over USB-C ??

experiment the '\$display' feature of verilog

find a way to grab messages from USB port to console → OpenOCD

[M2] GoWIN 7 segments display

This time, we'll make use of the same GoWIN Ministar board ([\[M2\] GoWIN "Hello World"](#)) but to drive a 4 digits 7 segment display (see [course](#) slide "[Lab] Ministar 7-segment" ---153)

Afterwards, this lab will get split in two versions:

1. GoWIN EMPU³ + GPIO → full software driving of the 7 segments leds by means of GPIO
2. GoWIN EMPU + [AXI/APB] IP module → hardware driven digits controlled by software

Required hardware

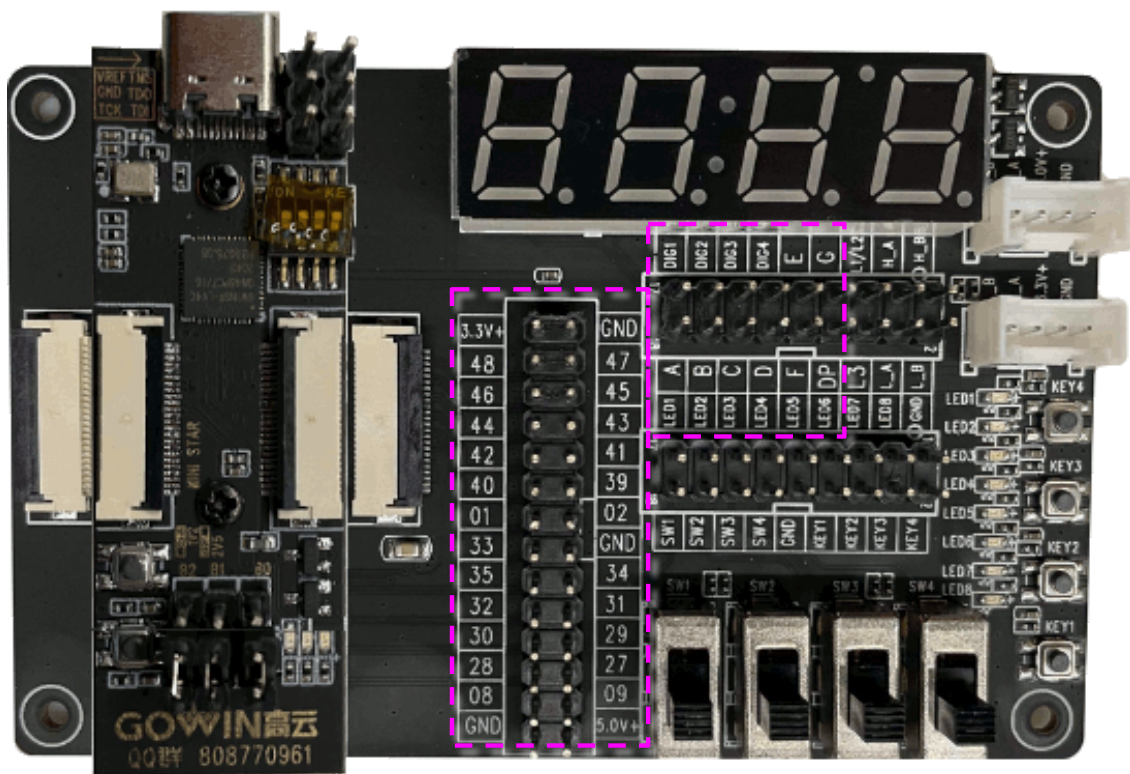
We'll make use of the GoWIN Ministar board (see [course](#) slide "GoWin Ministar dev. kit" ---114) with cabling to allow hardware access to the digits of the 7-seg. display:

- **[7 segments display]** Ministar dev. board | hardware setup

❖ IO cabling

7seg.	FPGA	GPIO*
a	30	[0]
b	32	[1]
c	35	[2]
d	33	[3]
e	42	[4]
f	44	[5]
g	46	[6]
dp	40	[7]
dig1	34	[11]
dig2	31	[10]
dig3	29	[9]
dig4	27	[8]

* software only version



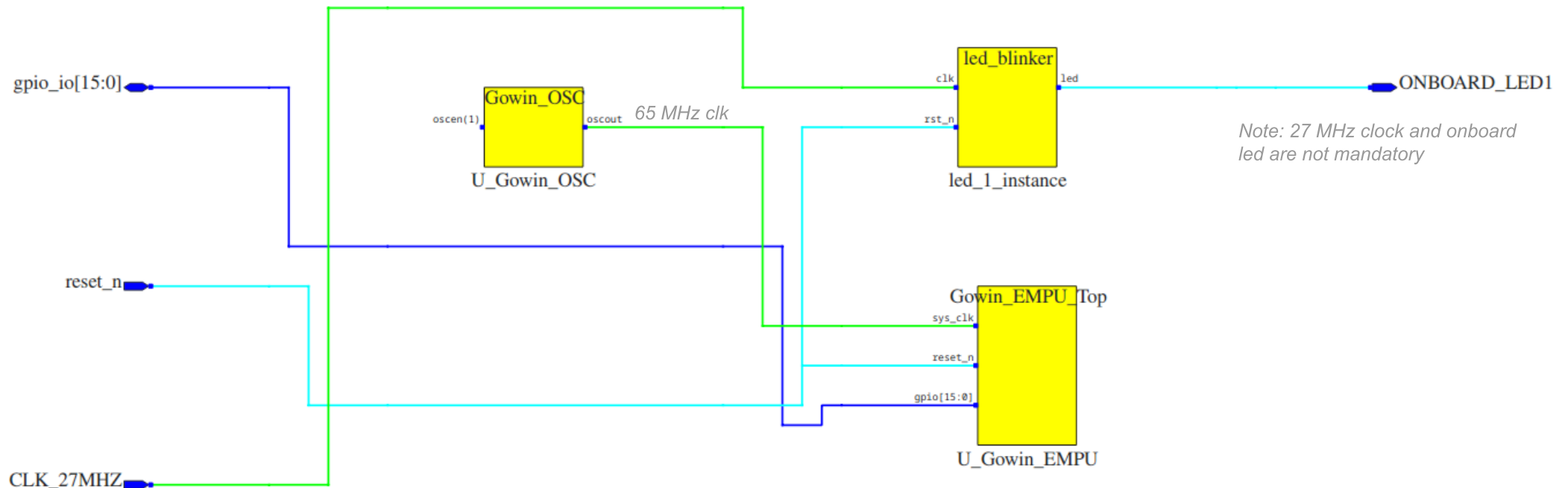
Connects FPGA pins and 7 seg. signals together according to the board

³ i.e Cortex M3

EMPU+GPIO | FPGA part

In this first part of the project, our EMPU (i.e Cortex-M3) will have its AHB-GPIO module activated to directly drive the 7 seg. digits through 16 GPIO lines. To achieve this, we'll setup the following:

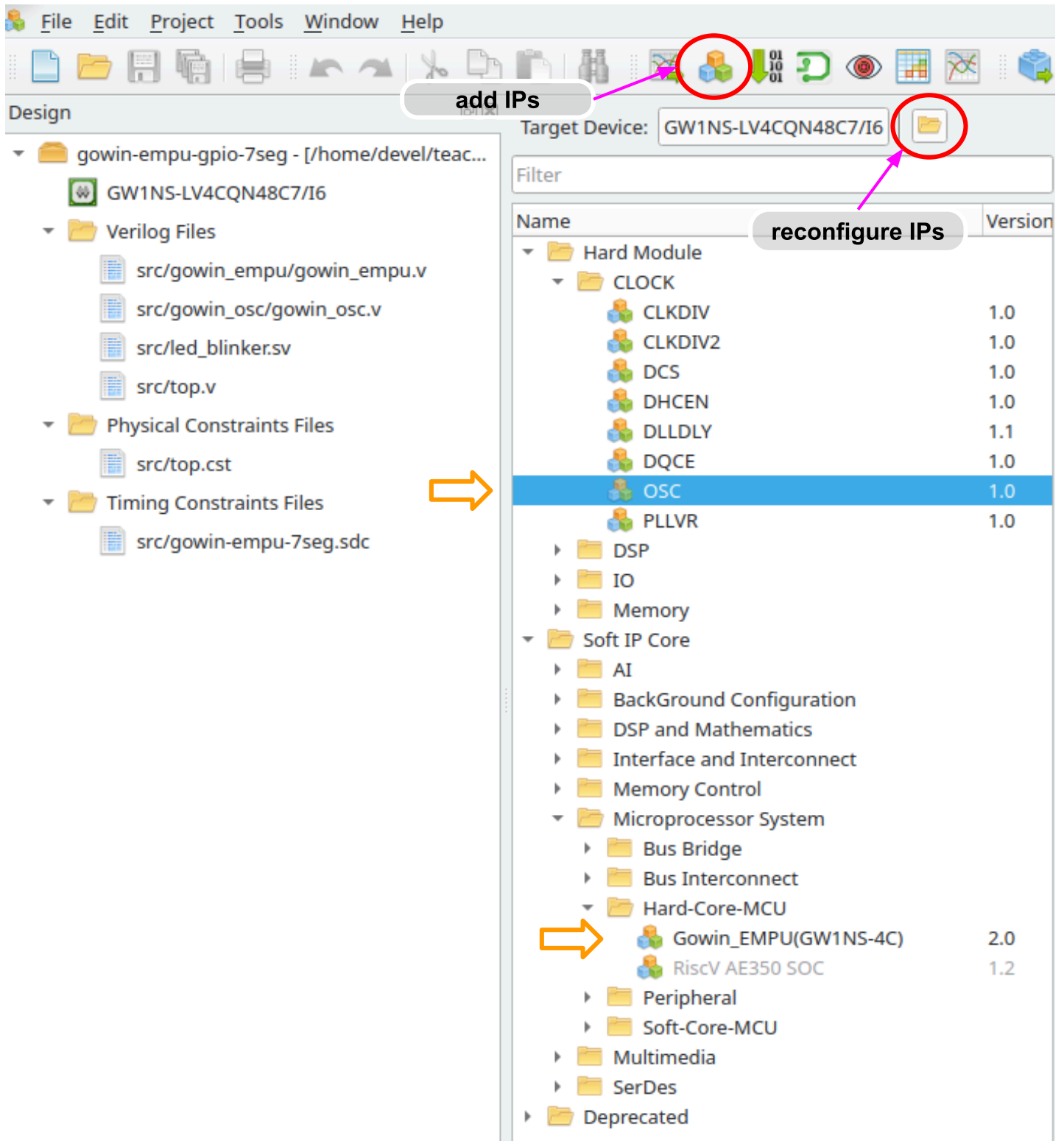
- Cortex M3 clocked at 65MHz
- Hard IP EMPU (Cortex-M3) with 4KB SRAM and GPIO module enabled



Okay, so as a first step, you'll create a project with the proper chip like what has been undertaken in the previous lab.

Then click to add IPs:

- ☐ OSC (i.e internal 260MHz clock) → configure dividing it by 4
- ☐ Hard MCU - Gowin EMPU → configure 4KB SRAM+ activate GPIO



Once you select and configure an IP, you'll have TWO generated files:

- ☐ a module file describing the component along with its interfaces at itself's end
- ☐ a temporary file describing what is expected within the top module → copy relevant part of the instantiation to your own top.v

- top.v

```
module top(
    inout [15:0] gpio_io,    // AHB GPIO module from EMPU
    input reset_n,
    input CLK_27MHZ,        // ext. hwclk 27 MHz
```

```

output ONBOARD_LED1    // Sortie vers la première LED
//output ONBOARD_LED2    // Sortie vers la deuxième LED
);

// declare global CLK signal
wire m3_clk;

// -----
// Instanciation 1: Contrôle la LED 12
// -----
led_blinker #(
    .OUTPUT_HZ (3)    // change from default HZ output
) led_1_instance (
    .clk (CLK_27MHZ), // Connecte l'horloge globale
    .rst_n (reset_n), // Connecte le signal de réinitialisation
    .led (ONBOARD_LED1) // Connecte la sortie à la LED1
);

// OSCillator instantiation
// Note: current FPGA internal oscillator is 260MHz
// IP configured to divide by 4 ==> sys_clk is 65MHz
Gowin_OSC U_Gowin_OSC (
    .oscout (m3_clk),
    .oscen (1'b1)
);

// Cortex M3 Hard IP instantiation
// - GPIO AHB device activated
Gowin_EMPU_Top U_Gowin_EMPU (
    .sys_clk (m3_clk), //input sys_clk
    .gpio (gpio_io), //inout [15:0] gpio
    .reset_n (reset_n) //input reset_n
);

endmodule

```

Note: of course the 'led_blinker' module is optional !

Physical Constraints

Now creating constraints file 'top.cst' ... this way you'll MAP top module IO to real life pins 😊

- top.cst

```

//File Title: Physical Constraints file
//GOWIN Version: 1.9.12
//Part Number: GW1NS-LV4CQN48C7/I6
//Device: GW1NSER-4C
//Ministar development board

// --- common -----

// LED1 (Ministar onboard)
IO_LOC "ONBOARD_LED1" 13;
//IO_PORT "ONBOARD_LED1" IO_TYPE=LVCOS33 PULL_MODE=UP DRIVE=8;
/*
// LED2 (Ministar onboard)
IO_LOC "ONBOARD_LED2" 14;
//IO_PORT "ONBOARD_LED2" IO_TYPE=LVCOS33 PULL_MODE=UP DRIVE=8;
*/
// 27MHz clk
IO_LOC "CLK_27MHZ" 22;

```

```
//IO_PORT "CLK_27MHZ" IO_TYPE=LVCMS18 PULL_MODE=UP;

// rst --Ministar onboard push button 2
IO_LOC "reset_n" 20;
IO_PORT "reset_n" PULL_MODE=UP;

// --- project specific -----
//DIG1
IO_LOC "gpio_io[11]" 34;
IO_PORT "gpio_io[11]" IO_TYPE=LVCMS33 PULL_MODE=UP DRIVE=8;

//DIG2
IO_LOC "gpio_io[10]" 31;
IO_PORT "gpio_io[10]" IO_TYPE=LVCMS33 PULL_MODE=UP DRIVE=8;

//DIG3
IO_LOC "gpio_io[9]" 29;
IO_PORT "gpio_io[9]" IO_TYPE=LVCMS33 PULL_MODE=UP DRIVE=8;

//DIG4
IO_LOC "gpio_io[8]" 27;
IO_PORT "gpio_io[8]" IO_TYPE=LVCMS33 PULL_MODE=UP DRIVE=8;

//DP
IO_LOC "gpio_io[7]" 40;
IO_PORT "gpio_io[7]" IO_TYPE=LVCMS33 PULL_MODE=UP DRIVE=8;

//G
IO_LOC "gpio_io[6]" 46;
IO_PORT "gpio_io[6]" IO_TYPE=LVCMS33 PULL_MODE=UP DRIVE=8;

//F
IO_LOC "gpio_io[5]" 44;
IO_PORT "gpio_io[5]" IO_TYPE=LVCMS33 PULL_MODE=UP DRIVE=8;

//E
IO_LOC "gpio_io[4]" 42;
IO_PORT "gpio_io[4]" IO_TYPE=LVCMS33 PULL_MODE=UP DRIVE=8;

//D
IO_LOC "gpio_io[3]" 33;
IO_PORT "gpio_io[3]" IO_TYPE=LVCMS33 PULL_MODE=UP DRIVE=8;

//C
IO_LOC "gpio_io[2]" 35;
IO_PORT "gpio_io[2]" IO_TYPE=LVCMS33 PULL_MODE=UP DRIVE=8;

//B
IO_LOC "gpio_io[1]" 32;
IO_PORT "gpio_io[1]" IO_TYPE=LVCMS33 PULL_MODE=UP DRIVE=8;

//A
IO_LOC "gpio_io[0]" 30;
IO_PORT "gpio_io[0]" IO_TYPE=LVCMS33 PULL_MODE=UP DRIVE=8;
```

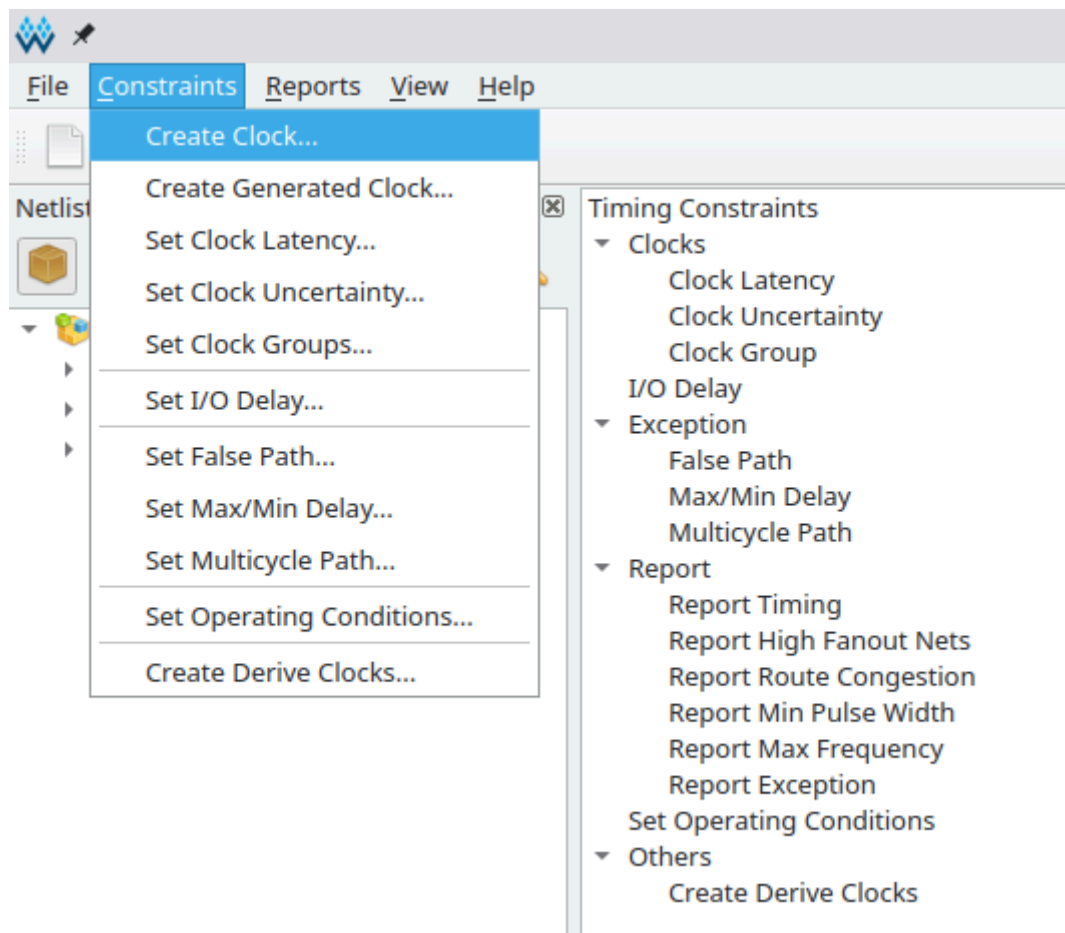
Timing Constraints

This time, we'll add a **definition** of the used clock. This will enable future timing constraints definitions.

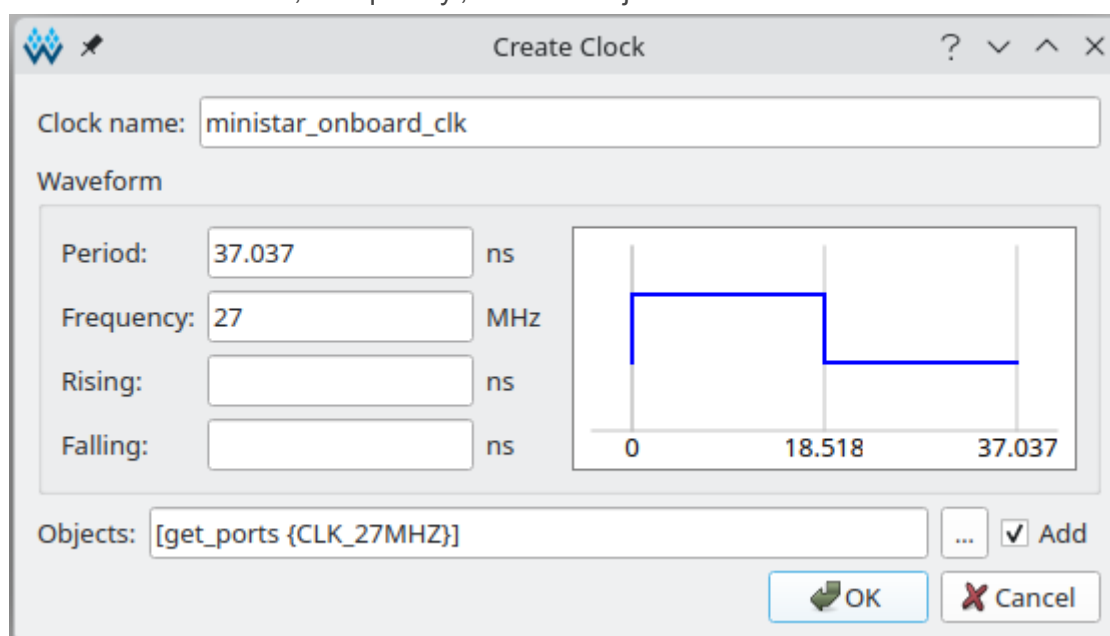
Define used clock

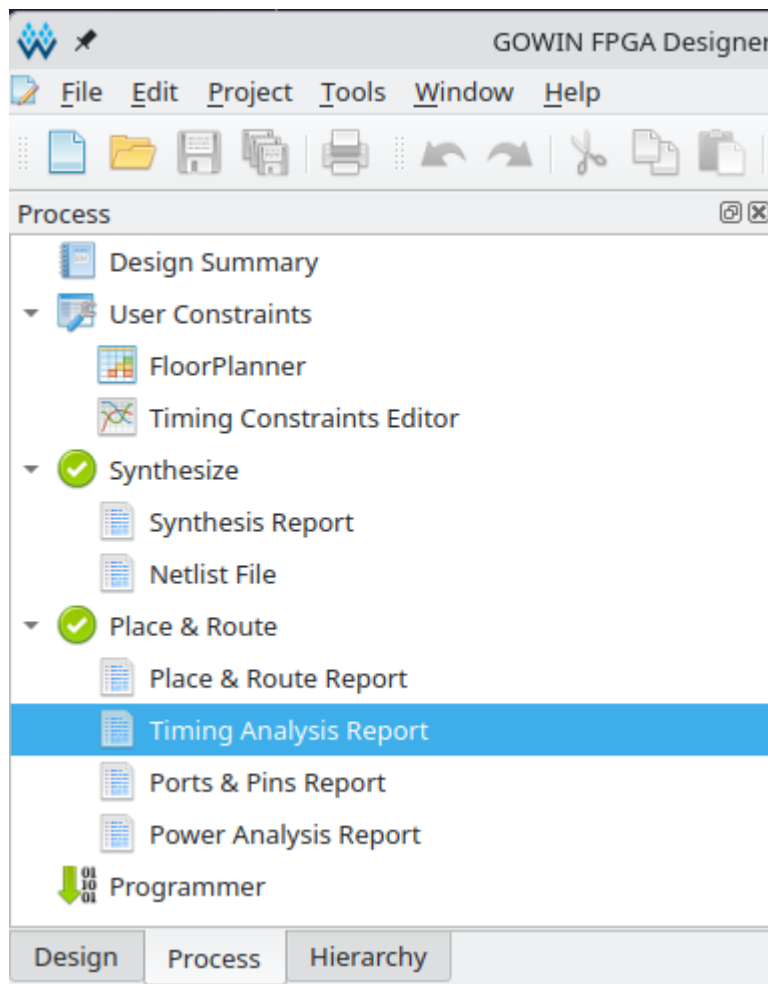
From the main project app., select *Tools* → *Timing Constraints Editor*

Then, from the Timing Constraints Editor app., select *Constraints* → *Create Clock*



Fill fields 'Clock name', 'Frequency', choose 'Objects' and tick 'Add'





In the Place & route → Timing Analysis Report, you should find, among others things, a summary of your used clocks.

Place 'n route | Clock Summary:

No	Clock Name	Type	Period	Freq(MHz)	Rise	Fall	Objects
1	ministar_onboard_clk	Base	37.037	27.000	0.000	18.518	CLK_27MHZ
2	U_Gowin_OSC/osc_inst/OSCOU T.default_clk	Base	15.385	65.000	0.000	7.692	U_Gowin_OSC/osc_inst/OSCOU

Programmer

Proceed as usual (see [Programmer](#)).

Remember, this is the 'FPGA' part ... let's continue with the MCU part

EMPU+GPIO | MCU part

It's now time to write your first application that will interact with the AHB GPIO module from the EMPU.

https://secil.univ-tlse3.fr/teaching/francois/UE-SSM/GoWIN/GoWIN_MCU-Designer_UserGuide_nov25.pdf

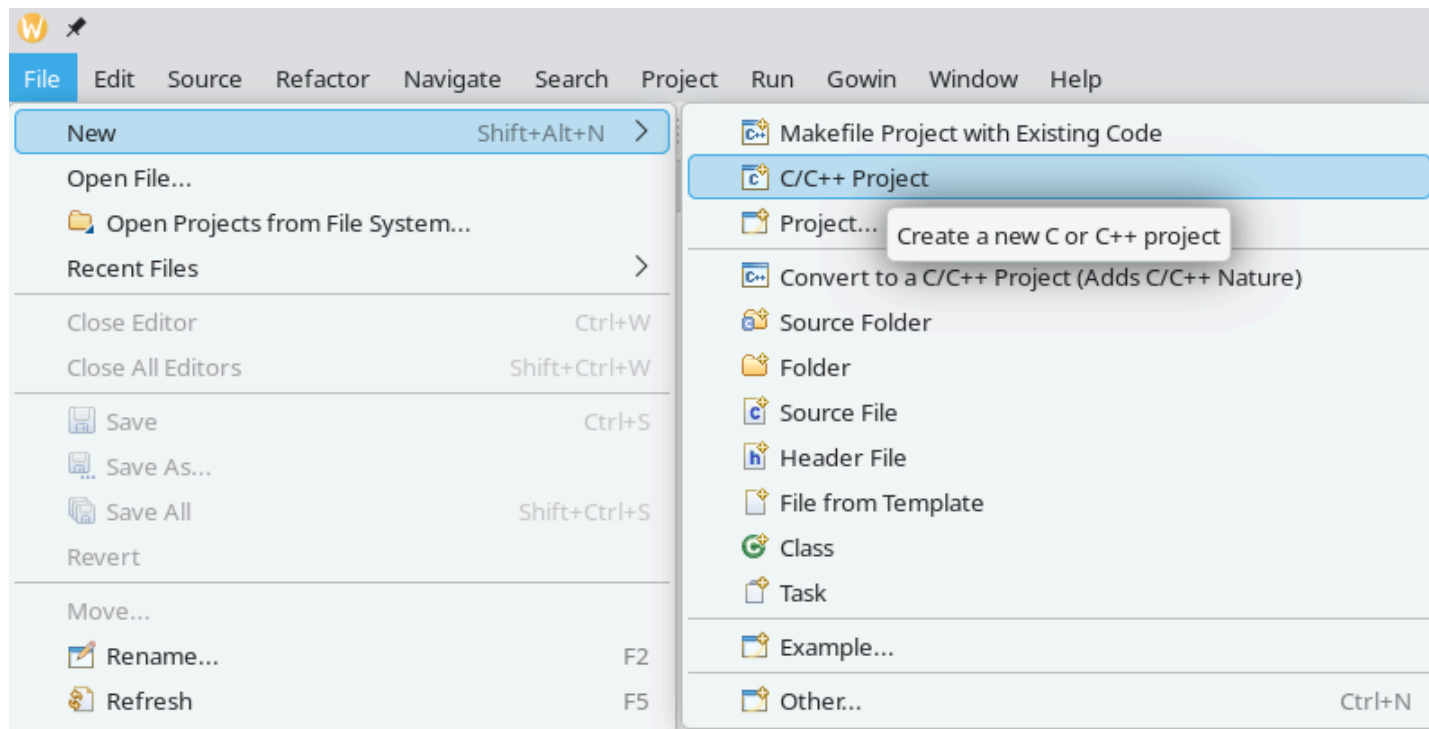
Launch GoWIN MCU IDE (2025.1 Linux version at the time of writing)

gowin_gmd

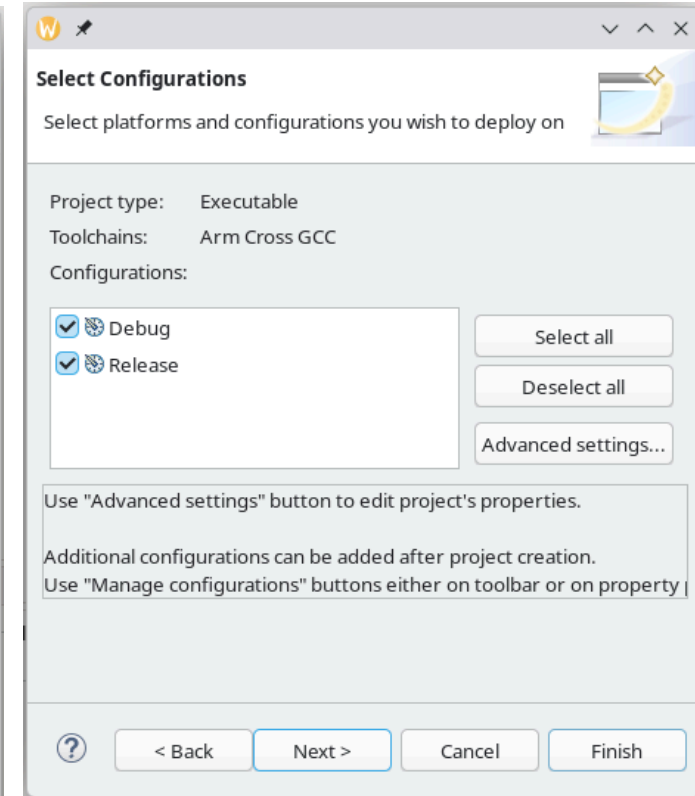
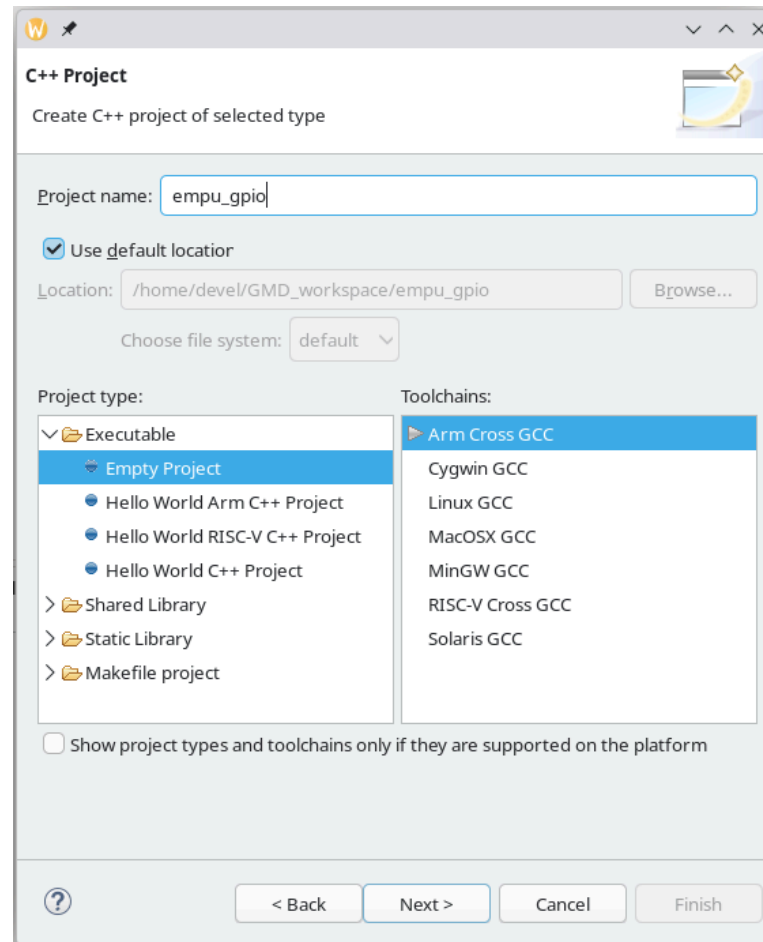
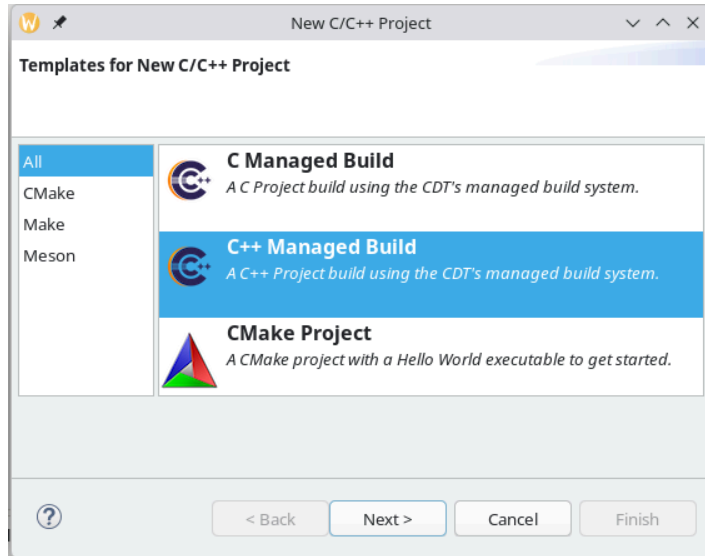
Note: set GoWIN GMD workspace as you want

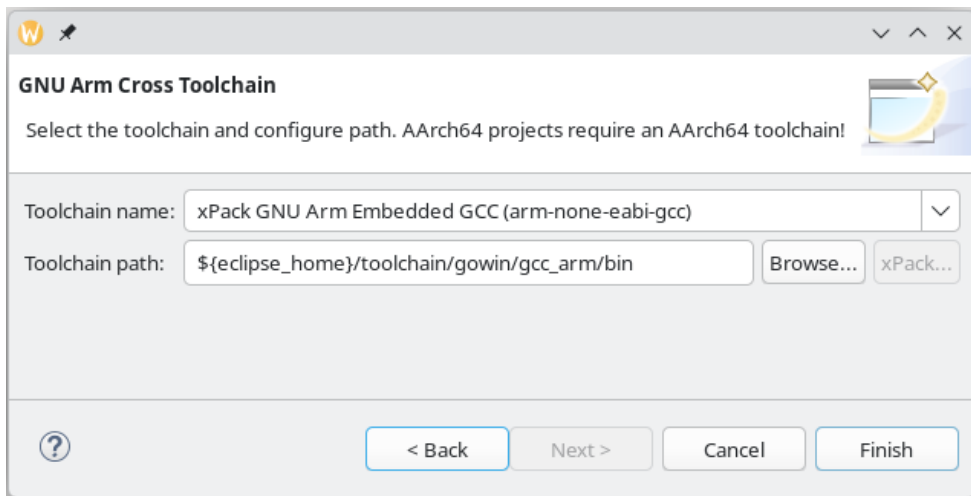
Create and configure MCU project

Then create a new C/C++ project: *File* → *New* → *C/C++ Project*



... select C++ Managed build ... Arm Cross GCC ... and name your project 'empu_gpio'





Ok, at this point, you're ready to start configuring your Cross-ARM toolchain ... however, since there are numerous traps in the process, we provide you the base files along with settings you'll import in your GMD_workspace as a side project (rename it as you want):

<https://secil.univ-tlse3.fr/teaching/francois/UE-SSM/GoWIN/gowin-empu-gpio-7seg.zip>

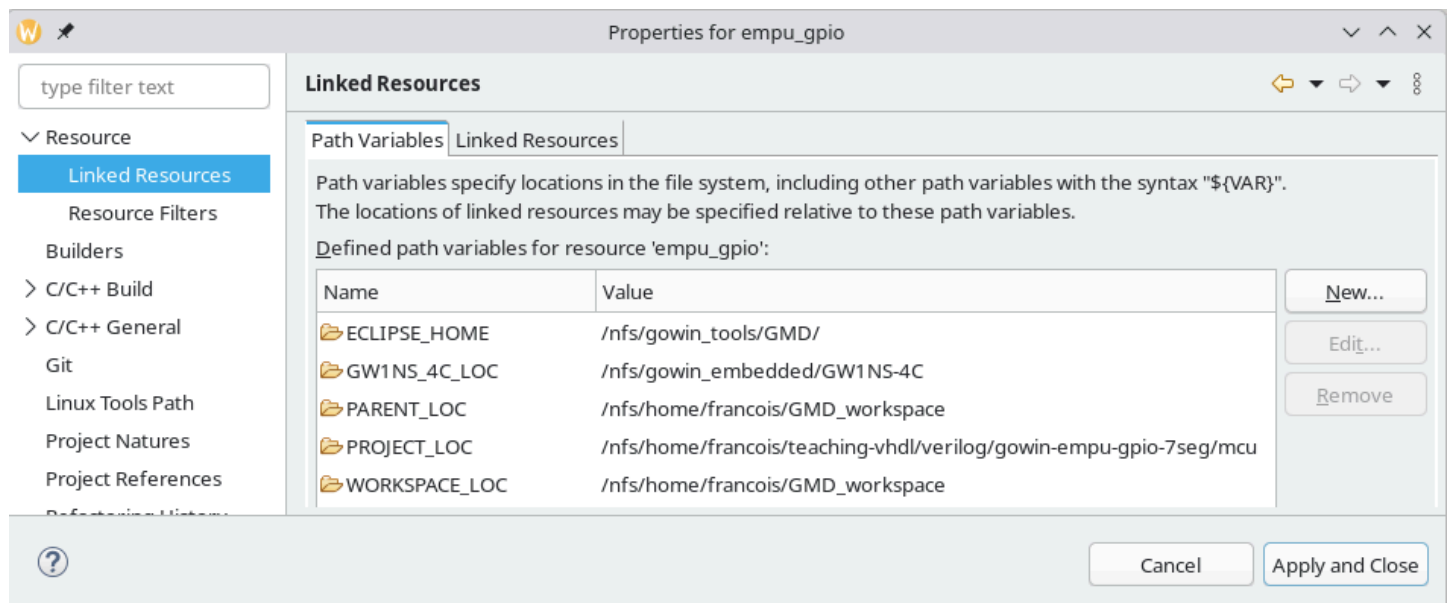
Add device specific libraries

We need to add the GW1NS-4C device libraries to our project.

Roughly speaking, you have two options:

- ☐ unzip the library in a subdirectory of your project
- ☐ take advantage of existing env. vars and the global availability of libraries (see /nfs/gowin_embedded)

First, add env. var. GW1NS_4C_LOC



Now we'll create a link to the GoWIN embedded library GW1NS-4C in your project:

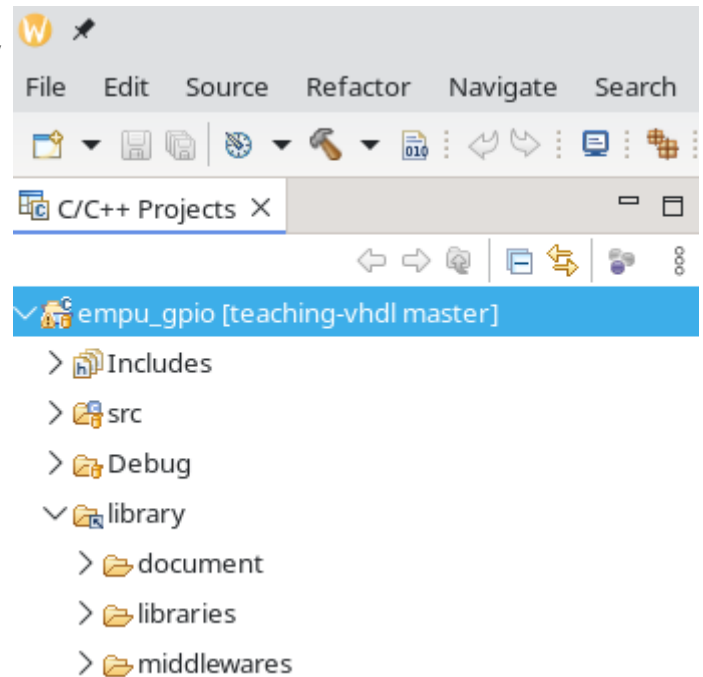
Right click in the name of your project,
then New → Source Folder

you'll try to take advantage of the already defined GW1NS_4C_LOC variable.

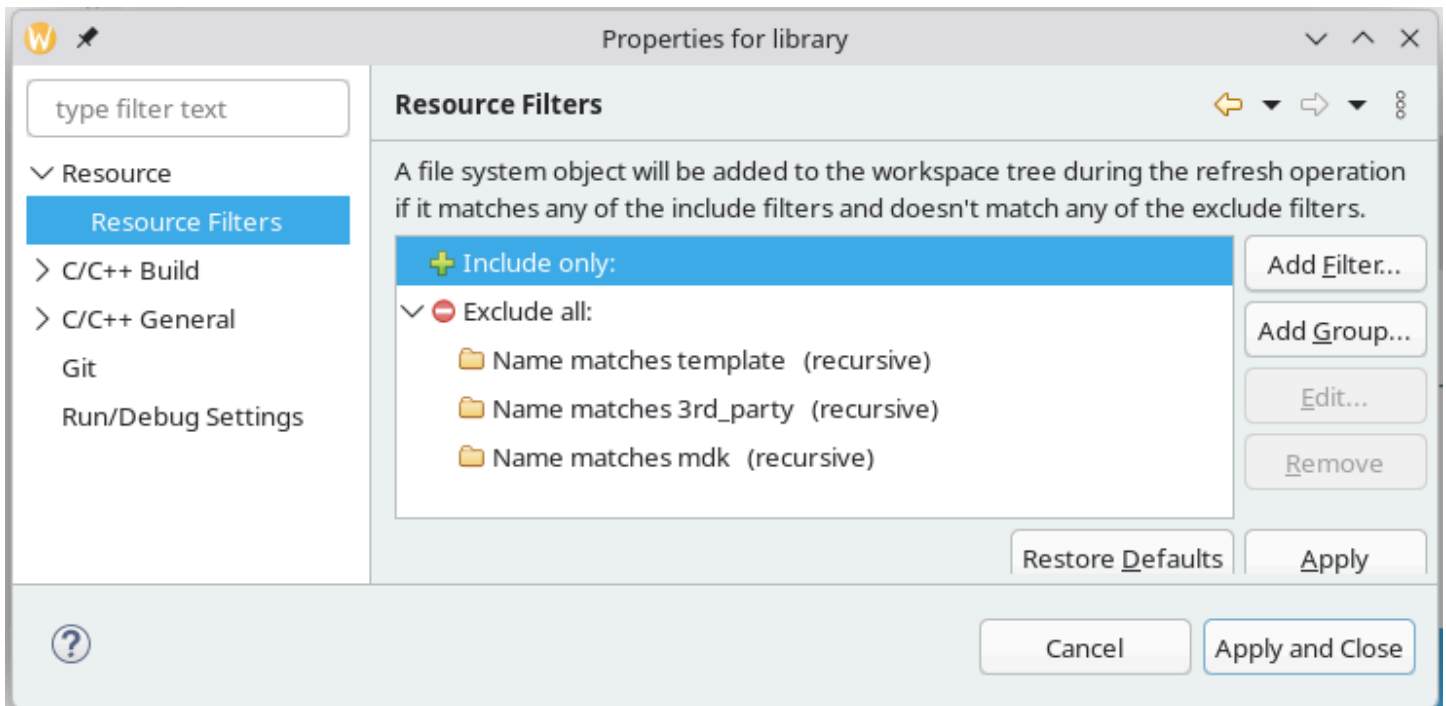
(link to /nfs/gowin_embedded/GW1NS_4C/library)

You should end-up with something like that:

(don't worry 'src' and 'Debug' folders will appear later)



Whatever the chosen solution, you'll need to undertake some cleaning/exclusion from the library:



Cross-ARM toolchain configuring

An important step where we'll do the following:

- [compiler] add headers directories to **both** ARM **gcc** and **g++** (see Include paths below)
- [linker] add linker script gw1ns4c_flash.ld
- [linker] add --defsym to enables you specifying FLASH and RAM size

Right click on the name of your project → *Properties* → *C/C++ Builds* → *settings*

Tool Settings | Toolchains | Devices | Container Settings | Build Steps | Build Artifact | Binary Parsers | Error Parsers

- Target Processor
- Optimization
- Warnings
- Debugging
- GNU Arm Cross Assembler
 - Preprocessor
 - Includes
 - Warnings
 - Miscellaneous
- GNU Arm Cross C Compiler
 - Preprocessor
 - Includes
 - Optimization
 - Warnings
 - Miscellaneous
- GNU Arm Cross C++ Compiler
 - Preprocessor
 - Includes
 - Optimization
 - Warnings
 - Miscellaneous
- GNU Arm Cross C++ Linker
 - General**
 - Libraries
 - Miscellaneous
- GNU Arm Cross Create Flash Image
 - General
- GNU Arm Cross Print Size
 - General

Script files (-T)

```
{GOWIN_EMPU_GW1NS_4C}/library/libraries/cmsis/cm3/device_support/startup/gmd/linker/gw1ns4c_flash.ld
```

☐ Do not use standard start files (-nostartfile:
☐ Do not use default libraries (-nodefaultlib:
☐ No startup or default libs (-nostdlib
☒ Remove unused sections (-Xlinker --gc-section:
☐ Print removed sections (-Xlinker --print-gc-section:

The screenshot displays the 'Tool Settings' window in STM32CubeIDE, specifically for the 'GNU Arm Cross C++ Linker' toolchain. The left sidebar shows a tree view of toolchain components, with 'GNU Arm Cross C++ Linker' expanded and 'Miscellaneous' selected. The main panel is divided into two sections: 'Linker flags (-Xlinker [option])' and 'Other objects'.

Linker flags (-Xlinker [option])

- defsym=RAM_LENGTH=0x1000
- defsym=FLASH_LENGTH=0x8000

Other objects

Generate map

- ☐ Cross reference (-Xlinker --cref)
- ☐ Print link map (-Xlinker --print-map)
- ☐ Use newlib-nano (--specs=nano.specs)
- ☐ Use float with nano printf (-u _printf_float)
- ☐ Use float with nano scanf (-u _scanf_float)
- ☐ Do not use syscalls (--specs=nosys.specs)
- ☐ Verbose (-v)

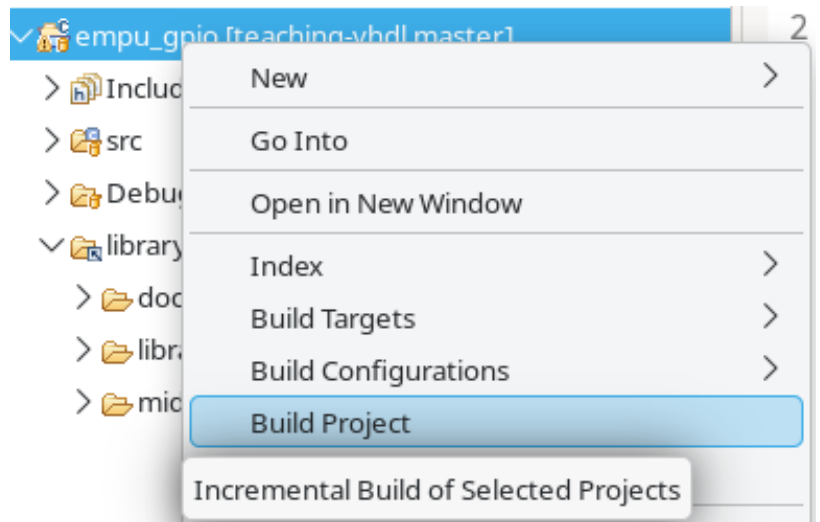
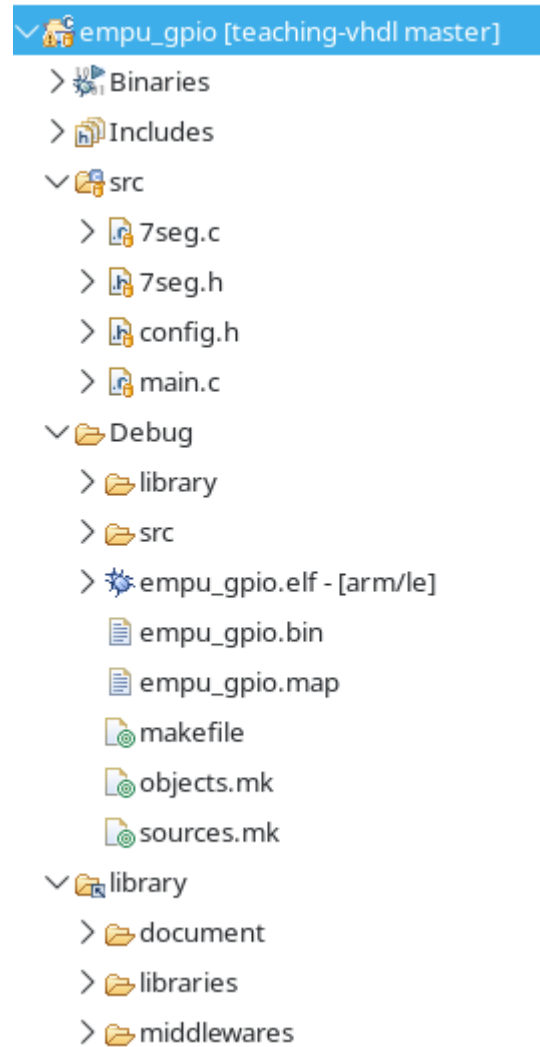
Other linker flags

Add source files

Finally, it's now time to start coding 😊

Add a new source file → `main.c`
(have a look to the already provided files)
... then *Build Project*

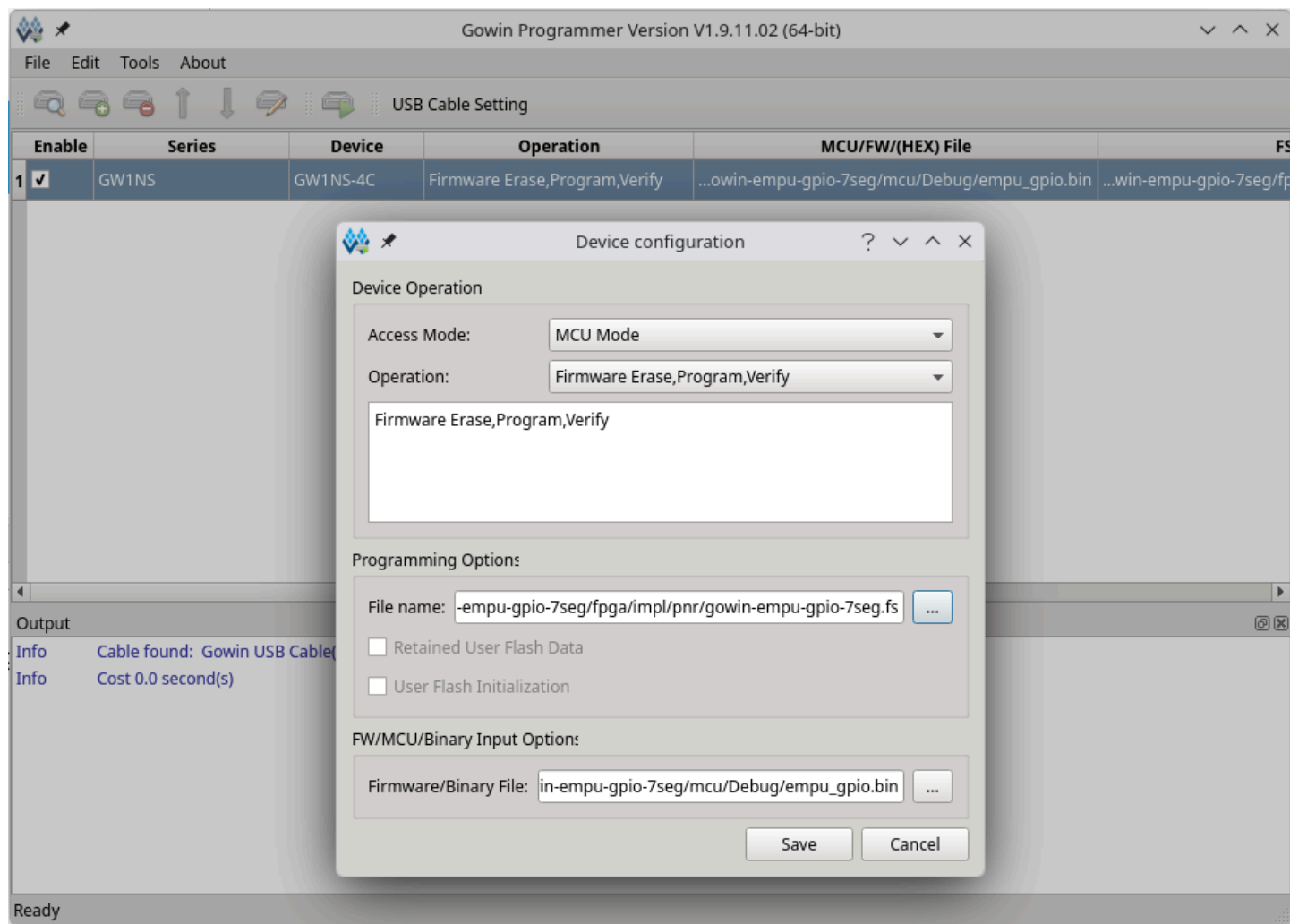
Generated `empu_gpio.bin` file



Next step is flashing both the FPGA and the MCU parts of our target device.

Programmer

Ok, this time, we'll program both the FPGA and the MCU parts. Click on upload icon, GoWIN programmer will open ... then click in the GW1NS blue line to change the operation mode to 'MCU Mode'



Once proper files are selected, press the upload icon ... and the digits of your board will start displaying things :)

Debugging

Ok, it does not work ... What's happening ??

We'll undertake debugging through JTAG embedded within the USB FTDI chip as the second serial link ...

TO BE CONTINUED

openocd

[M2] GoWIN 7 segments display / part2

So far so good, you reached part2 ... unfortunately, having been able to drive the 7 segment digits through EMPU's GPIO ... was the simplest part ;)

Now we want you to implement a hardware version of the 4 digits driver that will feature the following:

- ☐ AXI or APB bus interface to your custom Soft-IP
- ☐ Your IP will manage both the cycling of leds within a digit AND the cycling through the 4 digits*
- ☐ Refresh frequency of digits will be 50Hz, software configurable
- ☐ Ability to enable/disable each digit on an individual basis, software controlled,
- ☐ Ability to modulate leds intensity, software controlled
- ☐ EMPU will access registers of your IP to set the value to display

* at anytime, there will be only a single led ON

TO BE CONTINUED

END